

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2026 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
інформаційних систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Методи та засоби інтеграції системи Електронний кампус
з Moodle

Виконав студент IV курсу, групи ІП-21
(шифр групи)

Вдовиченко Станіслав Юрійович _____
(прізвище, ім'я, по батькові) (підпис)

Керівник доцент, к.т.н., Губський А.М. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Консультант ст.викл., Вітковська І.І. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Рецензент професор кафедри ОТ, д.т.н., проф., Корнага Я.І. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Засвідчую, що у цьому дипломному проєкті
немає запозичень з праць інших авторів без
відповідних посилань.

Студент _____
(підпис)

Київ – 2026

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії
Рівень вищої освіти – перший (бакалаврський)
Спеціальність – 121 Інженерія програмного забезпечення
Освітньо-професійна програма – Інженерія програмного забезпечення
інформаційних систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2026 р.

**ЗАВДАННЯ
на дипломний проєкт студенту**

Вдовиченку Станіславу Юрійовичу
(прізвище, ім'я, по батькові)

1. Тема проєкту Методи та засоби інтеграції системи Електронний кампус з Moodle

керівник проєкту Губський Андрій Миколайович, к.т.н., доцент
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «03» червня 2026 р. №2055-с

2. Термін подання студентом проєкту «13» червня 2026 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Загальні положення: основні визначення та терміни, опис предметного середовища, огляд ринку програмних продуктів, постановка задачі.

2) Інформаційне забезпечення: вхідні дані, вихідні дані, опис структури бази даних.

3) Математичне забезпечення: змістовна та математична постановки задачі, обґрунтування та опис методу розв'язання.

4) Програмне та технічне забезпечення: засоби розробки, вимоги до технічного забезпечення, архітектура програмного забезпечення, побудова звітів.

5) Технологічний розділ: керівництво користувача, методика випробувань програмного продукту.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використань

2) Схема структурна компонентів програмного забезпечення

3) Схема бази даних

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «15» березня 2026 року

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	15.03.2026	
2	Аналіз існуючих методів розв'язання задачі	03.04.2026	
3	Постановка та формалізація задачі	10.04.2026	
4	Розробка інформаційного забезпечення	13.04.2026	
5	Алгоритмізація задачі	21.04.2026	
6	Обґрунтування вибору використаних технічних засобів	22.04.2026	
7	Розробка програмного забезпечення	10.05.2026	
8	Налагодження програми	13.05.2026	
9	Виконання графічних документів	16.05.2026	
10	Оформлення пояснювальної записки	23.05.2026	
11	Подання ДП на попередній захист	10.06.2026	
12	Подання ДП рецензенту	12.06.2026	
13	Подання ДП на основний захист	15.06.2026	

Студент

(підпис)

Станіслав ВДОВИЧЕНКО

(ініціали, прізвище)

Керівник

(підпис)

Андрій ГУБСЬКИЙ

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з п'яти розділів, містить 28 таблиць, 24 рисунки та 14 джерел – загалом 82 сторінки.

Дипломний проєкт присвячений розробці серверного проміжного програмного забезпечення для автоматизованого перенесення відомостей успішності студентів між системами Moodle та Електронний кампус.

Мета проєкту – автоматизація операцій з перенесення оцінок та оптимізація робочого часу викладачів шляхом створення інтеграційного шлюзу між освітніми системами.

У першому розділі проведено передпроєктне обстеження предметної області: проаналізовано бізнес-процеси організації навчального процесу та фіксації успішності, розглянуто недоліки існуючих рішень інтеграції та обґрунтовано доцільність розробки власного програмного забезпечення.

Другий розділ присвячено розробленню вимог до програмного забезпечення: визначено варіанти використання системи, сформульовано функціональні, нефункціональні та системні вимоги, розраховано економічні показники проєкту та складено технічне завдання.

Третій розділ охоплює конструювання та розроблення програмного забезпечення: спроектовано архітектуру за моделлю С4, обґрунтовано вибір технологічного стеку, а також реалізовано алгоритми трансформації масивів даних.

У четвертому розділі виконано аналіз якості та тестування розробленого сервісу: перевірено якість вихідного коду за допомогою статичного аналізатора, проведено мануальне інтеграційне тестування ендпоінтів та наведено детальний контрольний приклад роботи системи.

П'ятий розділ детально описує процеси розгортання програмного забезпечення з використанням технологій контейнеризації та організацію його подальшого супроводу.

Програмне забезпечення реалізовано у вигляді інтеграційного серверного застосунку з використанням мови Java, фреймворку Spring Boot та архітектури REST API.

КЛЮЧОВІ СЛОВА: MOODLE, ЕЛЕКТРОННИЙ КАМПУС,
ІНТЕГРАЦІЯ, ПРОМІЖНЕ ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ, MIDDLEWARE,
REST API, ETL, JAVA, SPRING BOOT, DOCKER.

ABSTRACT

The explanatory note of the diploma project consists of five sections, contains 28 tables, 24 figures, and 14 sources – a total of 82 pages.

The diploma project is devoted to the development of server middleware for the automated transfer of students' academic performance records between Moodle and "Electronic Campus" systems.

The purpose of the project is to automate grade transfer operations and optimize teachers' working time by creating an integration gateway between the educational systems.

The first section covers the pre-project survey of the subject area: the business processes of educational process organization and academic performance recording are analyzed, the shortcomings of existing integration solutions are reviewed, and the feasibility of developing custom software is substantiated.

The second section is dedicated to the development of software requirements: system use cases are defined, functional, non-functional, and system requirements are formulated, project economic indicators are calculated, and the technical specification is compiled.

The third section covers software engineering and development: the architecture is designed according to the C4 model, the choice of the technology stack is substantiated, and data array transformation algorithms are implemented.

The fourth section performs quality analysis and testing of the developed service: source code quality is verified using a static analyzer, manual integration testing of endpoints is conducted, and a detailed test case of the system operation is provided.

The fifth section describes in detail the software deployment processes using containerization technologies and the organization of its further maintenance.

The software is implemented as an integration server application using the Java programming language, Spring Boot framework and REST API architecture.

KEYWORDS: MOODLE, ELECTRONIC CAMPUS, INTEGRATION, MIDDLEWARE, REST API, ETL, JAVA, SPRING BOOT, DOCKER.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**МЕТОДИ ТА ЗАСОБИ ІНТЕГРАЦІЇ СИСТЕМИ ЕЛЕКТРОННИЙ
КАМПУС З MOODLE**
Технічне завдання
КПІ.ПІ-2103.045430.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Андрій ГУБСЬКИЙ

Нормоконтроль:

_____ Ірина ВІТКОВСЬКА

Виконавець:

_____ Станіслав ВДОВИЧЕНКО

Київ – 2026

ЗМІСТ

1	НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	3
2	ПІДСТАВА ДЛЯ РОЗРОБКИ.....	4
3	ПРИЗНАЧЕННЯ РОЗРОБКИ	5
4	ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	6
4.1	Вимоги до функціональних характеристик	6
4.1.1	Користувацького інтерфейсу	6
4.1.2	Для системи-клієнта:.....	6
4.1.3	Додаткові вимоги:	6
4.2	Вимоги до надійності	7
4.3	Умови експлуатації.....	7
4.3.1	Вид обслуговування	7
4.3.2	Обслуговуючий персонал.....	7
4.4	Вимоги до складу і параметрів технічних засобів	7
4.5	Вимоги до інформаційної та програмної сумісності	8
4.5.1	Вимоги до вхідних даних	8
4.5.2	Вимоги до вихідних даних	8
4.5.3	Вимоги до мови розробки.....	8
4.5.4	Вимоги до середовища розробки.....	8
4.5.5	Вимоги до представленню вихідних кодів	8
4.6	Вимоги до маркування та пакування	9
4.7	Вимоги до транспортування та зберігання	9
4.8	Спеціальні вимоги.....	9
5	ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ	10
5.1	Попередній склад програмної документації	10
5.2	Спеціальні вимоги до програмної документації.....	10
6	СТАДІЇ І ЕТАПИ РОЗРОБКИ	11
7	ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ	12

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Методи та засоби інтеграції системи Електронний кампус з Moodle.

Галузь застосування:

Наведене технічне завдання поширюється на розробку проміжного програмного забезпечення (Middleware API) [КП.ІІ-2103.045430.02.81], котра використовується для автоматизованої синхронізації відомостей успішності студентів та призначена для застосування в системах університетів, а також у будь-яких освітніх чи корпоративних системах, що потребують інтеграції локальних баз даних із платформою Moodle.

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки Middleware API є завдання на дипломне проєктування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для забезпечення процесу автоматизованого перенесення оцінок студентів із системи Moodle до системи Електронний кампус.

Метою розробки є автоматизація рутинних операцій з ведення навчальних відомостей для оптимізації робочого часу викладачів.

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1 Користувацького інтерфейсу

Оскільки розроблюване програмне забезпечення є проміжним серверним шлюзом, воно не має графічного інтерфейсу користувача. Уся взаємодія із системою здійснюється виключно за допомогою HTTP-запитів через REST API ендпоінти.

4.1.2 Для системи-клієнта:

- міжсервісна авторизація: перевірка наявності та валідація секретного API-ключа в заголовках запиту для доступу до сервісу;
- отримання каталогу дисциплін: надання повного переліку доступних курсів із системи Moodle;
- ідентифікація викладачів: фільтрація користувачів курсу з метою повернення лише тих осіб, які мають права викладача;
- отримання контрольних заходів: вилучення переліку оцінюваних елементів курсу для налаштування мапінгу колонок;
- формування зведеного журналу: вилучення повного масиву оцінок, їх фільтрація, трансформація та повернення готового JSON-об'єкта для запису у відомість.

4.1.3 Додаткові вимоги:

- санітизація даних: очищення текстових відгуків та коментарів до оцінок від HTML-тегів;
- обробка помилок: перехоплення виняткових ситуацій та повернення кодів помилок без аварійного завершення роботи застосунку.

4.2 Вимоги до надійності

Система має бути побудована за принципами stateless (без збереження стану), що гарантує незалежність та стабільність паралельних сесій синхронізації.

У разі недоступності системи Moodle застосунок повинен лімітувати час очікування на рівні 10 секунд, після чого коректно розривати з'єднання.

Усі процеси агрегації та очищення масивів даних повинні виконуватися виключно в оперативній пам'яті сервера без створення тимчасових фізичних файлів на диску.

4.3 Умови експлуатації

Умови експлуатації згідно СанПін 2.2.2.542 – 96.

4.3.1 Вид обслуговування

Вимоги до виду обслуговування не висуваються.

4.3.2 Обслуговуючий персонал

Для розгортання та супроводу системи потрібен системний адміністратор із базовими навичками роботи з ОС Linux, технологією Docker та Docker Compose.

4.4 Вимоги до складу і параметрів технічних засобів

Програмне забезпечення повинно функціонувати на IBM-сумісних персональних комп'ютерах.

Мінімальна конфігурація технічних засобів:

- тип процесору: Intel Core i3;
- об'єм ОЗП: 2 Гб;
- підключення до мережі Інтернет зі швидкістю від 5 мегабіт.

Рекомендована конфігурація технічних засобів:

- тип процесору: Intel Core i5;

- об'єм ОЗП: 8 Гб;
- підключення до мережі Інтернет зі швидкістю від 50 мегабіт.

4.5 Вимоги до інформаційної та програмної сумісності

Програмне забезпечення повинно працювати під управлінням операційних систем сімейства Linux (Ubuntu 20.04+, Debian) або Windows Server.

4.5.1 Вимоги до вхідних даних

Вхідні дані повинні бути представлені у вигляді HTTP-запитів із передачею параметрів у URL та обов'язковою наявністю секретного API-ключа в заголовках.

4.5.2 Вимоги до вихідних даних

Результати повинні бути представлені у текстовому форматі JSON із дотриманням принципів REST API.

4.5.3 Вимоги до мови розробки

Розробку виконати на мові програмування Java за допомогою фреймворку Spring Boot.

4.5.4 Вимоги до середовища розробки

Розробку виконати за допомогою наступних інструментів розробки:

- середовище розробки: Visual Studio Code;
- система контейнеризації: Docker, Docker Compose;
- система тестування API: Postman;
- система контролю версій: Git.

4.5.5 Вимоги до представлення вихідних кодів

Вихідний код програми має бути представлений у вигляді репозиторію.

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не висуваються.

4.8 Спеціальні вимоги

Розроблене ПЗ має бути заповане в ізолюваний Docker-контейнер. Усі секретні ключі та адреси API мають підключатися через змінні середовища без втручання у вихідний код.

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Попередній склад програмної документації

У склад супроводжувальної документації повинні входити наступні документи на аркушах формату А4:

- пояснювальна записка;
- технічне завдання;
- текст програми;
- програма та методика тестування;
- керівництво користувача;
- керівництво системного програміста.

Графічна частина повинна бути виконана на аркушах формату А3 та містити наступні документи:

- схема структурна варіантів використання;
- схема структурна компонентів програмного забезпечення;
- схема бази даних.

5.2 Спеціальні вимоги до програмної документації

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення рекомендованої літератури	15.03.2026	
2.	Аналіз існуючих методів розв'язання задачі	03.04.2026	
3.	Постановка та формалізація задачі	10.04.2026	
4.	Розробка інформаційного забезпечення	13.04.2026	
5.	Алгоритмізація задачі	21.04.2026	
6.	Обґрунтування вибору використаних технічних засобів	22.04.2026	
7.	Розробка програмного забезпечення	10.05.2026	
8.	Налагодження програми	13.05.2026	
9.	Виконання графічних документів	16.05.2026	
10.	Оформлення пояснювальної записки	23.05.2026	
11.	Подання ДП на попередній захист	10.06.2026	
12.	Подання ДП рецензенту	12.06.2026	
13.	Подання ДП на основний захист	15.06.2026	

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

**Пояснювальна записка
до дипломного проєкту**

на тему: **Методи та засоби інтеграції системи Електронний кампус з Moodle**

КПІ.ПІ-2103.045430.02.81

ЗМІСТ

ВСТУП	5
1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	6
1.1 Постановка завдання дипломного проектування	6
1.2 Аналіз предметної області	6
1.3 Аналіз існуючих рішень	9
1.3.1 Аналіз відомих програмних продуктів	9
1.3.2 Аналіз відомих алгоритмічних та технічних рішень	13
1.4 Аналіз та моделювання бізнес-процесів	14
Висновки до розділу	18
2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	20
2.1 Варіанти використання програмного забезпечення	20
2.2 Розроблення функціональних вимог	25
2.3 Розроблення нефункціональних вимог	29
2.4 Аналіз системних вимог	31
2.5 Аналіз економічних показників програмного забезпечення	32
2.6 Постановка завдання на розробку програмного забезпечення	34
Висновки до розділу	35
3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	37
3.1 Архітектура програмного забезпечення	37
3.2 Архітектурні рішення та обґрунтування вибору засобів розробки	39
3.3 Конструювання програмного забезпечення	42
3.3.1 Розробка алгоритмів агрегації та трансформації даних	42
3.3.2 Опис програмних структур та моделей даних	44
3.3.3 Опис структури бази даних	46
3.3.4 Опис стороннього програмного забезпечення	49
3.4 Аналіз безпеки даних	51
Висновки до розділу	52

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	53
4.1 Аналіз якості ПЗ.....	53
4.2 Опис процесів тестування.....	55
4.3 Опис контрольного прикладу	62
Висновки до розділу	69
5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	70
5.1 Розгортання програмного забезпечення.....	70
5.2 Супровід програмного забезпечення	72
Висновки до розділу	73
ВИСНОВКИ.....	74
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	76
ДОДАТКИ.....	78
ДОДАТОК А ЗВІТ ПОДІБНОСТІ.....	78

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API	– Application programming interface - прикладний програмний інтерфейс
LMS	– Learning Management System - система управління навчанням
REST	– Representational State Transfer
ETL	– Extract, Transform, Load. Процес видобування, перетворення та завантаження даних
ELT	– Extract, Load, Transform. Процес видобування, завантаження та перетворення даних
JSON	– JavaScript Object Notation - текстовий формат обміну даними
HTTP/HTTPS	– HyperText Transfer Protocol / Secure. Протокол передачі гіпертексту / безпечний протокол
HTML	– HyperText Markup Language
SQL	– Structured Query Language
XSS	– Cross-Site Scripting
БД	– База даних

ВСТУП

У сучасному світі цифровізація освіти стала невід'ємною частиною діяльності вищих навчальних закладів. Однією з ключових проблем, з якою стикаються викладачі, є синхронізація даних між різними інформаційними системами. Саме тому актуальним є створення застосунку, який дасть змогу автоматизувати перенесення оцінок студентів між системами.

Актуальність розроблення проміжного програмного забезпечення для синхронізації даних між системами Moodle та «Електронний кампус» пояснюється зростаючими вимогами до оптимізації робочого часу викладачів, які прагнуть уникати дублювання рутинної роботи.

На сьогодні є низка готових плагінів та стандартів, орієнтованих на обмін освітніми даними. Проте більшість з них має суттєві недоліки: вони або орієнтовані на стандартизовані платформи і не здатні безпосередньо взаємодіяти з структурою бази даних університетських систем, або вимагають ручного експорту та імпорту файлів.

Метою роботи є автоматизація рутинних операцій з ведення навчальних відомостей для оптимізації робочого часу викладачів.

Сферами практичного застосування цього програмного забезпечення є інформаційне середовище університету, а також будь-які освітні чи корпоративні системи, що потребують інтеграції локальних баз даних із системою Moodle.

1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Постановка завдання дипломного проєктування

Дипломне проєктування передбачає виконання наступних завдань:

- аналіз предметної області та опис її ключових бізнес-процесів, визначення загального завдання розробки у рамках ДП;
- аналіз існуючих рішень (як програмних продуктів, так і алгоритмічних чи технічних рішень) обраного завдання розробки у рамках ДП;
- аналіз та моделювання бізнес-процесів;
- розроблення функціональних, нефункціональних та системних вимог до програмного забезпечення;
- аналіз економічних показників програмного забезпечення ДП;
- постановка завдання на розробку програмного забезпечення ДП;
- розроблення архітектури програмного забезпечення;
- розроблення архітектурних рішень та обґрунтування вибору засобів розробки програмного забезпечення;
- конструювання та розроблення програмного забезпечення;
- аналіз безпеки даних програмного забезпечення;
- аналіз якості та тестування програмного забезпечення;
- розгортання та супровід програмного забезпечення.

1.2 Аналіз предметної області

Стрімкий розвиток інформаційних технологій та цифровізація освіти призвели до масового впровадження систем управління навчанням (LMS) [1] у закладах вищої освіти. LMS являють собою складні програмні комплекси призначені для адміністрування, документування, відстеження, звітності та надання електронних навчальних курсів. Основною метою впровадження таких систем є створення єдиного віртуального освітнього середовища, яка

забезпечує взаємодію між студентами, викладачами та адміністрацією навчального закладу.

Сьогодні на ринку освітніх технологій домінують системи з відкритим вихідним кодом, серед яких беззаперечним лідером є Moodle (Modular Object-Oriented Dynamic Learning Environment) [2]. Архітектура Moodle передбачає реалізацію навчального процесу на основі активної взаємодії учасників та обміну знаннями. Система надає інструментарій для створення складних систем тестування, які дозволяють не лише оцінювати рівень підготовки, але й забезпечувати зворотний зв'язок між викладачем та студентом. Завдяки модульній структурі, ця система легко розширюється за допомогою плагінів, що робить її стандартом для організації дистанційного та змішаного навчання.

Паралельно з впровадженням LMS, сучасні університети використовують глобальні корпоративні інформаційні системи для управління адміністративними процесами. Якщо Moodle фокусується безпосередньо на процесі навчання, то такі системи відповідають за офіційний документообіг, формування навчальних планів та ведення відомостей.

У Національному технічному університеті України «Київський політехнічний інститут імені Ігоря Сікорського» функцію такої інтегрованої корпоративної системи виконує «Електронний кампус» [3]. Ця система є ядром цифрової інфраструктури університету, що акумулює офіційні дані про результати семестрового контролю, атестацій та успішність кожного студента.

На даний момент процеси організації навчання та поточного оцінювання розподілені між декількома освітніми платформами, тоді як офіційний облік успішності ведеться в системі Кампус, що створює проблему розрізненості даних.

Поточний бізнес-процес фіксації успішності виглядає наступним чином:

- викладач розробляє курс у середовищі Moodle;
- студенти виконують практичні завдання, проходять тестування та отримують бали, які агрегуються у журналі оцінок Moodle;

- при настанні періоду календарного контролю (перша та друга атестації) та семестрового контролю (залік, екзамен), викладач змушений формувати вибірку оцінок з Moodle;

- далі викладач здійснює вхід до системи “Електронний кампус”, відкриває відповідну електронну відомість і власноруч переносить бали для кожного студента;

- після завершення введення даних відомість зберігається та надсилається на опрацювання.

Такий підхід, побудований на ручній синхронізації даних між двома інформаційними системами, має низку критичних недоліків:

- ручне перенесення числових даних призводить до помилок;
- викладачі витрачають значний обсяг часу на рутинну роботу з дублювання даних;

- інформація в системі «Електронний кампус» оновлюється дискретно (лише під час атестацій або сесій), через це адміністрація та самі студенти не мають доступу до зрізу своєї актуальної успішності в режимі реального часу;

- при збільшенні кількості студентів або введенні нових форм контролю, навантаження на викладачів зростає.

Вирішення проблеми розрізненості даних в ІТ-інфраструктурі зазвичай досягається шляхом системної інтеграції. Серед можливих шляхів покращення ситуації можна виділити такі:

- інтеграція на рівні баз даних (Database-level integration): передбачає надання спільного доступу до таблиць БД обох систем; є швидким, але небезпечним з точки зору цілісності даних та порушує принципи інкапсуляції;

- файловий обмін (File-based integration): регулярне вивантаження CSV або XML файлів з однієї системи та імпорт в іншу; вимагає ручного втручання та не забезпечує синхронізацію в реальному часі;

– інтеграція на рівні прикладних програмних інтерфейсів (API-based integration): використання REST або SOAP архітектури для обміну даними між системами за допомогою стандартизованих запитів.

У рамках даного дипломного проєкту обрано розробку проміжного програмного забезпечення (middleware-сервісу), який виступатиме сполучною ланкою між Moodle та системою «Електронний кампус». Цей сервіс використовуватиме REST API системи Moodle (Web Services) для збору інформації про курси, користувачів та їх успішність.

Вибір архітектури на базі middleware зумовлений тим, що вона забезпечує високий рівень безпеки (системи не мають прямого доступу до баз даних одна одної) та дозволяє легко трансформувати дані з формату Moodle у формат, необхідний для Кампусу.

1.3 Аналіз існуючих рішень

Проаналізуємо відоме на сьогодні алгоритмічне забезпечення у даній області та технічні рішення, що допоможуть у реалізації Middleware API. Далі будуть розглянуті готові програмні рішення, допоміжні програмні засоби та засоби розробки.

1.3.1 Аналіз відомих програмних продуктів

Завдання синхронізації даних між середовищем Moodle та університетськими інформаційними системами не є новим. На ринку освітніх технологій існує кілька поширених підходів та готових програмних продуктів, які частково або повністю реалізують необхідний функціонал.

Першим і найбільш доступним рішенням є базовий інструментарій Moodle, зокрема механізм обміну даними Flat-file CSV Import/Export. Цей стандартний функціонал, вбудований безпосередньо у ядро системи, дозволяє адміністраторам та викладачам вивантажувати оцінки з електронного журналу у форматах CSV, Excel або XML. Приклад інтерфейсу для вибору формату,

налаштування параметрів та результату експорту оцінок представлено на рисунках 1.1, 1.2.

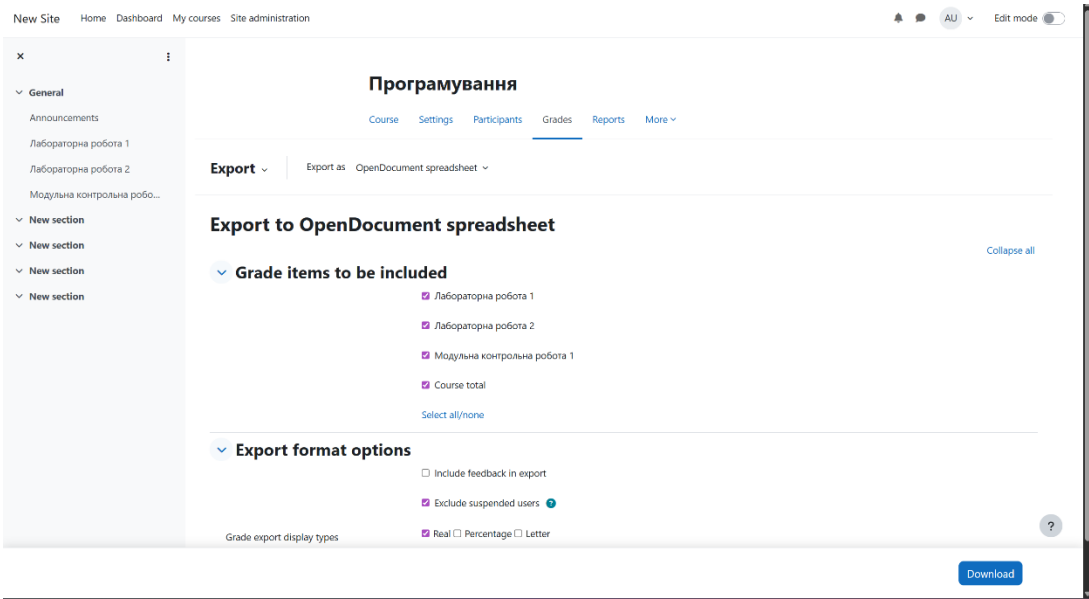


Рисунок 1.1 – Інтерфейс експорту оцінок у системі Moodle

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	First name, last name	ID number	Institution	Department	Email address	Assignment: Лабо	Assignment: Лабораторна робо	Assignment: Моду	Course total (Real)	Last downloaded from this course						
2	Oksana Bondar				oxbondar@gmail.com	11	12	13	36	1774810612						
3	Ivan Ivanenko				ivanivanenko@gmail.co	25	25	50	100	1774810612						
4	Yulia Kovalenko				kovalenkoyu@gmail.co	14	15	16	45	1774810612						
5	Roman Kravchenko				kravchenko@gmail.co	17	18	19	54	1774810612						
6	Pavlo Kuzmenko				kuzmenkop@gmail.co	9	20	40	69	1774810612						
7	Oleg Cilynyk				cilynyk@gmail.com	21	11	29	61	1774810612						
8	Alisa Petrenko				a.petrenko@gmail.com	17	18	20	55	1774810612						
9	Dmytro Romaniuk				romaniukdm@gmail.co	11	5	49	65	1774810612						
10	Sergiy Shevchenko				sergiyshev@gmail.com	11	10	22	43	1774810612						
11	Olia Tkachenko				tkachenkoo@gmail.co	25	10	30	65	1774810612						
12																
13																
14																
15																
16																
17																
18																
19																
20																
21																
22																
23																

Рисунок 1.2 – Вигляд результату експорту у форматі таблиці

Крім того, даний інструментарій підтримує масову реєстрацію користувачів через завантаження підготовлених текстових файлів. Хоча система дозволяє налаштувати автоматичний парсинг файлів з локального сервера через механізм Cron, будь-яка зміна структури документа або поява непередбачених символів призводить до критичного збою процесу імпорту.

Альтернативним методом виступають офіційні плагіни інтеграції, такі як External Database Authentication та Enrolment. Ці модулі забезпечують можливість підключення Moodle до зовнішньої реляційної бази даних для

автоматичної синхронізації списків користувачів. Типовий вигляд панелі конфігурації підключення до зовнішньої бази даних MariaDB зображено на рисунку 1.3.

The screenshot shows the 'External database' configuration page in Moodle. The page title is 'External database'. Below the title, there is a description: 'This method uses an external database table to check whether a given username and password is valid. If the account is a new one, then information from other fields may also be copied across into Moodle.' The page contains several form fields with labels and default values:

- Host** (auth_db | host): 127.0.0.1 (Default: 127.0.0.1)
- Database** (auth_db | type): mysql (Default: mysql)
- Use sybase quotes** (auth_db | sybasequoting): No (Default: No)
- DB name** (auth_db | name): (Default: Empty)
- DB user** (auth_db | user): (Default: Empty)
- Password** (auth_db | pass): (Default: Empty)
- Table** (auth_db | table): (Default: Empty)
- Username field** (auth_db | username): (Default: Empty)

There are also links for 'Click to enter text' and 'Click to enter text' next to the Password field. A help icon (?) is visible in the bottom right corner.

Рисунок 1.3 – Налаштування параметрів аутентифікації через зовнішню базу даних

Даний інструмент орієнтований на синхронізацію облікових записів та зарахування користувачів, проте не підтримує передачу результатів оцінювання. Крім того, пряме підключення до бази даних «Електронного кампусу» порушує принцип ізоляції даних та створює ризики цілісності структури БД при виконанні операцій запису зовнішніми скриптами.

Для масштабних проєктів у великих університетах часто застосовують комерційні рішення класу Enterprise Service Bus (ESB), такі як Ellucian ILP [4]. Дана платформа пропонує готові конектори для обміну даними між Moodle та потужними ERP-системами, використовуючи брокери повідомлень для гарантованої доставки інформації. Однак для локальних завдань інтеграції такі системи є надлишковими. Впровадження комерційної інтеграційної шини вимагає значних фінансових витрат на придбання ліцензій та розгортання серверних кластерів, що є недоцільним у межах даної розробки.

Окремим класом рішень для розв'язання задач синхронізації є хмарні платформи інтеграції типу iPaaS (Integration Platform as a Service), яскравим представником яких є сервіс Zapier [5]. Він дозволяє налаштовувати автоматизований обмін даними між Moodle та іншими системами через Webhooks та REST API за допомогою зручного візуального конструктора.

Проте використання платформи Zapier несе суттєві ризики для інформаційної безпеки, оскільки персональні дані студентів проходять через сторонні комерційні сервери, що знаходяться поза межами локальної мережі університету.

В таблиці 1.1 наведено порівняння власної розробки з аналогами.

Таблиця 1.1 – Порівняння з аналогами

Функціонал	Middleware API	Moodle Flat-file CSV	External DB Plugin	Ellucian ILP	Хмарні iPaaS (Zapier)
Автоматизація обміну даними	+	-	-	+	+
Синхронізація оцінок	+	+	-	+	+
Ізоляція бази даних	+	+	-	+	+
Відсутність ліцензійної оплати	+	+	+	-	-
Спеціалізація під Кампус	+	-	-	-	-

Як видно з таблиці 1.1, існуючі рішення Moodle не здатні забезпечити автоматизовану та безпечну передачу оцінок, а комерційні аналоги є занадто дорогими та архітектурно перевантаженими. Таким чином, розробка спеціалізованого проміжного програмного забезпечення є найбільш раціональним рішенням.

1.3.2 Аналіз відомих алгоритмічних та технічних рішень

У контексті системної інтеграції та забезпечення безпечного потоку даних між інформаційними системами застосовуються різні алгоритмічні та архітектурні підходи.

Базовим алгоритмічним рішенням для перенесення та обробки великих масивів інформації є застосування процесу ETL (Extract, Transform, Load). На відміну від прямої інтеграції «точка-точка» (Point-to-Point) або підходу ELT (Extract, Load, Transform), патерн ETL дозволяє витягувати дані з джерела (Extract), виконувати складну структурну реорганізацію та очищення в ізольованому проміжному середовищі (Transform), і лише після цього завантажувати готові пакети у цільову реляційну базу даних (Load). Такий підхід мінімізує навантаження на цільові системи та забезпечує високу якість даних.

З точки зору програмної інженерії, для зв'язування систем із фундаментально різними структурами найчастіше розглядають структурні шаблони проєктування, такі як Adapter та Facade. Патерн Adapter дозволяє об'єктам з несумісними інтерфейсами працювати разом, виступаючи своєрідним перехідником форматів. Патерн Facade, у свою чергу, надає простий уніфікований інтерфейс до складної підсистеми, приховуючи від кінцевого споживача (у нашому випадку - системи Кампус) всю внутрішню складність формування запитів.

Щодо вибору стилю мережевої взаємодії, сучасна індустрія пропонує стандарти SOAP та REST API. Хоча SOAP забезпечує високий рівень безпеки через протокол WS-Security, він є занадто перевантаженим XML-розміткою.

Архітектура REST API є більш легкою, загальновизнаною та оптимальною для вебсервісів, оскільки вона забезпечує високу швидкість обміну даними у форматі JSON, легко масштабується та дозволяє будувати stateless (безстанові) сервіси.

Для забезпечення безпеки передачі конфіденційної інформації базовими технічними рішеннями є використання криптографічного протоколу HTTPS та механізмів авторизації. Серед алгоритмів авторизації (Basic Auth, OAuth 2.0, JWT, Token-based) використання статичних або динамічних токенів доступу є найбільш збалансованим рішенням для міжсервісної (Server-to-Server) взаємодії, оскільки воно гарантує, що операції експорту даних виконуватимуть виключно авторизовані підсистеми.

Враховуючи проведений аналіз, для реалізації даного дипломного проєкту було обрано наступний стек рішень: алгоритмічний патерн ETL, шаблони Adapter та Façade, REST API, механізм авторизації на основі токенів поверх протоколу HTTPS.

1.4 Аналіз та моделювання бізнес-процесів

Для моделювання бізнес-процесу синхронізації результатів оцінювання використовується BPMN модель (рисунок 1.4).

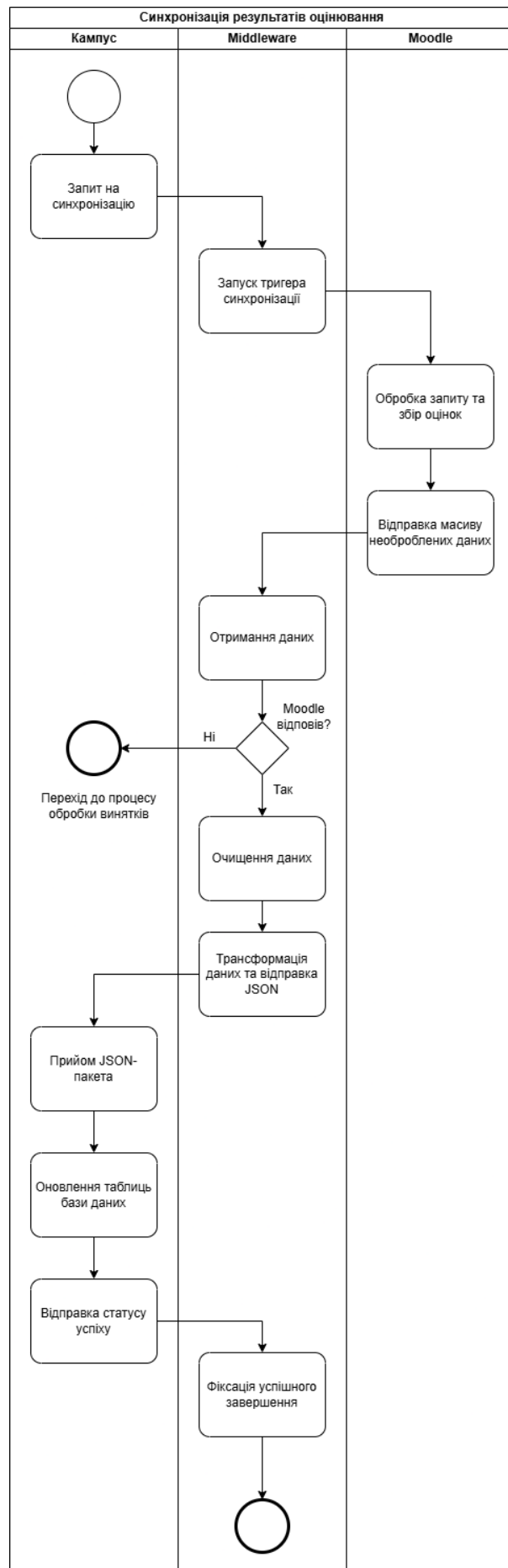


Рисунок 1.4 – Схема синхронізації результатів оцінювання

Опис моделі бізнес-процесу синхронізації результатів оцінювання:

- система «Електронний кампус» формує та надсилає запит на синхронізацію конкретної відомості;
- проміжне програмне забезпечення приймає сигнал, запускає тригер синхронізації та звертається до Moodle;
- система Moodle обробляє запит, збирає оцінки та відправляє масив необроблених даних;
- проміжне програмне забезпечення отримує дані та перевіряє статус відповіді;
- якщо система Moodle не відповіла або повернула помилку, виконання переривається та ініціюється перехід до процесу обробки винятків;
- за умови успішної відповіді проміжне програмне забезпечення очищує дані та виконує їх трансформацію;
- трансформований JSON-пакет надсилається до цільової системи;
- кампус приймає пакет, виконує оновлення таблиць бази даних та відправляє статус успіху;
- проміжний сервіс фіксує успішне завершення циклу.

Для моделювання бізнес-процесу авторизації та обробки помилок використовується BPMN модель (рисунок 1.5).

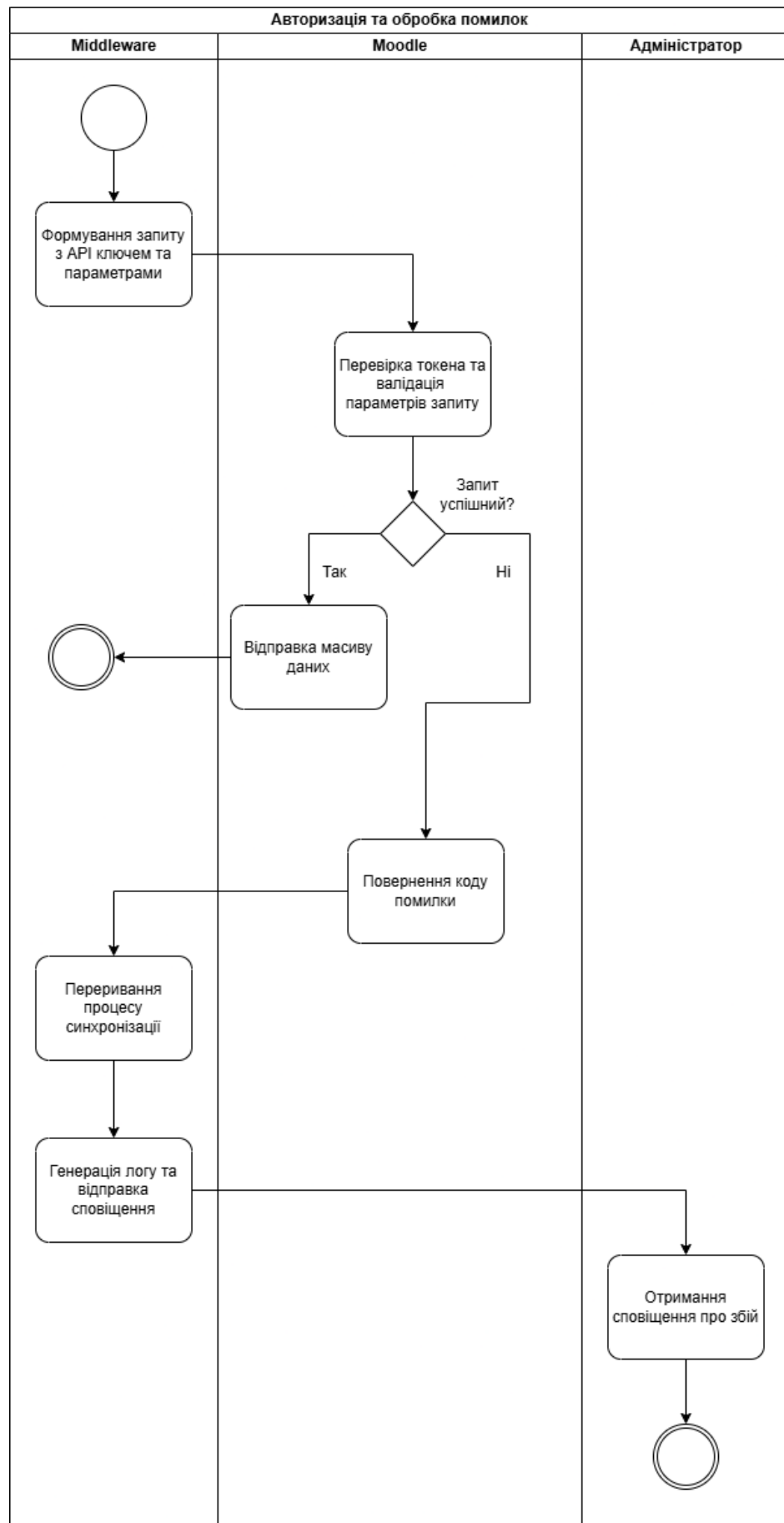


Рисунок 1.5 – Схема авторизації та обробки помилок

Опис моделі бізнес-процесу авторизації та обробки помилок:

- проміжний сервіс формує HTTP-запит, додаючи до нього ключ та необхідні параметри ідентифікації;
- система Moodle виконує перевірку токена та валідацію отриманих параметрів запиту;
- якщо параметри або ключ не є валідними, Moodle повертає відповідний код помилки;
- у разі успішної валідації система Moodle відправляє масив даних, після чого продовжується робота синхронізації;
- при отриманні коду помилки проміжний сервіс виконує переривання процесу синхронізації;
- сервіс ініціює генерацію логу з описом проблеми та відправляє сповіщення;
- адміністратор системи отримує сповіщення про збій.

Висновки до розділу

У першому розділі було проведено передпроектне обстеження предметної області та обґрунтовано необхідність розробки спеціалізованого програмного забезпечення.

У підрозділі 1.1 сформульовано загальну постановку завдання дипломного проєктування. Визначено ключові цілі та етапи роботи, необхідні для створення проміжного програмного забезпечення.

У підрозділі 1.2 проведено детальний аналіз предметної області, а саме особливостей використання LMS Moodle та інформаційної системи «Електронний кампус» у навчальному процесі університету.

У підрозділі 1.3 здійснено аналіз існуючих рішень. Розгляд існуючих програмних продуктів показав їх невідповідність вимогам локальної інтеграції. На основі аналізу відомих алгоритмічних та технічних рішень обґрунтовано доцільність власної розробки з використанням патерну ETL та

архітектурного стилю REST API як найоптимальнішого шляху реалізації проєкту.

У підрозділі 1.4 виконано аналіз та моделювання ключових бізнес-процесів за допомогою нотації BPMN.

2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Варіанти використання програмного забезпечення

Головною функцією програмного забезпечення є експорт оцінок із системи Moodle до Кампусу.

Діаграма варіантів використання з ключовими акторами та основними функціями знаходиться у графічному матеріалі, креслення 3.

В таблицях 2.1-2.6 наведено опис варіантів використання програмного забезпечення.

Таблиця 2.1 – Варіант використання UC-01

Use case name	Отримати каталог дисциплін
Use case ID	UC-01
Goals	Надати системі-клієнту перелік доступних курсів із бази Moodle
Actors	Система «Електронний кампус» (ініціатор), Система Moodle.
Trigger	Система «Електронний кампус» надсилає HTTP-запит до API.
Pre-conditions	Наявне активне мережеве з'єднання між API та системою Moodle.
Flow of Events	Система «Електронний кампус» ініціює запит на отримання списку курсів. Спрацьовує обов'язковий включений прецедент «Авторизуватися в Moodle» (<<include>>). API звертається до системи Moodle за каталогом. Отримані дані очищуються та повертаються до «Електронного кампусу».
Extension	Якщо під час авторизації або запиту виникає збій, виконується розширення «Обробка помилок».

Продовження таблиці 2.1

Post-Condition	Клієнт отримує валідний JSON-список із назвами та ідентифікаторами курсів.
----------------	--

Таблиця 2.2 – Варіант використання UC-02

Use case name	Отримати список викладачів
Use case ID	UC-02
Goals	Витягнути та відфільтрувати користувачів курсу для перевірки прав викладача.
Actors	Система «Електронний кампус» (ініціатор), Система Moodle.
Trigger	Запит від клієнтської системи на ідентифікацію викладачів обраного курсу.
Pre-conditions	Ідентифікатор курсу (course_id) передано в параметрах запиту.
Flow of Events	Система «Електронний кампус» робить виклик до API. Виконується авторизація API в системі Moodle (<<include>>). API запитує в Moodle всіх зарахованих на курс користувачів. API фільтрує користувачів, залишаючи лише тих, хто має роль викладача. Відфільтрований масив повертається клієнту.
Extension	-
Post-Condition	«Електронний кампус» отримує список підтверджених викладачів.

Таблиця 2.3 – Варіант використання UC-03

Use case name	Отримати список контрольних заходів
Use case ID	UC-03
Goals	Отримати перелік доступних елементів оцінювання (колонок журналу) з Moodle.
Actors	Система «Електронний кампус» (ініціатор), Система Moodle.
Trigger	Система «Електронний кампус» потребує даних для мапінгу колонок відомості.
Pre-conditions	Курс успішно ідентифіковано.
Flow of Events	Ініціюється виклик відповідного ендпоінту проміжного API. API проходить авторизацію (<<include>>). API звертається до Moodle для отримання структури контрольних заходів. Дані форматуються і відправляються у відповідь.
Extension	Якщо в курсі Moodle немає жодного створеного заходу, повертається порожній список.
Post-Condition	Кампус отримує структуру заходів.

Таблиця 2.4 – Варіант використання UC-04

Use case name	Отримати зведений журнал
Use case ID	UC-04
Goals	Відфільтрувати та трансформувати масив оцінок для їх перенесення у відомість.
Actors	Система «Електронний кампус» (ініціатор), Система Moodle.
Trigger	Команда на імпорт результатів від системи-клієнта.
Pre-conditions	Успішно отримано id курсу та налаштовано мапінг контрольних заходів.
Flow of Events	Система «Електронний кампус» ініціює запит на отримання оцінок. API автоматично авторизується (<<include>>). API витягує весь масив оцінок із Moodle. API виконує трансформацію даних та фільтрацію за конкретним викладачем. Фінальний структурований JSON повертається клієнту.
Extension	-
Post-Condition	«Електронний кампус» отримує готові до запису у відомість оцінки.

Таблиця 2.5 – Варіант використання UC-05

Use case name	Авторизуватися в Moodle
Use case ID	UC-05
Goals	Забезпечити безпечний доступ API до вебсервісів Moodle та керувати можливими збоями.
Actors	Система Moodle (джерело)
Trigger	Автоматичний виклик під час ініціації будь-якого з основних варіантів використання (UC-01 – UC-04).
Pre-conditions	Проміжне API містить згенерований ключ доступу (токен).
Flow of Events	API додає системний токен до заголовків HTTP-запиту. API звертається до Moodle. Moodle перевіряє токен і дозволяє доступ до даних.
Extension	UC-06 (Обробка помилок). Спрацьовує у разі невалідності токена або відсутності відповіді від сервера (таймаут).
Post-Condition	Процес ідентифікації пройдено, доступ до запитаного дозволено

Таблиця 2.6 – Варіант використання UC-06

Use case name	Обробка помилок
Use case ID	UC-06
Goals	Забезпечити коректне перехоплення виняткових ситуацій (недоступність сервера Moodle, недійсний токен) та інформування клієнта без аварійного завершення роботи API.
Actors	Система «Електронний кампус» (отримувач результату)
Trigger	Виникає мережева помилка, таймаут або помилка валідації під час виконання базового прецеденту «Авторизуватися в Moodle» (спрацьовує умова розширення <<extend>>).
Pre-conditions	Ініційовано процес звернення до вебсервісів системи Moodle.
Flow of Events	Базовий процес (UC-05) фіксує неможливість виконання запиту. Проміжне API перехоплює виняткову ситуацію (exception). Логіка API блокує подальше виконання основного алгоритму. API формує стандартизовану JSON-відповідь із кодом та описом помилки. Сформована відповідь відправляється до системи «Електронний кампус» для інформування користувача (викладача).
Extension	-
Post-Condition	Поточний запит коректно перервано, проміжне API продовжує роботу.

2.2 Розроблення функціональних вимог

Програмне забезпечення розділене на модулі. Кожен модуль має свій певний набір функцій. В таблиці 2.7 наведено загальну модель вимог, а в

таблиці 2.8 наведений опис функціональних вимог до програмного забезпечення. Матриця трасування вимог зображена на рисунку 2.1.

Таблиця 2.7 – Загальна модель вимог

№	Назва	ID Вимоги	Пріоритети	Ризики
1	Забезпечення міжсервісної авторизації	FR-1	Високий	Високий
1.1	Ін'єкція токена авторизації	FR-2	Високий	Високий
1.2	Блокування доступу	FR-3	Високий	Середній
2	Отримання та обробка навчальних даних	FR-4	Високий	Низький
2.1	Запит каталогу доступних дисциплін	FR-5	Високий	Низький
2.2	Фільтрація викладачів за ролями	FR-6	Високий	Середній
2.3	Отримання структури контрольних заходів	FR-7	Високий	Низький
3	Трансформація та експорт оцінок	FR-8	Високий	Високий
3.1	Завантаження масиву оцінок	FR-9	Високий	Середній
3.2	Фільтрація оцінок за ідентифікатором викладача	FR-10	Високий	Високий
3.3	Видалення HTML-розмітки	FR-11	Середній	Низький
3.4	Формування вихідного JSON	FR-12	Високий	Середній
4	Обробка помилок	FR-13	Середній	Низький
4.1	Обробка мережесих таймаутів	FR-14	Середній	Низький

Продовження таблиці 2.7

№	Назва	ID Вимоги	Пріоритети	Ризики
4.2	Обробка некоректних вхідних параметрів	FR-15	Середній	Низький

Таблиця 2.8 – Перелік функціональних вимог

Назва	Опис
FR-1	Система міжсервісної авторизації. Функціональність, яка забезпечує безпечну ідентифікацію Middleware API у зовнішніх вебсервісах Moodle.
FR-2	Ін'єкція токена авторизації. Система автоматично додає сконфігурований секретний ключ до заголовків кожного HTTP-запиту для отримання доступу до даних.
FR-3	Блокування несанкціонованого доступу. Система перериває виконання запиту та повертає помилку, якщо токен доступу не пройшов верифікацію на стороні Moodle.
FR-4	Система обробки навчальних даних. Функціональність, що реалізує збір, структурування та підготовку метаданих курсів для системи «Електронний кампус».
FR-5	Запит каталогу доступних дисциплін. API надає можливість отримати повний перелік курсів із Moodle.
FR-6	Фільтрація викладачів за ролями. Система ідентифікує користувачів курсу та повертає лише тих осіб, які мають права викладача з можливістю редагування.
FR-7	Отримання структури контрольних заходів. API витягує перелік усіх оцінюваних елементів курсу (тести, завдання) для налаштування мапінгу.

Продовження таблиці 2.8

Назва	Опис
FR-8	Система трансформації та експорту оцінок. Функціональність, що забезпечує повний цикл перенесення оцінок між системами.
FR-9	Завантаження масиву оцінок. Програмне забезпечення виконує виклик до Moodle для отримання всього журналу оцінок обраної дисципліни.
FR-10	Фільтрація оцінок за ідентифікатором викладача. Система аналізує журнал та залишає в пакеті даних лише оцінки, виставлені конкретним викладачем.
FR-11	Видалення HTML-розмітки. Автоматичне очищення текстових відгуків та коментарів від технічних тегів.
FR-12	Формування вихідного JSON-об'єкта. Конвертація оброблених оцінок у стандартизований пакет даних.
FR-13	Система обробки помилок. Функціональність, що забезпечує безперервну роботу API та інформування клієнта помилки.
FR-14	Обробка мережесих таймаутів. Система коректно перериває сесію та повертає статус 503, якщо зовнішній сервер Moodle не відповідає протягом заданого часу.
FR-15	Валідація вхідних параметрів. Перевірка ідентифікаторів курсів та завдань на коректність.

	FR-1	FR-2	FR-3	FR-4	FR-5	FR-6	FR-7	FR-8	FR-9	FR-10	FR-11	FR-12	FR-13	FR-14	FR-15
UC-1				+	+										
UC-2						+									
UC-3							+								
UC-4								+	+	+	+	+			
UC-5	+	+	+												
UC-6													+	+	+

Рисунок 2.1 – Матриця трасування вимог

Як видно з рисунку 2.1, всі вимоги покриті відповідними варіантами використання.

2.3 Розроблення нефункціональних вимог

Для стабільної роботи проміжного програмного забезпечення та забезпечення надійної інтеграції між системою «Електронний кампус» і Moodle було визначено такі нефункціональні вимоги:

а) продуктивність:

1) система повинна забезпечувати обробку легких інформаційних запитів (отримання каталогу дисциплін, структури заходів) не довше ніж за 1,5 секунди;

2) трансформація, очищення від HTML-розмітки та фільтрація великих масивів оцінок повинні виконуватися виключно в оперативній пам'яті сервера без створення тимчасових фізичних файлів на диску, що дозволяє уникнути вузьких місць вводу-виводу;

3) архітектура API повинна підтримувати паралельну обробку множинних одночасних запитів на експорт відомостей від різних викладачів без суттєвого збільшення часу відгуку;

б) надійність:

- 1) у разі недоступності системи Moodle проміжне API не повинно блокувати свої робочі потоки нескінченим очікуванням;
 - 2) система має жорстко лімітувати час очікування відповіді на рівні 10 секунд, після чого коректно розривати з'єднання та повідомляти клієнта;
 - 3) програмне забезпечення повинно базуватися на архітектурі без збереження стану;
 - 4) збій при обробці журналу одного конкретного викладача не повинен впливати на стабільність паралельних сесій інших користувачів;
- в) безпека:
- 1) секретні ключі та API-токени доступу до Moodle Web Services категорично заборонено зберігати у відкритому вигляді у файлах з вихідним кодом;
 - 2) усі чутливі дані мають динамічно підтягуватися зі змінних оточення або захищених конфігураційних файлів;
 - 3) уся передача даних між клієнтською системою, проміжним API та Moodle повинна здійснюватися виключно через зашифровані канали зв'язку з використанням протоколу HTTPS (TLS 1.2 або вище) для запобігання перехопленню інформації;
- г) сумісність:
- 1) програмне забезпечення повинно суворо відповідати архітектурним принципам REST;
 - 2) усі вхідні параметри та згенеровані вихідні результати мають передаватися виключно у стандартизованому форматі JSON із коректним використанням HTTP-статусів;
 - 3) система повинна запускатися в будь-якому серверному середовищі, де встановлено Java Runtime Environment версії 17;
- д) підтримка:
- 1) логіка бізнес-процесів має бути реалізована у вигляді окремих сервісів для спрощення їх тестування;

2) програмне забезпечення також повинно підтримувати пакування в ізольований Docker-контейнер для швидкого розгортання в хмарному середовищі;

е) масштабованість: архітектура проміжного API повинна дозволити легке підключення нових ендпоінтів для розширення функціоналу інтеграції без критичних змін існуючої кодової бази.

2.4 Аналіз системних вимог

Оскільки розроблене ПЗ є серверним компонентом, вимоги поділено на програмні та апаратні. Для стабільної роботи системи було визначено такі системні вимоги:

а) програмні вимоги до середовища виконання:

1) програмне забезпечення повинно запускатися в операційних системах сімейства Linux (рекомендовано Ubuntu 20.04+, Debian) або Windows Server;

2) обов'язковою умовою є наявність встановленого середовища виконання Java Runtime Environment або Java Development Kit версії 17 або вище;

3) для забезпечення швидкого розгортання та ізоляції залежностей система повинна мати підтримку пакування в Docker-контейнер (версія Docker Engine 20.10+);

б) апаратні вимоги до сервера:

1) для забезпечення штатної швидкодії при обробці запитів від «Електронного кампусу» сервер повинен мати процесор із тактовою частотою не менше 1.5 GHz (мінімум 1 ядро, рекомендовано 2 ядра для ефективної роботи GC у Java);

2) обсяг оперативної пам'яті має становити не менше 1 GB (з виділенням під Heap-пам'ять JVM не менше 512 MB), рекомендований обсяг - 2 GB;

- 3) дисковий простір повинен складати не менше 500 МВ для розміщення виконуваного файлу та збереження логів роботи;
- в) вимоги до мережевої інфраструктури:
 - 1) проміжне ПЗ повинно мати стабільний доступ до мережі Інтернет або локальної мережі університету для зв'язку з вебсервісами Moodle через захищений протокол HTTPS;
 - 2) для вхідних запитів від системи «Електронний кампус» у системному брандмауері має бути відкрито відповідний порт (за замовчуванням 8080) або налаштовано перенаправлення трафіку через зворотний проксі-сервер;
- г) вимоги до безпеки середовища:
 - 1) серверна ОС повинна підтримувати механізми обмеження прав доступу, щоб процес Java-застосунку запускався від імені окремого системного користувача з мінімальними привілеями;
 - 2) система повинна забезпечувати можливість зберігання секретних токенів авторизації у змінних оточення для запобігання їх витоку;
- д) вимоги до супроводу та моніторингу:
 - 1) середовище розгортання має забезпечувати доступ до системного виводу (stdout/stderr) для збору логів застосунку;
 - 2) рекомендовано наявність інструментів автоматичного перезапуску процесу у разі критичного збою.

2.5 Аналіз економічних показників програмного забезпечення

Економічне обґрунтування розробки проміжного програмного забезпечення базується на визначенні трудовитрат та вартості проєкту через кількісний аналіз його обсягу. Для розрахунку обрано базову конструктивну модель вартості СОСОМО [6], яка дозволяє оцінити ресурси на основі обсягу програмного коду.

Відповідно до класифікації моделі COCOMO, проєкт розробки інтеграційного API відноситься до типу *organic*, оскільки виконується одним фахівцем у знайомому середовищі (Java, Spring Boot) зі зрозумілими вимогами до безпеки та обробки даних.

На основі архітектурних рішень (контролери для обробки запитів, сервіси фільтрації викладачів та оцінок, HTTP-клієнт для взаємодії з Moodle Web Services), орієнтовний обсяг вихідного коду становить 1500 рядків. Розмір проєкту складає: $SIZE = 1,5$ KLOC (тисяч рядків коду). Згідно з моделлю COCOMO, трудомісткість розробки (E , людино-місяці) для органічного типу проєкту обчислюється за формулою 2.1:

$$E = a * SIZE^b \quad (2.1),$$

де $a = 2.4$, $b = 1.05$ – коефіцієнти для даного типу систем.

Отримаємо:

$$E = 2.4 * 1.5^{1.05} = 3.67 \text{ людино-місяці.}$$

Оптимальний календарний час розробки (T , місяці) розраховується за формулою 2.2:

$$T = c * E^d \quad (2.2),$$

де $c = 2.5$, $d = 0.38$.

Виконавши розрахунок, отримаємо:

$$T = 2.5 * 3.67^{0.38} = 4.1 \text{ місяця.}$$

Для розрахунку бюджету використовується медіанна заробітна плата Java-розробника в Україні. За актуальними аналітичними даними IT-ринку [7],

середня заробітна плата такого спеціаліста становить 109 500 грн на місяць. Загальна вартість розробки (Cost) дорівнює:

$$Cost = E * Salary = 3.67 * 109500 = 401\,865 \text{ грн.}$$

Економічний ефект від впровадження розробленого API досягається за рахунок автоматизації процесу перенесення оцінок із системи Moodle до відомостей Кампусу. Це дозволяє усунути ручну працю викладачів та мінімізувати помилки при мапінгу даних. Припустимо, що автоматизація економить у середньому 15 годин на рік для одного викладача. Для факультету зі штатом у 150 викладачів загальна економія робочого часу складе:

$$E_{\text{год}} = 150 * 15 = 2250 \text{ годин/рік.}$$

При середній вартості робочої години викладача 100 грн, річна економія коштів установи (E_p) становить:

$$E_p = 2250 * 100 = 225000 \text{ грн/рік}$$

Термін окупності проєкту ($T_{\text{ок}}$) розраховується як відношення вартості розробки до річної економії:

$$T_{\text{ок}} = \frac{Cost}{E_p} = \frac{401865}{225000} = 1.79 \text{ року (близько 21 місяця).}$$

2.6 Постановка завдання на розробку програмного забезпечення

Метою розробки є створення проміжного програмного забезпечення, яке забезпечить синхронізацію навчальних даних та зведених журналів

успішності між інформаційною системою «Електронний кампус» та середовищем дистанційного навчання Moodle.

Предметом розробки є серверний інтеграційний застосунок на базі Java та фреймворку Spring Boot без графічного інтерфейсу користувача. Застосунок виступатиме як шлюз даних, що приймає запити від системи «Електронний кампус», звертається до Moodle Web Services, витягує необхідні масиви інформації, проводить їхню трансформацію та повертає готовий результат у стандартизованому вигляді.

Для досягнення поставленої мети під час конструювання програмного забезпечення необхідно вирішити такі основні задачі:

- розробити структуру застосунку за принципами RESTful API;
- реалізувати механізм міжсервісної авторизації з використанням захищених токенів;
- створити надійний HTTP-клієнт для взаємодії з API Moodle, здатний отримувати каталоги дисциплін, структуру контрольних заходів та повні масиви оцінок;
- реалізувати алгоритми трансформації даних;
- розробити глобальний обробник виняткових подій та імплементувати механізми обмеження часу очікування;
- запакувати готове програмне забезпечення у Docker-контейнер для забезпечення його кросплатформності.

Висновки до розділу

У другому розділі було виконано розроблення та обґрунтування вимог до проміжного програмного забезпечення.

У підрозділі 2.1 за допомогою методології UML та побудови діаграми варіантів використання було змодельовано поведінку системи та визначено ключових акторів.

У підрозділі 2.2 на основі побудованих моделей було сформовано детальні функціональні вимоги, які охоплюють процеси безпечної авторизації,

автоматизованого вилучення оцінок з Moodle та їхньої подальшої трансформації. Повноту та узгодженість функціоналу було підтверджено розробленою матрицею трасування вимог.

У підрозділі 2.3 було сформовано нефункціональні вимоги до продуктивності, надійності, інформаційної безпеки та сумісності розроблюваного застосунку.

У підрозділі 2.4 виконано аналіз системних вимог, у межах якого визначено необхідні апаратні параметри та склад програмного забезпечення серверного середовища, необхідного для розгортання та ефективної експлуатації системи.

У підрозділі 2.5 за допомогою моделі COSOMO було проведено аналіз економічних показників програмного забезпечення.

За результатами розділу сформовано технічне завдання на розробку програмного забезпечення.

3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Архітектура програмного забезпечення

Для реалізації проміжного програмного обрано багатoshарову архітектуру у поєднанні з принципами побудови stateless сервісів. Такий підхід забезпечує слабку зв'язність компонентів системи, високу швидкість обробки запитів та спрощує масштабування інтеграційного шлюзу.

Опис архітектури розробленої системи виконано з використанням моделі C4.

Для опису систем використано перший рівень моделі (рис. 3.1).



Рисунок 3.1 – Діаграма C4 рівня 1

На найвищому рівні абстракції розроблене програмне забезпечення представляє собою інтеграційний шлюз, який слугує комунікаційним мостом між двома інформаційними платформами. Ініціатором бізнес-процесу виступає викладач, який через графічний інтерфейс системи «Електронний кампус» формує запит на синхронізацію навчальних відомостей. Система «Електронний кампус» виступає єдиним авторизованим клієнтом для розробленого API. Розроблене ПЗ формує запити до Moodle через механізм Moodle Web Services та повертає очищений результат.

На другому рівні деталізації архітектура системи представлена у вигляді єдиного виконуваного контейнера (рис. 3.2).

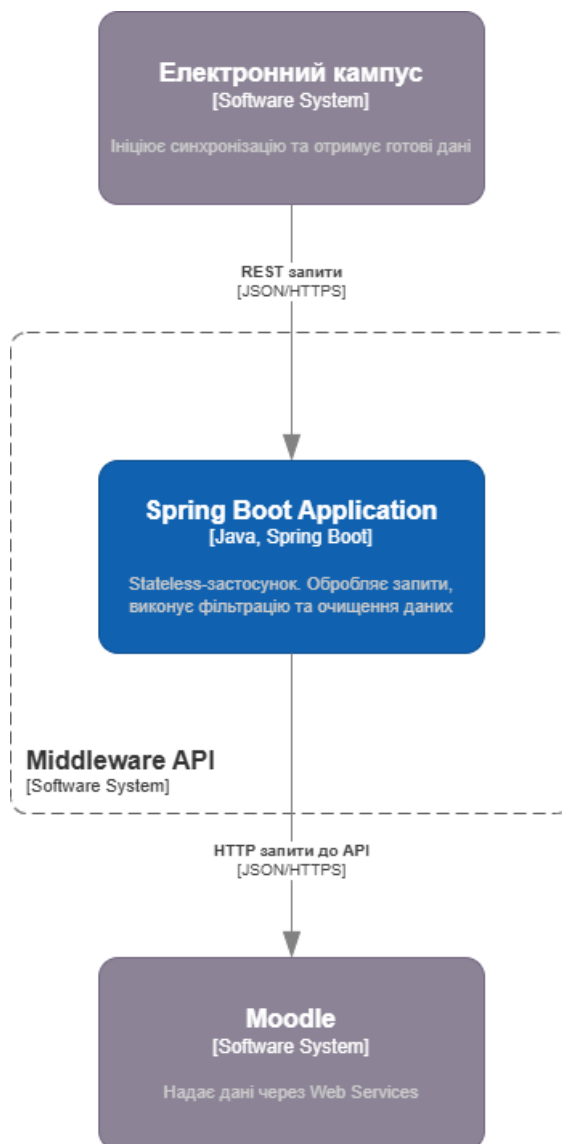


Рисунок 3.2 – Діаграма C4 рівня 2

Основним контейнером системи є Spring Boot Application — серверний застосунок на мові Java. Уся обробка даних (десеріалізація JSON, фільтрація масивів, очищення тексту від HTML-тегів) відбувається виключно в оперативній пам'яті під час виконання HTTP-запиту. Токени доступу та параметри конфігурації зберігаються у змінних оточення.

Для опису компонентів контейнера використовується третій рівень моделі. Діаграма C4 третього рівня знаходиться у графічному матеріалі, креслення 1.

Внутрішня структура Spring Boot застосунку декомпонована на функціональні компоненти за патерном Controller-Service-Client. Основними внутрішніми компонентами системи є:

- security filter: забезпечує авторизацію запитів від «Електронного кампусу» шляхом валідації API-ключів у заголовках HTTPS;
- integration controller: відповідає за прийом вхідних параметрів та маршрутизацію;
- integration service: ядро системи, де реалізовано алгоритми вибірки даних, їх фільтрації за ролями та мапінгу структур;
- html sanitizer: допоміжний сервіс для безпечного видалення HTML-розмітки з текстових полів;
- moodle API client: HTTP-клієнт для взаємодії з зовнішнім API Moodle.

3.2 Архітектурні рішення та обґрунтування вибору засобів розробки

Під час проєктування проміжного програмного забезпечення було прийнято та обґрунтовано низку стратегічних рішень, спрямованих на забезпечення безпеки, продуктивності та надійності інтеграції.

Оскільки застосунок виступає технічним шлюзом між двома системами, він не реалізує власну базу користувачів та механізм реєстрації. Прийнято рішення про використання міжсервісної авторизації на основі API-ключів

(Shared Secret). Система «Електронний кампус» ін'єктує токен у заголовок запиту, який валідується API. Для доступу до даних Moodle застосунок використовує системний токен Web Services.

Для взаємодії обрано архітектурний стиль REST через його універсальність та нативну підтримку в обох інтегрованих системах. Взаємодія здійснюється виключно за протоколом HTTPS, що гарантує шифрування даних під час передачі мережею.

Під час аналізу вимог було прийнято рішення щодо реалізації stateless-архітектури. Це означає, що застосунок не використовує жодних типів баз даних. Усі операції з фільтрації масивів та очищення HTML-розмітки виконуються в оперативній пам'яті під час обробки транзакційного запиту.

Надійність реалізована через механізм централізованої обробки виняткових ситуацій, що запобігає передачі внутрішніх помилок стеку на бік клієнта. Продуктивність забезпечується використанням неблокуючого HTTP-клієнта для паралельних запитів до Moodle.

Для вибору оптимальної мови програмування та серверного середовища для розробки проміжного програмного забезпечення було проведено порівняльний аналіз трьох найпопулярніших технологічних стеків: Node.js (Express) [8], Python (Fast API) [9] та Java (Spring Boot) [10].

Платформа Node.js (з використанням фреймворку Express) відрізняється високою продуктивністю під час обробки великої кількості одночасних мережових запитів завдяки асинхронній подієво-орієнтованій архітектурі (Event Loop). Проте її основним недоліком у контексті даного проєкту є динамічна типізація мови JavaScript. Відсутність жорсткого контролю типів значно підвищує ризик виникнення помилок під час десеріалізації та обробки непередбачуваних, багаторівневих JSON-структур, які генеруються зовнішньою системою Moodle.

Використання мови Python (на базі мікрофреймворку FastAPI) дозволяє суттєво пришвидшити процес написання коду завдяки лаконічності синтаксису та наявності інструментів валідації. Незважаючи на підтримку

анотацій типів, Python залишається динамічно типізованою мовою на етапі виконання програми. Крім того, наявність глобального блокування інтерпретатора обмежує можливості повноцінної багатопотокової обробки об'ємних масивів даних, що знижує пропускну здатність API під час пікових навантажень.

Платформа Java у поєднанні з фреймворком Spring Boot історично є світовим стандартом для розробки складних інтеграційних рішень. На відміну від попередніх варіантів, Java забезпечує строгу статичну типізацію на етапі компіляції, що гарантує абсолютну безпеку при проєктуванні моделей даних. Віртуальна машина Java (JVM) забезпечує високу продуктивність безперервної роботи завдяки механізмам JIT-компіляції та ефективному збиранню сміття, що ідеально підходить для stateless-архітектури.

Враховуючи необхідність забезпечення надійної обробки персональних даних та переваги статичної типізації при роботі зі складними структурами Moodle, для програмної реалізації Middleware API було обрано технологічний стек Java.

Використання Spring Boot дозволяє створювати самодостатні виконувані модулі із вбудованим сервером застосунків Apache Tomcat. Це критично важливо для спрощення подальшої контейнеризації шлюзу за допомогою Docker, оскільки забезпечує швидке розгортання в ізольованих середовищах без необхідності попереднього налаштування серверної інфраструктури.

Процес автоматизації збірки та управління залежностями проєкту реалізовано за допомогою інструменту Maven. Його вибір обґрунтований жорсткою стандартизацією життєвого циклу збірки та наявністю декларативного підходу до конфігурації. Це забезпечує передбачуваність процесу компіляції та зручну інтеграцію з плагінами Spring Boot для генерації виконуваних архівів.

3.3 Конструювання програмного забезпечення

Процес конструювання проміжного програмного забезпечення базується на принципах модульності та асинхронної обробки даних. Основна логіка системи зосереджена у сервісному шарі, де реалізовано трансформацію складних JSON-структур зовнішньої системи Moodle у структури, адаптовані для системи «Електронний кампус».

3.3.1 Розробка алгоритмів агрегації та трансформації даних

Центральним елементом конструювання є алгоритм формування журналу. Специфіка API Moodle полягає в тому, що дані про оцінки та дані про викладачів, які їх виставили, знаходяться у різних вузлах системи, що вимагає розробки багатокрокового процесу агрегації.

Алгоритм обробки даних включає наступні етапи:

- а) запит та десеріалізація:
 - 1) виконується отримання повного масиву оцінок;
 - 2) для ефективної роботи з динамічними полями використовується ієрархія об'єктів JsonNode;
- б) пошук ініціатора оцінювання: для кожного елемента типу завдання алгоритм ініціює додатковий мережевий запит (це дозволяє ідентифікувати унікальний номер викладача, який перевіряв роботу);
- в) нормалізація:
 - 1) перед включенням текстових даних у фінальний об'єкт виконується очищення від HTML-тегів;
 - 2) для цього використано алгоритм на основі регулярних виразів `replaceAll("<[^>]*>", "")`, що дозволяє швидко видаляти розмітку без залучення сторонніх парсерів;
- г) групування: за допомогою структур `HashMap` та `ArrayNode` дані розподіляються за ідентифікаторами викладачів, формуючи персоналізовані колонки журналу для кожного користувача.

Логіку процесу обробки даних та прийняття рішень наведено на UML-діаграмі діяльності (рисунок 3.3).

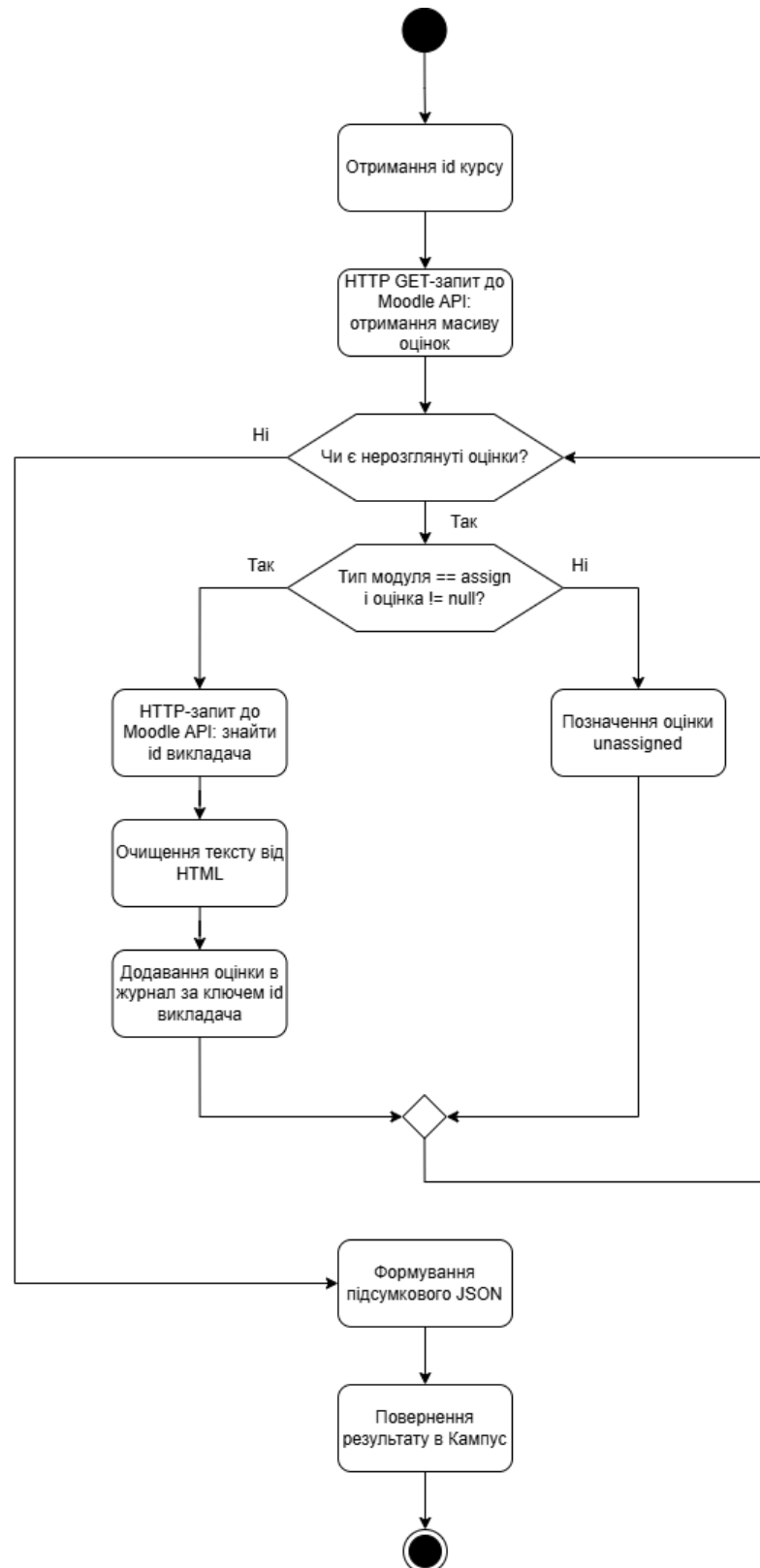


Рисунок 3.3 – Діаграма діяльності алгоритму формування журналу

3.3.2 Опис програмних структур та моделей даних

Під час конструювання ПЗ використано сучасні програмні структури мови Java, а саме:

- `java records`: для опису моделей обміну даними використано `immutable` структури `record`, це дозволяє автоматично генерувати методи доступу та гарантує незмінність даних під час їх агрегації;
- динамічні об'єкти JSON: використання класів `ObjectNode` та `ArrayNode` бібліотеки Jackson дозволяє гнучко маніпулювати структурами відповідей Moodle, додаючи обчислені поля без необхідності створення статичних моделей для кожного варіанту відповіді;
- типізовані колекції: для групування даних використано `LinkedHashMap`, що забезпечує збереження порядку слідування колонок журналу, який було встановлено викладачем у системі Moodle.

Статичну структуру компонентів системи та моделей передачі даних (DTO) наведено на UML-діаграмі класів (рис. 3.4).

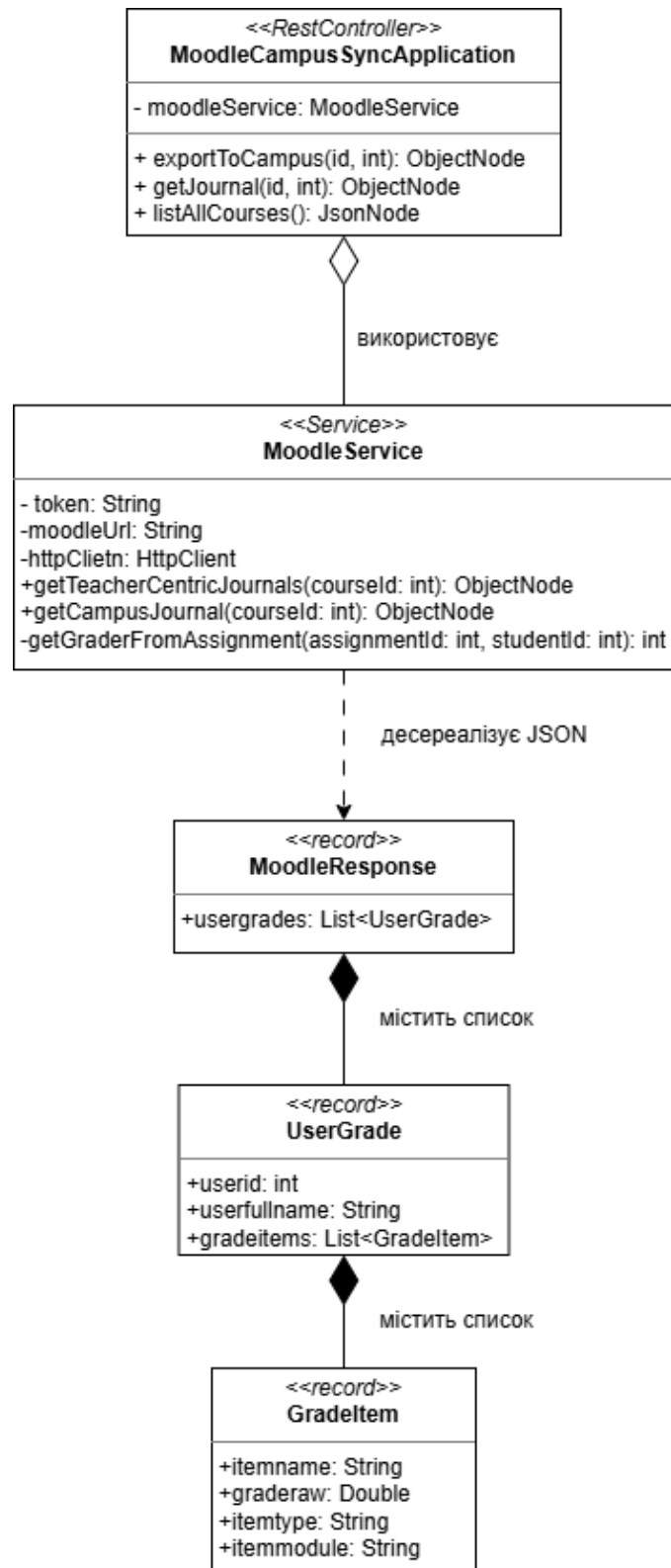


Рисунок 3.4 – Діаграма класів

3.3.3 Опис структури бази даних

Проміжне програмне забезпечення розроблене як stateless система, що не потребує власного постійного сховища. Проте важливим етапом конструювання є забезпечення відповідності вихідних структур даних до логічної моделі бази даних системи «Електронний кампус». Для цього було проаналізовано фрагмент бази даних Кампусу, з якою безпосередньо взаємодіє розроблений API.

Опис таблиць бази даних, що використовуються для збереження даних, що надходять з Moodle через API, наведено у таблицях 3.1-3.4. Модель бази даних наведена у графічному матеріалі, креслення 2.

Таблиця 3.1 – Опис таблиці cmonitoringemployee

Назва поля	Тип даних	Опис
cMonitoringEmployeeId	int	унікальний ідентифікатор журналу викладача (PK)
cMonitoringId	int	посилання на загальний моніторинг
eEmployees1Id	int	ідентифікатор викладача у системі
Name	varchar	назва навчальної дисципліни або конкретної відомості
vcActuality	char	ознака актуальності запису в системі
vcChangeDate	datetime	дата та час останнього редагування журналу

Таблиця 3.2 — Опис таблиці cmonitoringcolumn

Назва поля	Тип даних	Опис
cMonitoringColumnId	int	унікальний ідентифікатор колонки (контрольного заходу) (PK)
cMonitoringEmployeeId	int	посилання на запис у таблиці cmonitoringemployee (FK)
DcLoadId	int	ідентифікатор навчального навантаження
DateMark	datetime	календарна дата проведення контрольного заходу
Description	varchar	назва завдання
vcActuality	char	ознака актуальності запису
vcChangeDate	datetime	дата та час останньої зміни колонки
IsNotSum	char	ознака того, чи враховується колонка у підсумкову суму балів

Таблиця 3.3 — Опис таблиці cmonitoringstudent

Назва поля	Тип даних	Опис
cMonitoringStudentId	int	унікальний ідентифікатор студента в журналі (PK)
cMonitoringEmployeeId	int	посилання на запис у таблиці cmonitoringemployee (FK)
sPersonalityId	int	ідентифікатор студента у загальній базі
vcActuality	char	ознака актуальності запису
vcChangeDate	datetime	дата та час останньої зміни

Таблиця 3.4 — Опис таблиці cmonitoringrow

Назва поля	Тип даних	Опис
cMonitoringRowId	int	унікальний ідентифікатор запису про оцінку (PK)
cMonitoringColumnId	int	посилання на колонку завдання таблиці cmonitoringcolumn (FK)
sPersonalityId	int	ідентифікатор особи студента (FK)
Mark	float	числове значення отриманої оцінки або балу
Description	varchar	додатковий текстовий коментар до оцінки
vcActuality	char	ознака актуальності запису
vcChangeDate	datetime	дата та час виставлення або зміни оцінки
Presence	varchar	відмітка про присутність студента на занятті
cMonitoringStudentId	int	посилання на запис у таблиці cmonitoringstudent (FK)

3.3.4 Опис стороннього програмного забезпечення

Опис утиліт, бібліотек та іншого стороннього програмного забезпечення, що використовується у розробці наведено в таблиці 3.5.

Таблиця 3.5 – Опис утиліт

№ п/п	Назва утиліти	Опис застосування
1	VS Code	Головне середовище розробки програмного забезпечення серверної частини курсової роботи.
2	Git	Система контролю версій
3	Postman	Програмне забезпечення необхідне для тестування REST запитів. Використовувалось для тестування API.
4	Apache Maven	Засіб автоматизації збірки проєктів на основі Java. Використовувався для централізованого управління залежностями та компіляції виконуваного JAR-файлу.
5	Spring Boot Starter Web	Базовий фреймворк для створення RESTful веб-сервісів. Забезпечує маршрутизацію HTTP-запитів, інверсію управління та автоматичне налаштування вбудованого веб-сервера.
6	Jackson	Бібліотека для серіалізації та десеріалізації об'єктів. Використовувалась для перетворення відповідей Moodle API у деревоподібні структури JsonNode та Records.

Продовження таблиці 3.5

№ п/п	Назва утиліти	Опис застосування
7	Java HTTP Client	Стандартний мережевий клієнт. Використовувався для виконання асинхронних GET/POST запитів до зовнішніх систем.

Тексти програмного коду наведені в окремому документі «Текст програми».

3.4 Аналіз безпеки даних

Процес управління вразливостями базується на превентивному підході до захисту від поширених веб-загроз. Оскільки застосунок не формує запитів до власної реляційної бази, ризик класичних SQL-ін'єкцій у ньому відсутній. Натомість основну увагу приділено захисту від міжсайтового скриптингу (XSS) шляхом імплементації механізму санітизації текстових даних. Усі описові поля, що надходять з Moodle API, проходять фільтрацію за допомогою регулярних виразів. Застосування анотацій Spring Boot унеможливорює передачу непередбачених типів даних у маршрутах, що автоматично блокується фреймворком.

Автентифікаційний токен доступу до зовнішніх сервісів Moodle не зберігається у вихідному коді застосунку. Його значення безпечно впроваджується через змінні середовища за допомогою механізму інверсії управління.

Безпека транспортного рівня забезпечується політикою виконання мережевих викликів. Усі звернення від інтеграційного шлюзу ініціюються через вбудований клієнт `java.net.http.HttpClient` виключно за захищеним протоколом HTTPS із використанням сучасних алгоритмів шифрування сімейства TLS. Це гарантує конфіденційність трафіку та захищає систему від атак типу «людина посередині» (Man-in-the-Middle) під час міграції оцінок.

Крім того, у застосунку налаштовано обробку виняткових ситуацій: у разі виникнення таймаутів з'єднання або логічних помилок клієнту повертаються лише стандартизовані HTTP-статуси.

Усі процеси агрегації та трансформації масивів оцінок відбуваються виключно в оперативній пам'яті сервера. Відразу після успішної відправки сформованої відповіді до системи «Електронний кампус» посилання на об'єкти вивільняються, і транзитні дані знищуються збирачем сміття. Такий архітектурний підхід зводить до мінімуму ризику компрометації конфіденційної інформації навіть у випадку потенційного несанкціонованого доступу до операційної системи сервера застосунку.

Висновки до розділу

У третьому розділі було проведено конструювання та реалізацію програмного забезпечення.

У підрозділі 3.1 представлено архітектуру програмного забезпечення, побудовану за архітектурним стилем REST та принципами stateless взаємодії. Описано логіку функціонування Middleware API.

У підрозділі 3.2 обґрунтовано вибір технологічних рішень для реалізації проєкту. Визначено доцільність використання мови Java та фреймворку Spring Boot.

У підрозділі 3.3 розроблено програмну структуру застосунку, включаючи алгоритми трансформації складних JSON-структур та формування журналів успішності. Визначено основні сутності на основі Java Records та ієрархію об'єктів JsonNode, що було візуалізовано за допомогою UML-діаграми класів, блок-схеми алгоритму та ER-моделі цільової бази даних. Також наведено опис сторонніх бібліотек та утиліт.

У підрозділі 3.4 виконано аналіз безпеки даних.

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Аналіз якості ПЗ

Метриками для оцінки якості ПЗ обрано наступні:

- цикломатична складність;
- надійність коду;
- підтримуваність коду;
- безпечність коду.

Для аналізу якості коду було обрано статичний аналізатор SonarQube [11]. На рисунку 4.1 зображено повний звіт після аналізу коду.

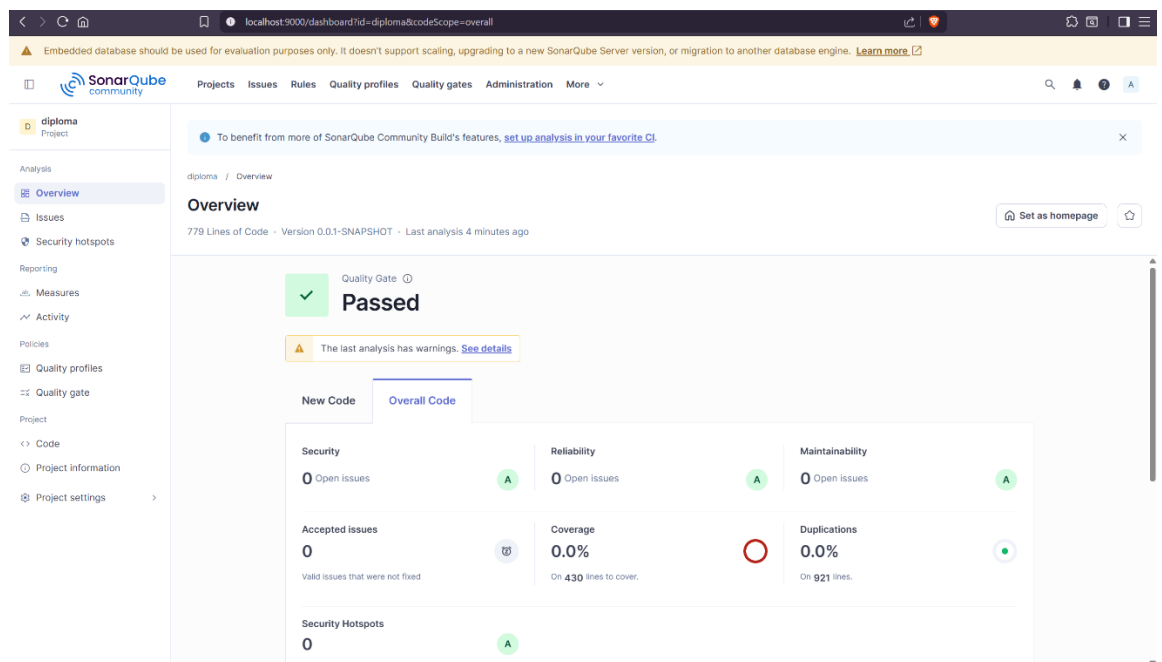


Рисунок 4.1 – Звіт аналізатора коду

З рисунку 4.1 видно, що проєкт проходить тести якості. Метрики безпечність (security), надійність (reliability) та підтримуваність (maintainability) отримали максимальні бали, помилок не виявлено.

Показники цикломатичної складності зображено на рисунку 4.2.

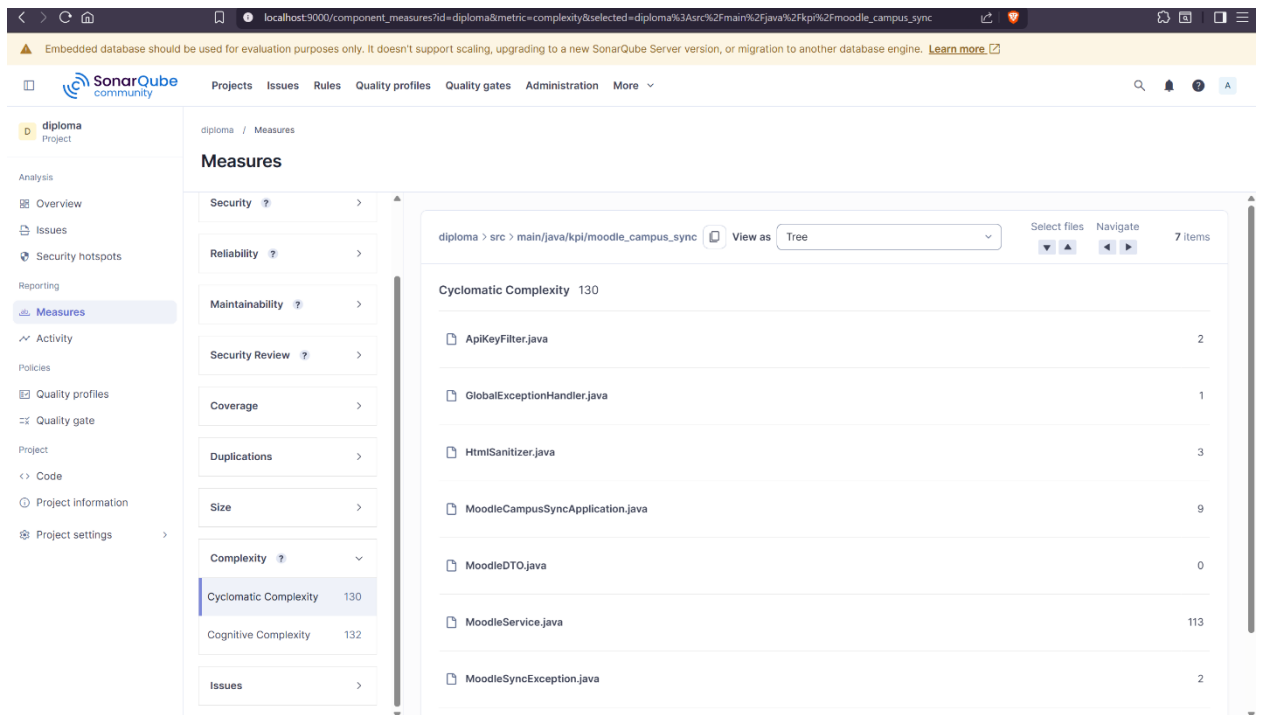


Рисунок 4.2 – Звіт про цикломатичну складність застосунку

З рисунку 4.2 видно, що загальна цикломатична складність становить 130. Найнижчі показники складності мають класи передачі даних та обробки помилок. Зокрема, компонент MoodleDTO.java (CC = 0) має нульову складність, оскільки виконує виключно роль контейнера для зберігання даних і не містить жодних розгалужень чи бізнес-логіки. Класи GlobalExceptionHandler.java (CC = 1) та MoodleSyncException.java (CC = 2) також мають мінімальні значення, вони описують систему обробки виняткових ситуацій.

Фільтр авторизації ApiKeyFilter.java (CC = 2) та компонент для очищення вхідного тексту HtmlSanitizer.java (CC = 3) мають низку складність, що гарантує максимальну швидкодію цих вузлів під час обробки кожного вхідного HTTP-запиту.

Головний контролер застосунку MoodleCampusSyncApplication.java має складність 9 одиниць. Він не виконує жодних самостійних обчислень, а виключно приймає мережеві запити і делегує їх виконання сервісному рівню.

Уся основна логіка сконцентрована у класі MoodleService.java, загальна складність якого становить 113 одиниць. Цей клас відповідає за всі етапи ETL-

процесу та інкапсулює всі бізнес-логіку програми. Він складається з 33 окремих методів, середня складність одного методу становить 3.42 одиниці. Отже, архітектура розробленого сервісу відповідає критеріям “чистого коду” та є передбачуваною.

4.2 Опис процесів тестування

Тестування виконується згідно документу «Програма та методика тестування».

Оскільки архітектура розробленої системи базується на тісній мережевій взаємодії зі стороннім сервісом та обміні даними у форматі JSON, основним інструментом для перевірки ендпоінтів було обрано середовище Postman [12]. Застосування мануального підходу до інтеграційного тестування є доцільним, оскільки воно дозволяє в реальному часі оцінити коректність мережових запитів, валідність агрегованих даних та реакцію системи на реальні затримки відповідей зовнішнього сервера.

Було виконане мануальне тестування програмного забезпечення, опис відповідних тестів наведено у таблицях 4.1 – 4.14.

Таблиця 4.1 – Тест 1.1 Отримання списку курсів

Початковий стан системи	Сервер запущено. Moodle API доступний. У конфігурації вказано дійсний токен авторизації.
Вхідні дані	GET-запит на ендпоінт /courses із валідним API-ключем у заголовку.
Опис проведення тесту	Виконується запит для отримання переліку всіх доступних дисциплін. Система звертається до Moodle, отримує дані та очищує їх.
Очікуваний результат	HTTP Статус 200 ОК. Повертається JSON-масив з об'єктами курсів.

Продовження таблиці 4.1

Фактичний результат	Збігається з очікуванням.
---------------------	---------------------------

Таблиця 4.2 – Тест 1.2 Отримання списку викладачів

Початковий стан системи	Існує курс з id = 5, до якого в Moodle прив'язані викладачі.
Вхідні дані	GET-запит на ендпоінт /course/5/teachers із валідним ключем.
Опис проведення тесту	Система запитує всіх зарахованих на курс користувачів та фільтрує їх, залишаючи у вибірці лише тих, чия роль містить ключове слово "teacher".
Очікуваний результат	HTTP Статус 200 ОК. Повертається JSON з масивом викладачів.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.3 – Тест 1.3 Отримання контрольних заходів курсу

Початковий стан системи	У курсі id = 5 створено завдання типу assign.
Вхідні дані	GET-запит на ендпоінт /course/5/activities.
Опис проведення тесту	Виконується запит для отримання структури контрольних заходів курсу. З усієї структури відбираються лише модулі, що підлягають оцінюванню.

Продовження таблиці 4.3

Очікуваний результат	HTTP Статус 200 ОК. Повертається масив контрольних заходів із назвами завдань та їхньою максимальною оцінкою.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.4 – Тест 1.4 Формування загального журналу

Початковий стан системи	У курсі id = 5 є студенти, яким виставлено оцінки за виконані завдання.
Вхідні дані	GET-запит на ендпоінт /course/5/journal.
Опис проведення тесту	Система завантажує дані курсу, користувачів та їхні оцінки, після чого агрегує їх у зведену таблицю, де стовпцями є завдання, а рядками — оцінки конкретних студентів.
Очікуваний результат	HTTP Статус 200 ОК. JSON-відповідь містить масив columns, у кожному з яких є масив marks із деталями оцінювання.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.5 – Тест 1.5 Експорт журналів згрупованих за викладачами

Початковий стан системи	Оцінки в курсі id = 5 виставлялися різними викладачами.
Вхідні дані	GET-запит на ендпоінт /course/10/export-campus.

Продовження таблиці 4.5

Опис проведення тесту	Формується вибірка оцінок із групуванням за id викладача, що перевіряв завдання.
Очікуваний результат	HTTP Статус 200 ОК. JSON-відповідь містить масив, де оцінки розділені по ідентифікаторах викладачів.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.6 – Тест 1.6 Експорт оцінок за списком email

Початковий стан системи	Файл students.txt містить валідні електронні адреси зареєстрованих студентів.
Вхідні дані	GET-запит на ендпоінт /export-students.
Опис проведення тесту	Система зчитує email-адреси з локального файлу, шукає відповідних користувачів через API Moodle та збирає всі їхні оцінки з усіх активних курсів.
Очікуваний результат	HTTP Статус 200 ОК. Повертається масив студентів із їхніми оцінками.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.7 – Тест 1.7 Невалідний або відсутній ключ при авторизації

Початковий стан системи	Сервер працює, налаштовано компонент безпеки ApiKeyFilter.
Вхідні дані	GET-запит на будь-який ендпоінт без передачі заголовка або з невалідним з API-ключем.

Продовження таблиці 4.7

Опис проведення тесту	Симулюється спроба несанкціонованого доступу до закритого API.
Очікуваний результат	HTTP Статус 401 Unauthorized. Доступ заблоковано, бізнес-логіка не виконується, дані не повертаються.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.8 – Тест 1.8 Невалідний ключ Moodle API

Початковий стан системи	Сервер працює. У конфігураційному файлі системи вказано невалідний токен доступу до Moodle API.
Вхідні дані	GET-запит на будь-який ендпоінт.
Опис проведення тесту	Система формує HTTP-запит до сервера Moodle, підставляючи невалідний wstoken.
Очікуваний результат	HTTP Статус 200 OK. Сервер Moodle повертає JSON із повідомленням про помилку авторизації.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.9 – Тест 1.9 Запит даних неіснуючого курсу

Початковий стан системи	Сервер працює.
Вхідні дані	GET-запит на ендпоінт /course/99999/journal.

Продовження таблиці 4.9

Опис проведення тесту	Перевірка стійкості сервісу на обробку порожніх або невалідних відповідей від зовнішнього API.
Очікуваний результат	HTTP Статус 200 ОК. Структура JSON повертається без помилок, масиви даних порожні, а поле <code>course_name</code> містить значення "Unknown Course".
Фактичний результат	Збігається з очікуванням.

Таблиця 4.10 – Тест 1.10 Студент відсутній у базі

Початковий стан системи	Сервер працює. Файл <code>students.txt</code> містить електронну адресу, якої немає в базі Moodle.
Вхідні дані	GET-запит на ендпоінт <code>/export-students</code> .
Опис проведення тесту	Тест перевіряє, чи не зупиниться обробка всього списку через одного невалідного користувача.
Очікуваний результат	HTTP Статус 200 ОК. Для помилкового email повертається об'єкт <code>{"status": "error", "message": "User not found"}</code> . Інші студенти обробляються успішно.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.11 – Тест 1.11 Відсутність групи у студента

Початковий стан системи	У курсі є зареєстрований студент, який не прикріплений до жодної групи.
Вхідні дані	GET-запит на ендпоінт <code>/course/5/journal</code> .

Продовження таблиці 4.11

Опис проведення тесту	Перевірка логіки призначення значень за замовчуванням.
Очікуваний результат	Поле <code>student_group</code> приймає значення "Без групи".
Фактичний результат	Збігається з очікуванням.

Таблиця 4.12 – Тест 1.12 Ігнорування завдань без оцінок

Початковий стан системи	Студент відправив роботу в Moodle, але викладач ще не виставив оцінку (поле <code>graderaw</code> = null).
Вхідні дані	GET-запит на ендпоінт <code>/course/5/journal</code> .
Опис проведення тесту	Перевірка фільтрації порожніх даних, щоб не передавати неперевірені завдання.
Очікуваний результат	Завдання без оцінки ігнорується парсером і не додається до масиву <code>marks</code> для даного студента.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.13 – Тест 1.13 Перевірка очищення HTML

Початковий стан системи	У Moodle викладач залишив відгук до оцінки із шкідливим кодом: <code><script>alert('xss')</script></code> Відмінно!
Вхідні дані	GET-запит на ендпоінт <code>/course/5/journal</code> .

Продовження таблиці 4.13.

Опис проведення тесту	Перевірка роботи компонента HtmlSanitizer для запобігання XSS-ін'єкціям під час передачі текстових даних у Кампус.
Очікуваний результат	HTML-теги повністю вирізані регулярним виразом.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.14 – Обробка недоступності Moodle API

Початковий стан системи	Moodle API заблоковано (імітація падіння зовнішнього сервера або відсутності інтернет-з'єднання).
Вхідні дані	GET-запит на будь-який ендпоінт.
Опис проведення тесту	Перевірка роботи компонента GlobalExceptionHandler під час недоступності стороннього сервісу.
Очікуваний результат	HTTP Статус 500 Internal Server Error. Система перехоплює виняток і повертає JSON із повідомленням “Сталася внутрішня помилка сервера: Помилка виконання HTTP запиту”.
Фактичний результат	Збігається з очікуванням.

4.3 Опис контрольного прикладу

Користувач має дійсний сервер Moodle, на якому створює курси, контрольні заходи, на курси реєструє викладачів і студентів, студенти мають оцінки за завдання (рисунки 4.3 – 4.6).

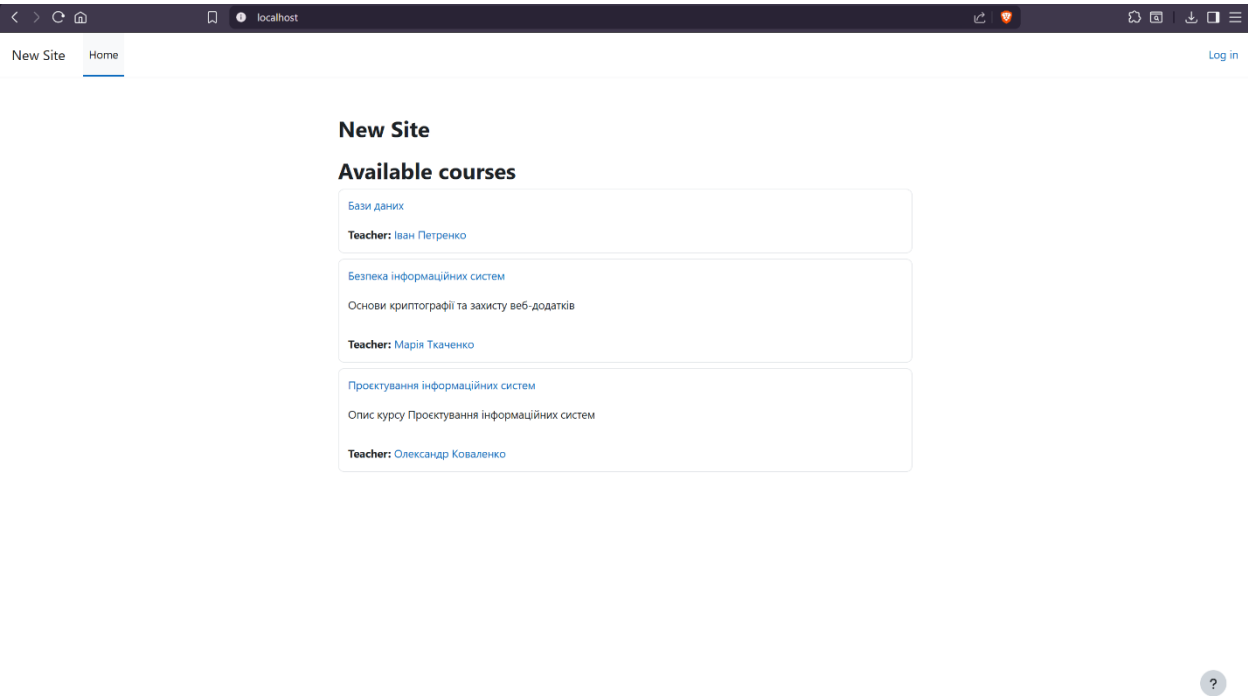


Рисунок 4.3 – Головна сторінка системи Moodle

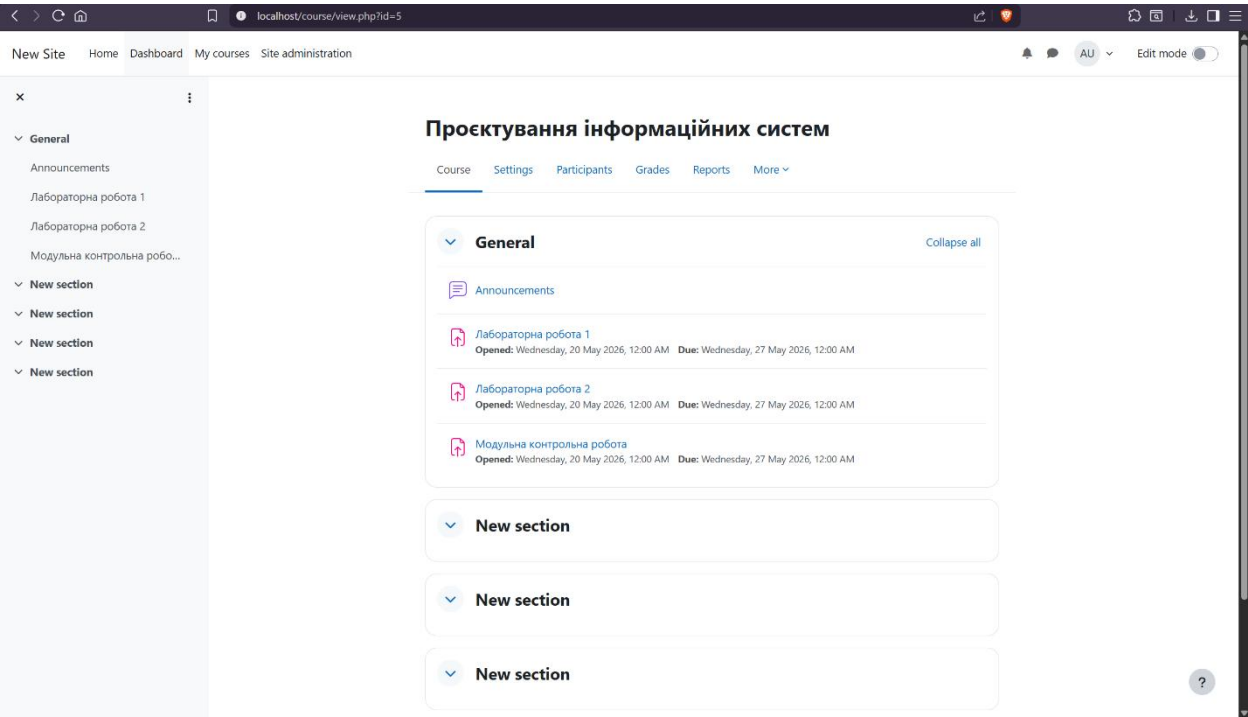


Рисунок 4.4 – Сторінка курсу

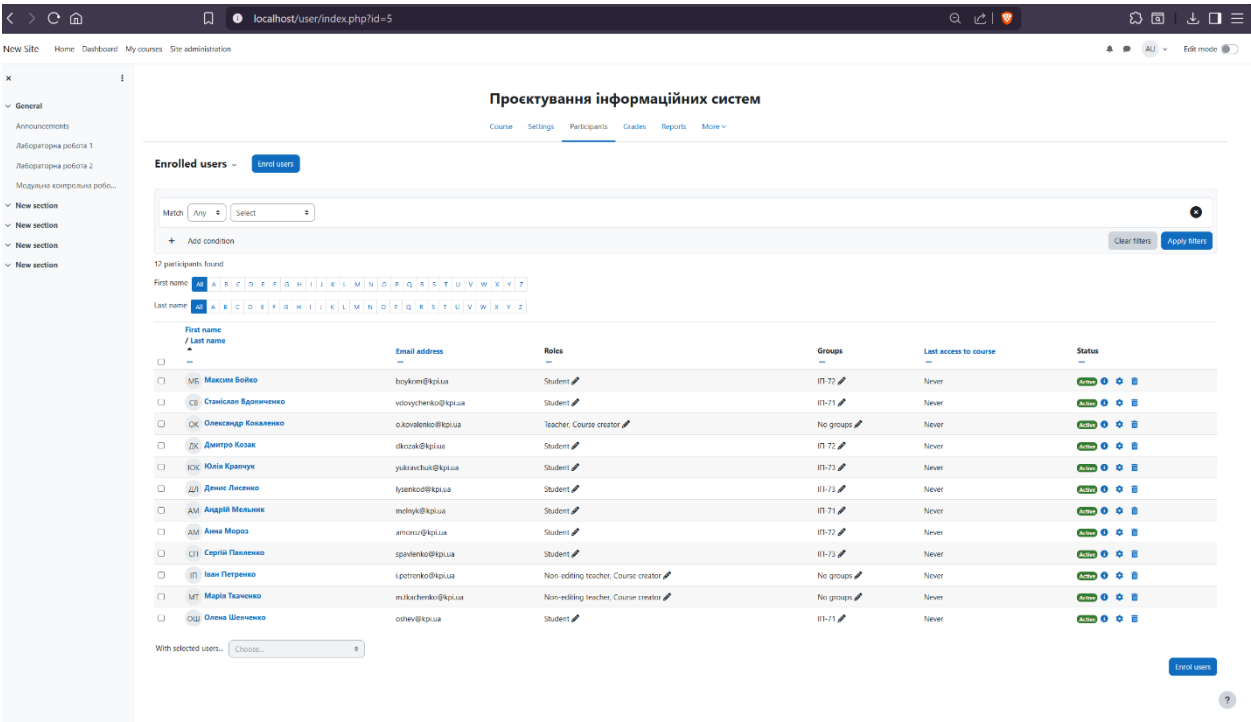


Рисунок 4.5 – Список викладачів та студентів курсу

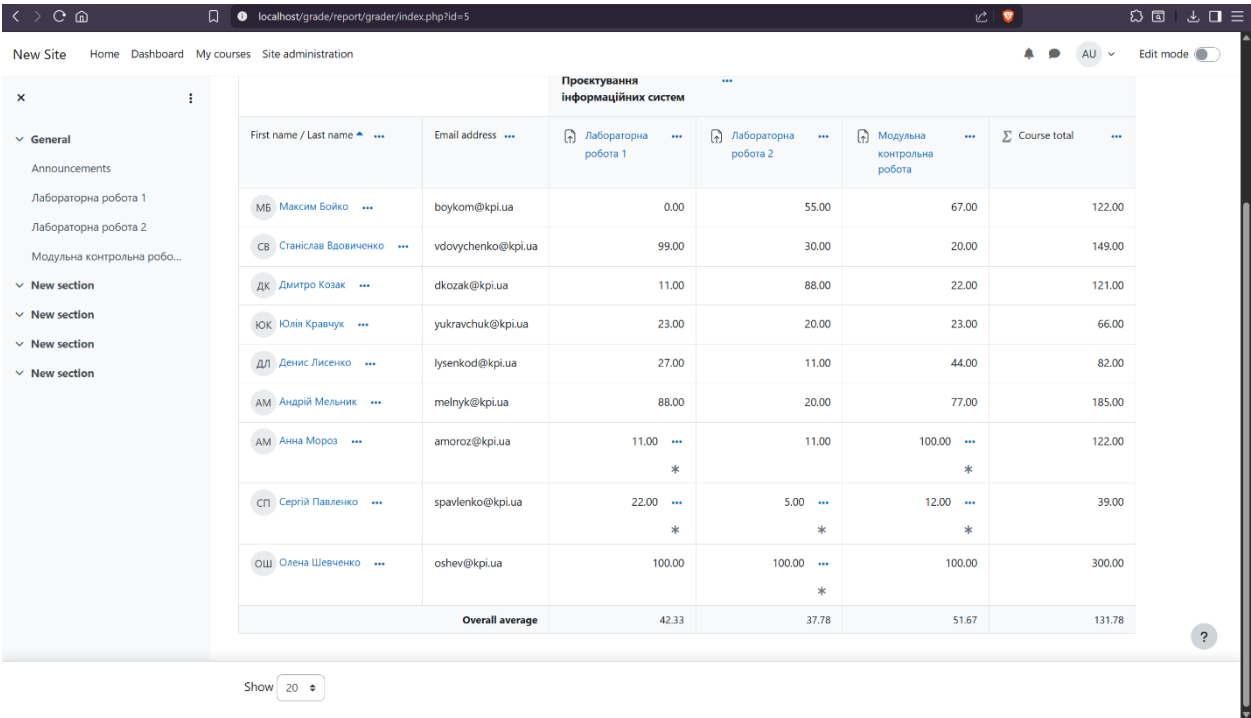


Рисунок 4.6 – Список оцінок студентів

Далі користувач використовуючи ключ до Moodle API та ключ для авторизації робить HTTP запити (для прикладу буде використана платформа для тестування API Postman).

Користувач отримує список курсів за ендпоінтом /courses (рисунк 4.7).

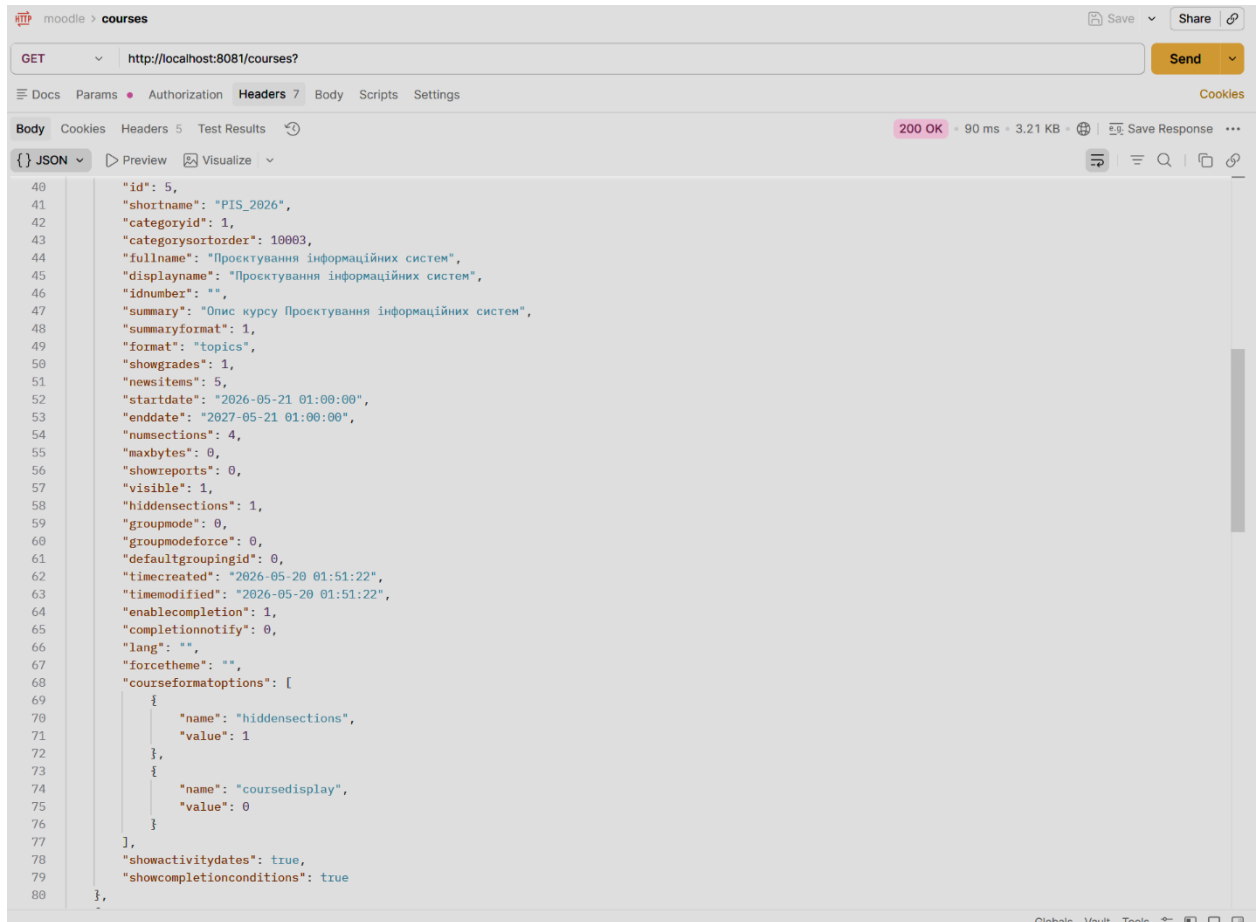


Рисунок 4.7 – Отриманий список курсів

Далі користувач отримує id бажаного курсу (для прикладу $id = 5$) та може зробити запит для отримання викладачів курсу /course/5/teachers (рисунк 4.8).

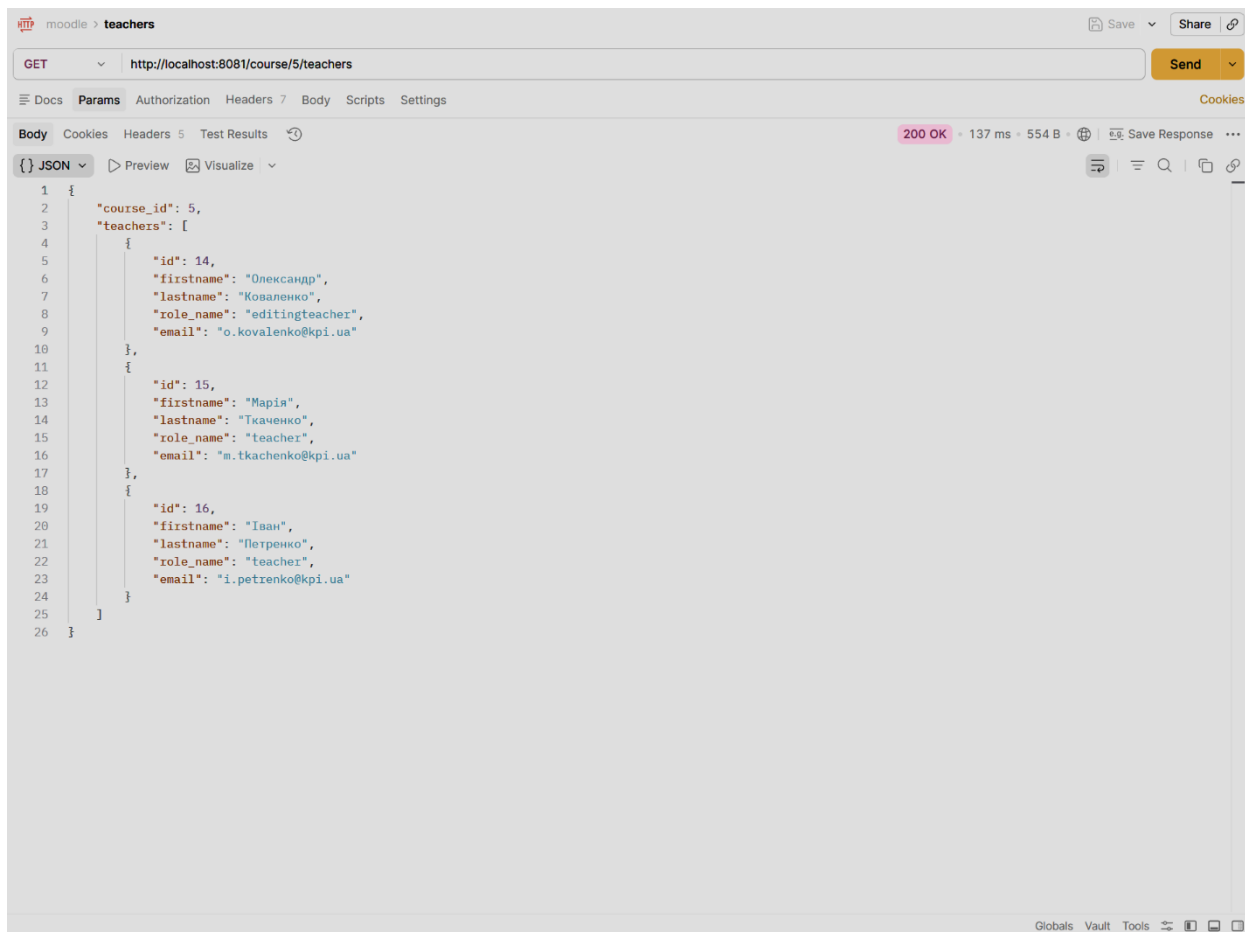


Рисунок 4.8 – Отриманий список викладачів

Користувач може зробити запит для отримання списку контрольних заходів курсу `/course/5/activities` (рисунок 4.9).

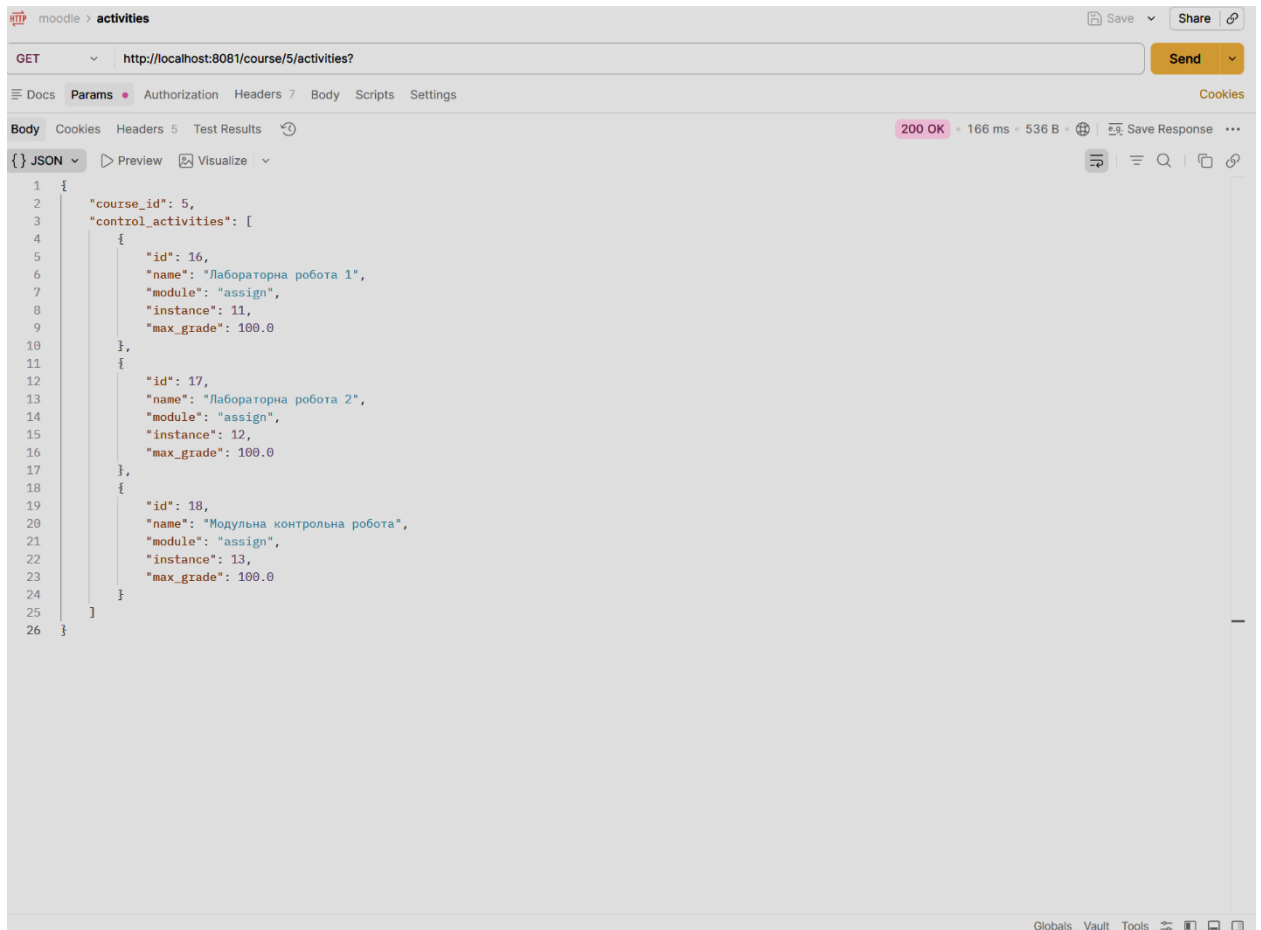


Рисунок 4.9 – Список отриманих контрольних заходів курсу

Користувач може зробити запит для отримання журналу оцінок (відсортований за контрольними заходами) `/course/5/journal` (рисунок 4.10).

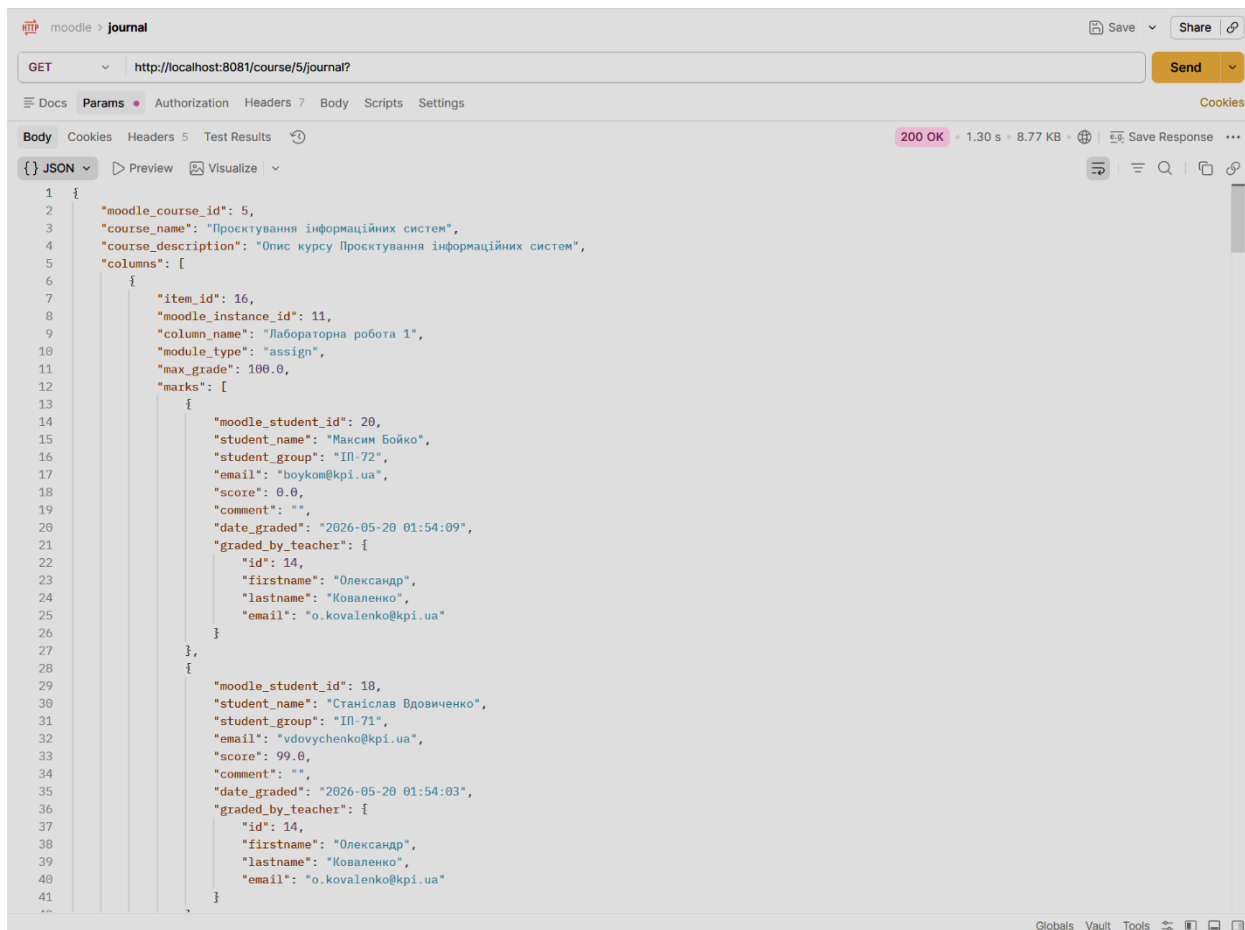


Рисунок 4.10 – Отриманий журнал оцінок (відсортований за контрольними заходами)

Далі користувач може зробити запит для отримання відформатованого для відомості Кампусу JSON `/course/5/export-campus` (рисунок 4.11).

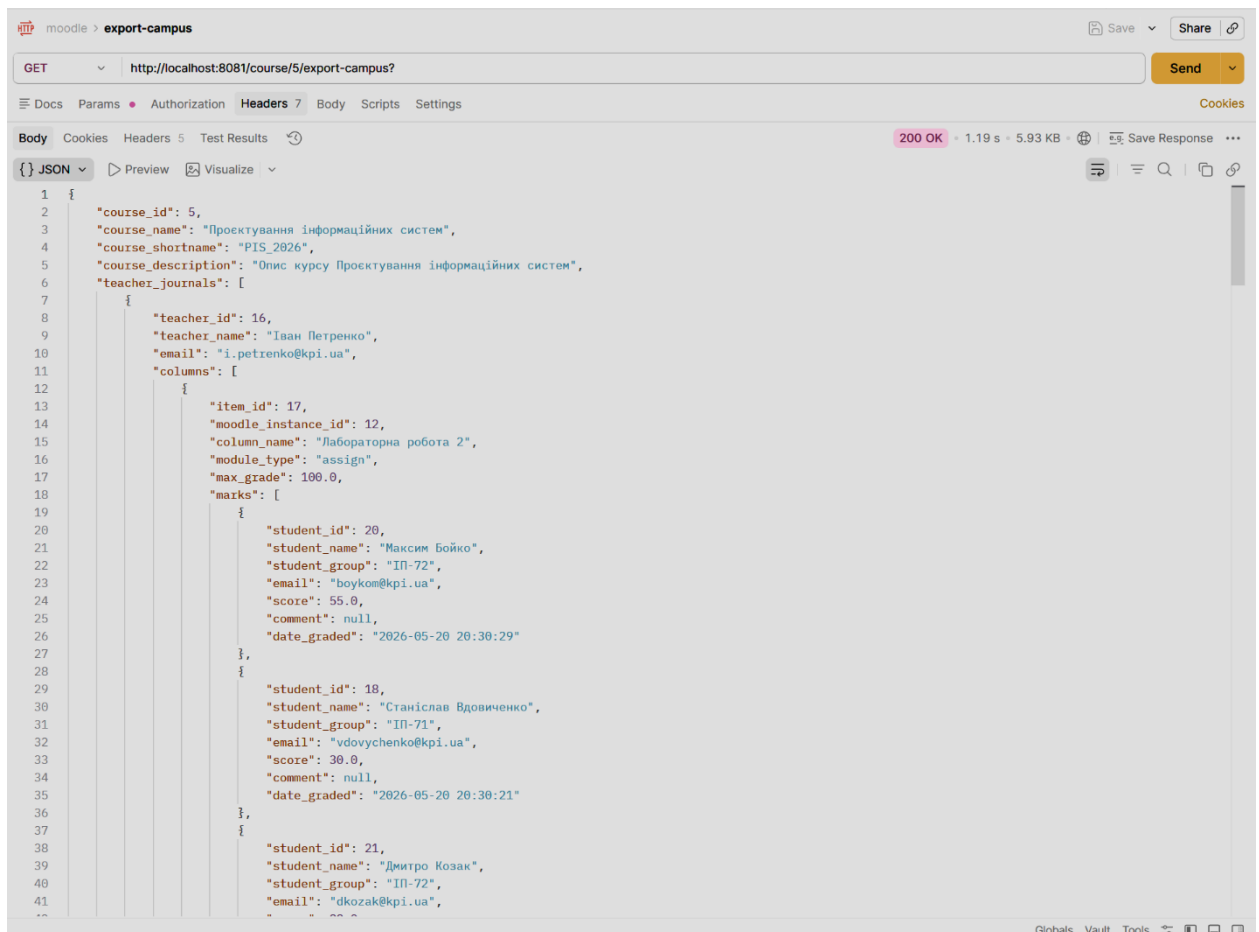


Рисунок 4.11 – Отриманий журнал оцінок (відсортований за викладачами)

Висновки до розділу

У четвертому розділі було проведено оцінку якості, тестування та демонстрацію роботи розробленого програмного забезпечення.

У підрозділі 4.1 виконано аналіз якості вихідного коду за допомогою платформи статичного аналізу SonarQube. Оцінено ключові метрики надійності та підтримуваності, також проаналізовано цикломатичну складність компонентів системи, що підтвердило відповідність архітектури стандартам чистого коду.

У підрозділі 4.2 описано процеси мануального інтеграційного тестування REST API за допомогою середовища Postman.

У підрозділі 4.3 наведено повний опис контрольного прикладу роботи застосунку.

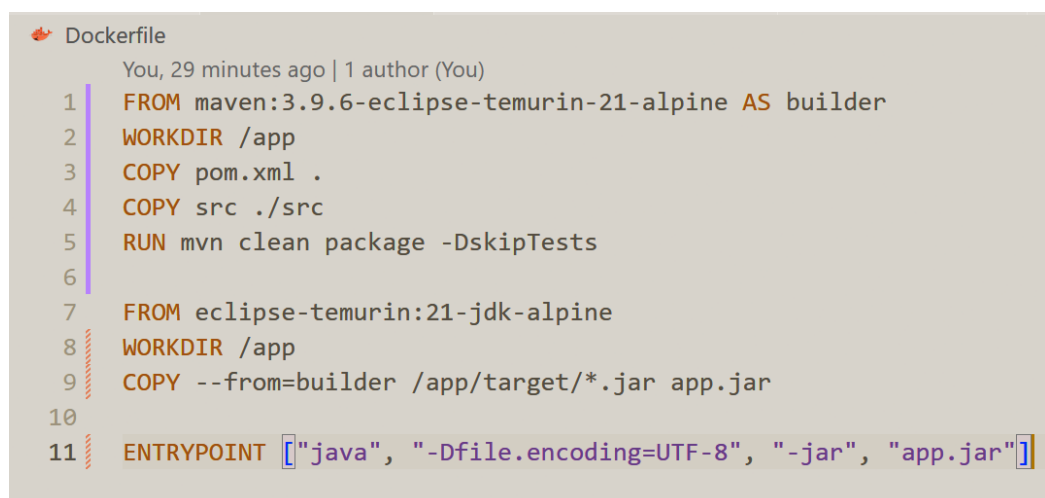
5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

5.1 Розгортання програмного забезпечення

Для розгортання додатку було обрано технологію контейнеризації Docker [13]. Оскільки застосунок функціонує як незалежний мікросервіс, його життєвий цикл та процес розгортання є повністю ізольованими від самої платформи Moodle. Це дозволяє розгортати модуль у будь-якому хмарному або локальному середовищі, де є мережевий доступ до цільової системи.

Для забезпечення гнучкості налаштування всі динамічні параметри системи, такі як мережева адреса Moodle API, секретні токени та ключі доступу до створеного API, винесені на рівень змінних середовища. Це гарантує високий рівень безпеки, оскільки секретні дані не зберігаються у статичному вигляді у скомпільованому архіві чи системі контролю версій. Управління контейнером та інжекція конфігурацій реалізується за допомогою інструменту оркестрації Docker Compose [14].

Для деплою системи на цільовому сервері адміністратору достатньо мати встановлене середовище Docker. Процес компіляції та створення артефакту делеговано самому Docker-рушію. Для цього розроблено двоетапний маніфест Dockerfile, наведений на рисунку 5.1.

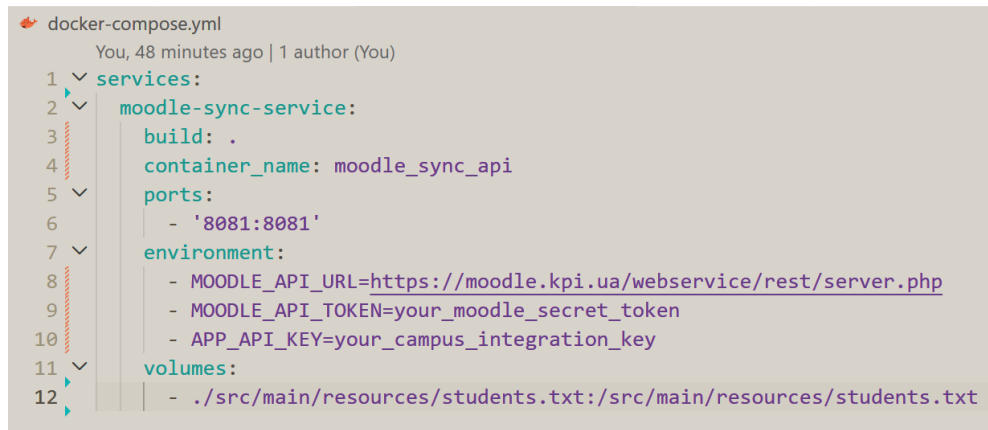


```
Dockerfile
You, 29 minutes ago | 1 author (You)
1 FROM maven:3.9.6-eclipse-temurin-21-alpine AS builder
2 WORKDIR /app
3 COPY pom.xml .
4 COPY src ./src
5 RUN mvn clean package -DskipTests
6
7 FROM eclipse-temurin:21-jdk-alpine
8 WORKDIR /app
9 COPY --from=builder /app/target/*.jar app.jar
10
11 ENTRYPOINT ["java", "-Dfile.encoding=UTF-8", "-jar", "app.jar"]
```

Рисунок 5.1 – Вміст файлу Dockerfile

На першому етапі завантажується середовище Maven для компіляції коду, на другому - формується образ на базі eclipse-temurin:21-jdk-alpine для виконання готового .jar файлу.

Для запуску додатка адміністратор заповнює файл docker-compose.yml, передаючи актуальні мережеві адреси та ключі авторизації через блок environment. Структуру маніфесту наведено на рисунку 5.2.



```
docker-compose.yml
You, 48 minutes ago | 1 author (You)
1  services:
2    moodle-sync-service:
3      build: .
4      container_name: moodle_sync_api
5      ports:
6        - '8081:8081'
7      environment:
8        - MOODLE_API_URL=https://moodle.kpi.ua/webservice/rest/server.php
9        - MOODLE_API_TOKEN=your_moodle_secret_token
10       - APP_API_KEY=your_campus_integration_key
11     volumes:
12       - ./src/main/resources/students.txt:/src/main/resources/students.txt
```

Рисунок 5.2 – Вміст файлу docker-compose.yml

Розгортання системи в робочому середовищі ініціюється командою консолі: `docker-compose up -d --build`. Під час старту фреймворк Spring Boot зчитує передані змінні середовища і автоматично підставляє їх у внутрішній контекст програми, замінюючи значення за замовчуванням у файлі `application.properties`. Застосунок успішно ініціалізується на порту 8081 і переходить у режим очікування запитів.

Діаграма розгортання зображена на рисунку 5.3.

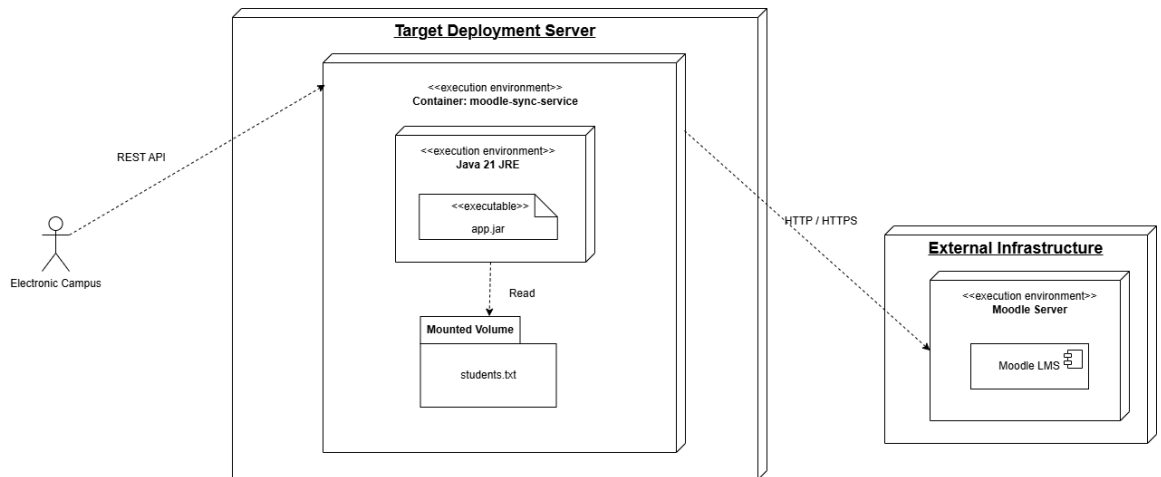


Рисунок 5.3 – Діаграма розгортання застосунку

5.2 Супровід програмного забезпечення

Інструкція системного адміністратора наведена в окремому документі “Керівництво системного адміністратора”.

Процес супроводу та підтримки розробленого програмного забезпечення базується на використанні системи контролю версій Git та технології контейнеризації Docker. Оскільки додаток розгортається у вигляді ізольованого Docker-контейнера, його оновлення відбувається швидко, безпечно і без впливу на роботу суміжних систем, зокрема основної платформи Moodle. Управління версіями вихідного коду здійснюється через репозиторій на платформі GitHub.

Життєвий цикл випуску нової версії починається з етапу локальної розробки, під час якого виконується виправлення помилок, оптимізація алгоритмів або додавання нового функціоналу. Після успішного мануального тестування та перевірки працездатності змін у локальному середовищі, розробник створює новий коміт та відправляє оновлений вихідний код у репозиторій.

Користувачі системи, які виконують роль системних адміністраторів, отримують нову версію безпосередньо на сервері. Для цього адміністратору достатньо підключитися до робочої станції та виконати команду системи

керування версіями для завантаження оновлень у директорії проєкту. Це дозволяє безпечно отримати актуальні змінені файли вихідного коду та конфігурації без ризику втрати локальних налаштувань. Після синхронізації файлів ініціюється процес оновлення самого сервісу за допомогою команди ініціалізації Docker Compose.

Завдяки налаштованій багатетапній збірці у Dockerfile, система автоматично виявляє отримані зміни, завантажує необхідні інструменти та компілює оновлений виконуваний файл безпосередньо в ізолюваному середовищі. Після успішної компіляції Docker зупиняє стару версію контейнера і запускає нову.

Висновки до розділу

У п'ятому розділі було розглянуто процеси розгортання та супроводу розробленого програмного забезпечення.

У підрозділі 5.1 описано покроковий процес створення ізолюваного середовища виконання за допомогою технології контейнеризації Docker та інструменту оркестрації Docker Compose.

У підрозділі 5.2 описано організацію процесу супроводу та оновлення застосунку. Визначено алгоритм доставки нових версій на базі системи контролю версій Git.

ВИСНОВКИ

В ході виконання дипломного проєкту було розроблено серверне проміжне програмне забезпечення, що реалізує автоматизовану синхронізацію відомостей успішності студентів між системою дистанційного навчання Moodle та інформаційною системою «Електронний кампус». Було проведено аналіз предметної області організації навчального процесу та існуючих на даний момент рішень інтеграції. В них було ідентифіковано ключові проблеми, а саме – висока частка ручної праці викладачів при перенесенні оцінок, дискретність оновлення інформації, а також архітектурна переважаність або недостатня безпека існуючих готових плагінів інтеграції баз даних.

Мети дипломного проєкту було досягнуто в повній мірі. Розроблений застосунок дозволяє автоматично агрегувати дані про курси, контрольні заходи та отримані студентами бали безпосередньо з вебсервісів Moodle. Система забезпечує фільтрацію масивів даних за конкретними викладачами та трансформацію багаторівневих відповідей у стандартизований формат, готовий для запису у відомості Кампусу.

Процес обробки даних покращено шляхом реалізації алгоритму трансформації за патерном ETL. Основою системи є механізм десеріалізації та агрегації JSON-структур в оперативній пам'яті сервера. Було реалізовано алгоритми пошуку ініціаторів оцінювання, групування балів за ідентифікаторами викладачів та санітизації текстових даних.

В результаті етапу конструювання програмного забезпечення було створено stateless-застосунок, що функціонує за принципами REST API та забезпечує захищений обмін даними за допомогою механізму токен-авторизації. Програмне забезпечення повністю відповідає поставленим функціональним та нефункціональним вимогам.

Застосунок було успішно протестовано на рівні якості вихідного коду та інтеграційної взаємодії. За допомогою статичного аналізу підтверджено відповідність архітектури стандартам чистого коду, високу надійність та

безпеку розроблених компонентів. Мануальне інтеграційне тестування довело коректність роботи всіх ендпоінтів та системи обробки помилок у різних сценаріях.

Продовження розробки у даній предметній області є цілком доцільним. Архітектура розробленого API є масштабованою і дозволяє легко підключати нові ендпоінти для розширення функціоналу інтеграції без критичних змін у коді. У перспективі дане програмне забезпечення може бути адаптоване для будь-яких інших освітніх чи корпоративних платформ, що потребують надійної інтеграції локальних баз даних із системою Moodle.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) What in the World is a Learning Management System? / EdTech Books [Електронний ресурс]. – Режим доступу: https://edtechbooks.org/learning_management_systems/introduction (дата звернення: 29.03.2026).
- 2) Moodle Docs [Електронний ресурс] / Moodle.org. – Режим доступу: <https://docs.moodle.org/> (дата звернення: 29.03.2026).
- 3) Electronic Campus of Igor Sikorsky Kyiv Polytechnic Institute [Електронний ресурс] / GitHub. – Режим доступу: <https://github.com/kpi-ua/ecampus.kpi.ua> (дата звернення: 29.03.2026).
- 4) Ellucian Intelligent Learning Platform [Електронний ресурс]. – Режим доступу: <https://www.ellucian.com/assets/en/solution-sheet/ellucian-intelligent-learning-platform.pdf> (дата звернення: 09.04.2026).
- 5) Zapier [Електронний ресурс]. – Режим доступу: <https://zapier.com/> (дата звернення: 10.04.2026).
- 6) Boehm B. W. Software Engineering Economics [Електронний ресурс] / Barry W. Boehm. – Режим доступу: <https://staff.emu.edu.tr/alexanderchefranov/Documents/CMPE412/Boehm1981%20COCOMO.pdf> (дата звернення: 10.05.2026).
- 7) Звіт про заробітні плати ІТ-спеціалістів України [Електронний ресурс] / Портал Dou. – Режим доступу: <https://dou.ua/lenta/articles/salary-report-devs-winter-2026/> (дата звернення: 10.05.2026).
- 8) Express.js Documentation [Електронний ресурс] / OpenJS Foundation – Режим доступу: <https://expressjs.com/> (дата звернення: 11.05.2026).
- 9) FastAPI Documentation [Електронний ресурс] / Sebastián Ramírez – Режим доступу: <https://fastapi.tiangolo.com/> (дата звернення: 11.05.2026).
- 10) Spring Boot Documentation [Електронний ресурс] / VMWare, Inc. – Режим доступу: <https://docs.spring.io/spring-boot/index.html> (дата звернення: 11.05.2026).

- 11) SonarQube Documentation [Электронный ресурс] / SonarSource Sàrl — Режим доступа: <https://docs.sonarsource.com/sonarqube-server> (дата звернения 26.05.2026)
- 12) Postman Docs [Электронный ресурс] / Postman, Inc. — Режим доступа: <https://learning.postman.com/> (дата звернения 27.05.2026).
- 13) Docker Docs [Электронный ресурс] / Docker Inc. — Режим доступа: <https://docs.docker.com/> (дата звернения 28.05.2026)
- 14) Docker Compose [Электронный ресурс] / Docker Inc. — Режим доступа: <https://docs.docker.com/compose/> (дата звернения 28.05.2026)

ДОДАТКИ

ДОДАТОК А ЗВІТ ПОДІБНОСТІ



Документ прийнятий

Метадані

ДОКУМЕНТ

Заголовок

ІП-21_Вдовиченко_ПЗ

Автор

ІП-21_Вдовиченко

Науковий керівник / Експерт

Губський А.М.

ІД документу

334196033

ОРГАНІЗАЦІЯ

Назва організації

National Technical University of Ukraine Igor Sikorskyi Kyiv
Polytech Institute

підрозділ

ФІОТ, К-а інформатики та програмної інженерії

ЗВІТ

Дата звіту

6/5/2026

Дата редагування

6/6/2026

Обсяг знайдених подібностей

Коефіцієнт подібності визначає, який відсоток тексту по відношенню до загального обсягу тексту було знайдено в різних джерелах. Зверніть увагу, що високі значення коефіцієнта не автоматично означають плагіат. Звіт має аналізувати компетентна / уповноважена особа.



9090

Кількість слів

71713

Кількість символів

Тривога

У цьому розділі ви знайдете інформацію щодо текстових спотворень. Ці спотворення в тексті можуть говорити про МОЖЛИВІ маніпуляції в тексті. Спотворення в тексті можуть мати навмисний характер, але частіше характер технічних помилок при конвертації документа та його збереженні, тому ми рекомендуємо вам підходити до аналізу цього модуля відповідально. У разі виникнення запитань, просимо звертатися до нашої служби підтримки.

Заміна букв	Б	0
Інтервали	A→	0
Мікропробіли		0
Білі знаки		0
Парафрази (SmartMarks)	a	67

Джерела

Нижче наведений список джерел. В цьому списку є джерела із різних баз даних. Колір тексту означає в якому джерелі він був знайдений. Ці джерела і значення Коефіцієнту Подібності не відображають прямого плагіату. Необхідно відкрити кожне джерело і проаналізувати зміст і правильність оформлення джерела.

10 найдовших фраз		Колір тексту
#	НАЗВА ТА АДРЕСА ДЖЕРЕЛА URL (НАЗВА БАЗИ)	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)

1	ІП-12_Спаських_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	89 (0.98 %)
2	ІП-13_Лопоша_ПЗ 5/30/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	52 (0.57 %)
3	ІП-13_Ал_Хадам_ПЗ 6/5/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	43 (0.47 %)
4	ІП-13_Карамян_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	35 (0.39 %)
5	ІП-24_Болотюк_ПЗ 6/4/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	31 (0.34 %)
6	ІП-13_Шиманська_ПЗ 6/1/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	24 (0.26 %)
7	ІП-21_Симоненко_ПЗ 5/29/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	22 (0.24 %)
8	ІП-13_Дем'янчук_ПЗ 5/30/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	21 (0.23 %)
9	ІП-11_Фукс_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	20 (0.22 %)
10	Конкурсна робота_Парковка 4/17/2025 Kherson national technical university (Херсонський національний технічний університет)	20 (0.22 %)

Домашня база даних (11.61 %)



#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	ІП-24_Болотюк_ПЗ 6/4/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	120 (9) (1.32 %)
2	ІП-12_Спаських_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	89 (1) (0.98 %)

National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)		
3	ІП-21_Симоненко_ПЗ 5/29/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	83 (6) (0.91 %)
4	ІП-21_Поплавський_ПЗ 5/25/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	82 (6) (0.9 %)
5	ІП-13_Дем'янюк_ПЗ 5/30/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	76 (7) (0.84 %)
6	ІП-13_Карамян_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	69 (6) (0.76 %)
7	ІП-13_Ал_Хадам_ПЗ 6/5/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	63 (4) (0.69 %)
8	ІП-13_Лопоша_ПЗ 5/30/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	62 (3) (0.68 %)
9	ІП-23_Востріков_ПЗ 6/5/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	45 (4) (0.5 %)
10	ІП-12_Луговець_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	41 (3) (0.45 %)
11	ІП-11_Фукс_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	40 (3) (0.44 %)
12	ІП-12_Шоман_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	35 (4) (0.39 %)
13	ІП-13_Шиманська_ПЗ 6/1/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	30 (2) (0.33 %)
14	ІП-15_Рибалка_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	28 (3) (0.31 %)
15	ІП-23_Луговенко_ПЗ 6/3/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	26 (3) (0.29 %)

Національний технічний університет України Ігор Сікорського Київський політехнічний інститут (ФІОТ, К-а інформатики та програмної інженерії)		
16	ІП-12_Сімчук_ПЗ 6/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	21 (2) (0.23 %)
17	ІП-13_Дейнега_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	19 (2) (0.21 %)
18	ІП-11_Калашніков_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	16 (2) (0.18 %)
19	ІП-21_Коваль_ПЗ 6/3/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	15 (2) (0.17 %)
20	ІП-15_Шляхтун_ПЗ 5/30/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	14 (2) (0.15 %)
21	Подійно-орієнтована бібліотека для створення користувацьких інтерфейсів на основі власної мови розмітки. Подійно-орієнтована бібліотека для створення користувацьких інтерфейсів. 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	13 (1) (0.14 %)
22	ІП-22_Дзюба_ПЗ 6/4/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	12 (1) (0.13 %)
23	ІП-13_Головач_ПЗ 6/11/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	10 (2) (0.11 %)
24	Архітектурне рішення для файлового сховища 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	10 (1) (0.11 %)
25	ІП-13_Шевцова_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	7 (1) (0.08 %)
26	Графічний конструктор SQL-запитів для взаємодії з базами даних 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	7 (1) (0.08 %)
27	ІП-15_Кондрацька_ПЗ 5/30/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	6 (1) (0.07 %)
28	ІП-13_Недельчев_ПЗ 6/6/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	6 (1) (0.07 %)

Національний технічний університет України Ігор Сікорського Київський політехнічний інститут (ФІОТ, К-а інформатики та програмної інженерії)		
29	ІП-15_Шабанов_ПЗ 6/1/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	5 (1) (0.06 %)
30	Вебзастосунок для менеджменту RSS-підписок 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	5 (1) (0.06 %)
Програма обміну базами даних (0.22 %)		
#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
31	Конкурсна робота_Парковка 4/17/2025 Kherson national technical university (Херсонський національний технічний університет)	20 (1) (0.22 %)
Інтернет (1.3 %)		
#	ДЖЕРЕЛО URL	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
32	https://ela.kpi.ua/server/api/core/bitstreams/57b0733d-2cbf-4df7-b177-9bccae88c98/content	42 (3) (0.46 %)
33	https://ela.kpi.ua/items/e8f7ecda-fb83-464f-a966-6cf5dcb5b4fd	31 (4) (0.34 %)
34	https://ela.kpi.ua/bitstream/123456789/53537/1/Tsakielov_Shabanov_bakalavr.pdf	17 (1) (0.19 %)
35	https://elartu.tntu.edu.ua/bitstream/lib/44485/1/dyplom_Kostur_2023.pdf	15 (1) (0.17 %)
36	https://ela.kpi.ua/bitstreams/81c0ef93-56cc-4a4b-8ef5-4eb3ae62f7e2/download	13 (2) (0.14 %)
☒ Список прийнятих фрагментів		
#	ЗМІСТ	КІЛЬКІСТЬ ОДНАКОВИХ СЛІВ (ФРАГМЕНТІВ)

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**МЕТОДИ ТА ЗАСОБИ ІНТЕГРАЦІЇ СИСТЕМИ ЕЛЕКТРОННИЙ
КАМПУС З MOODLE**

Текст програми

КПІ.ПІ-2103.045430.03.12

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Андрій ГУБСЬКИЙ

Нормоконтроль:

_____ Ірина ВІТКОВСЬКА

Виконавець:

_____ Станіслав ВДОВИЧЕНКО

Київ – 2026

Посилання на репозиторій з повним текстом програмного коду
<https://github.com/xnquaint/moodle-campus-sync>

Файл MoodleService.java

Реалізація функціональної задачі агрегації та трансформації оцінок.

```
public ObjectNode getCampusJournal(int courseId) {
    String courseUrl = UriComponentsBuilder.fromUriString(moodleUrl)
        .queryParams(PARAM_WSTOKEN, token)
        .queryParams(PARAM_WSFUNC, "core_course_get_courses_by_field")
        .queryParams("field", "id")
        .queryParams("value", courseId)
        .queryParams(PARAM_FORMAT, VAL_JSON)
        .build().toUriString();

    JsonNode courseRoot = readJson(sendGetRequest(courseUrl));
    JsonNode courses = courseRoot.path("courses");

    String courseName = UNKNOWN_COURSE;
    String courseSummary = "";

    if (courses.isArray() && !courses.isEmpty()) {
        JsonNode cNode = courses.get(0);
        courseName = cNode.path(FIELD_FULLNAME).asText();
        courseSummary =
htmlSanitizer.sanitize(cNode.path(FIELD_SUMMARY).asText());
    }

    String gradesUrl = UriComponentsBuilder.fromUriString(moodleUrl)
        .queryParams(PARAM_WSTOKEN, token)
        .queryParams(PARAM_WSFUNC, WS_FUNC_GRADE_ITEMS)
        .queryParams(FIELD_COURSEID, courseId)
        .queryParams(PARAM_FORMAT, VAL_JSON)
        .build().toUriString();

    JsonNode userGrades =
readJson(sendGetRequest(gradesUrl)).path(FIELD_USERGRADES);
    Map<Integer, ObjectNode> columnsMap = new LinkedHashMap<>();
    Map<Integer, JsonNode> enrolledUsers = getEnrolledUsersMap(courseId);

    if (userGrades.isArray()) {
        for (JsonNode studentNode : userGrades) {
            processStudentForCampusJournal(studentNode, columnsMap,
enrolledUsers);
        }
    }

    ObjectNode result = objectMapper.createObjectNode();
    result.put("moodle_course_id", courseId);
    result.put("course_name", courseName);
    result.put("course_description", courseSummary);

    ArrayNode columnsArray = objectMapper.createArrayNode();
    columnsMap.values().forEach(columnsArray::add);
    result.set(OUT_COLUMNS, columnsArray);

    return result;
}

public ObjectNode getTeacherCentricJournals(int courseId) {
```

```

String courseUrl = UriComponentsBuilder.fromUriString(moodleUrl)
    .queryParams(PARAM_WSTOKEN, token)
    .queryParams(PARAM_WSFUNC, "core_course_get_courses_by_field")
    .queryParams("field", "id")
    .queryParams("value", courseId)
    .queryParams(PARAM_FORMAT, VAL_JSON)
    .build().toUriString();

JsonNode courseRoot = readJson(sendGetRequest(courseUrl));
JsonNode courses = courseRoot.path("courses");

String courseName = UNKNOWN_COURSE;
String courseShortname = "";
String courseDescription = "";

if (courses.isArray() && !courses.isEmpty()) {
    JsonNode courseData = courses.get(0);
    courseName =
courseData.path(FIELD_FULLNAME).asText(UNKNOWN_COURSE);
    courseShortname = courseData.path(FIELD_SHORTNAME).asText("");
    courseDescription =
htmlSanitizer.sanitize(courseData.path(FIELD_SUMMARY).asText(""));
}

String gradesUrl = UriComponentsBuilder.fromUriString(moodleUrl)
    .queryParams(PARAM_WSTOKEN, token)
    .queryParams(PARAM_WSFUNC, WS_FUNC_GRADE_ITEMS)
    .queryParams(FIELD_COURSEID, courseId)
    .queryParams(PARAM_FORMAT, VAL_JSON)
    .build().toUriString();

JsonNode userGrades =
readJson(sendGetRequest(gradesUrl)).path(FIELD_USERGRADES);
Map<Integer, JsonNode> enrolledUsers = getEnrolledUsersMap(courseId);

ObjectNode result = objectMapper.createObjectNode();
result.put(OUT_COURSE_ID, courseId);
result.put("course_name", courseName);
result.put("course_shortname", courseShortname);
result.put("course_description", courseDescription);

Map<Integer, ObjectNode> teacherJournalsMap = new HashMap<>();
Map<Integer, ObjectNode> unassignedColumnsMap = new HashMap<>();

if (userGrades.isArray()) {
    for (JsonNode studentNode : userGrades) {
        processStudentForTeacherCentric(studentNode, enrolledUsers,
teacherJournalsMap, unassignedColumnsMap);
    }
}

ArrayNode teacherArray = objectMapper.createArrayNode();
for (ObjectNode j : teacherJournalsMap.values()) {
    ObjectNode finalJournal = objectMapper.createObjectNode();
    finalJournal.put(OUT_TEACHER_ID, j.get(OUT_TEACHER_ID).asInt());
    finalJournal.put(OUT_TEACHER_NAME,
j.get(OUT_TEACHER_NAME).asText());
    finalJournal.set(FIELD_EMAIL, j.get(FIELD_EMAIL));
    ArrayNode colArray = objectMapper.createArrayNode();
    j.get(OUT_COLUMNS).fields().forEachRemaining(entry ->
colArray.add(entry.getValue()));
    finalJournal.set(OUT_COLUMNS, colArray);
    teacherArray.add(finalJournal);
}

```



```

        result.set("teacher_journals", teacherArray);

        ArrayNode unassignedArray = objectMapper.createArrayNode();
        unassignedColumnsMap.values().forEach(unassignedArray::add);
        result.set("unassigned_activities", unassignedArray);

        return result;
    }

    private void processStudentForTeacherCentric(JsonNode studentNode,
        Map<Integer, JsonNode> enrolledUsers,
        Map<Integer, ObjectNode> teacherJournalsMap,
        Map<Integer, ObjectNode> unassignedColumnsMap) {
        int studentId = studentNode.path(FIELD_USERID).asInt();
        String studentName = studentNode.path("userfullname").asText();

        JsonNode userNode = enrolledUsers.get(studentId);
        String groupName = getGroupName(userNode);
        String studentEmail = (userNode != null) ?
            userNode.path(FIELD_EMAIL).asText(null) : null;

        for (JsonNode item : studentNode.path(FIELD_GRADEITEMS)) {
            if (VAL_MOD.equals(item.path(FIELD_ITEMTYPE).asText()) &&
                !item.path(FIELD_GRADERAW).isNull()) {

                Integer graderId =
                    VAL_ASSIGN.equals(item.path(FIELD_ITEMMODULE).asText()) ?
                        getGraderFromAssignment(item.path(FIELD_ITEMMODULE).asText(), studentId) : null;

                if (graderId != null && graderId > 0) {
                    ObjectNode journal =
                        teacherJournalsMap.computeIfAbsent(graderId, id ->
                            createTeacherJournalNode(id, enrolledUsers));
                    addMarkToTeacherColumnNode((ObjectNode)
                        journal.get(OUT_COLUMNS), item.path(FIELD_ID).asInt(), item, studentId,
                        studentName, groupName, studentEmail);
                } else {
                    addMarkToTeacherColumnMap(unassignedColumnsMap,
                        item.path(FIELD_ID).asInt(), item, studentId, studentName, groupName,
                        studentEmail);
                }
            }
        }
    }

    private ObjectNode createCampusMarkNode(JsonNode item, int studentId, String
        studentName, Map<Integer, JsonNode> enrolledUsers) {
        ObjectNode markNode = objectMapper.createObjectNode();
        markNode.put("moodle_student_id", studentId);
        markNode.put("student_name", studentName);

        // Логіка користувача
        JsonNode userNode = enrolledUsers.get(studentId);
        markNode.put("student_group", getGroupName(userNode));
        markNode.put(FIELD_EMAIL, userNode != null ?
            userNode.path(FIELD_EMAIL).asText(null) : null);

        // Логіка оцінки
        markNode.put("score", item.path(FIELD_GRADERAW).asDouble());
        markNode.put(OUT_COMMENT, item.hasNonNull(FIELD_FEEDBACK) ?
            htmlSanitizer.sanitize(item.path(FIELD_FEEDBACK).asText()) : null);
    }

```

```

        markNode.put(OUT_DATE_GRADED,
formatUnixDate(item.path(FIELD_DATEGRADED).asLong(0)));

        // Логіка вчителя
        if (VAL_ASSIGN.equals(item.path(FIELD_ITEMMODULE).asText())) {
            Integer graderId =
getGraderFromAssignment(item.path(FIELD_ITEMINSTANCE).asInt(), studentId);
            appendTeacherData(markNode, graderId, enrolledUsers);
        }
        return markNode;
    }

private ObjectNode createTeacherCentricMarkNode(JsonNode item, int studentId,
String studentName, String groupName, String studentEmail) {
    ObjectNode mark = objectMapper.createObjectNode();
    mark.put("student_id", studentId);
    mark.put("student_name", studentName);
    mark.put("student_group", groupName);
    mark.put(FIELD_EMAIL, studentEmail);
    mark.put("score", item.path(FIELD_GRADERAW).asDouble());

    if (item.hasNonNull(FIELD_FEEDBACK) &&
!item.path(FIELD_FEEDBACK).asText().isBlank()) {
        mark.put(OUT_COMMENT,
htmlSanitizer.sanitize(item.path(FIELD_FEEDBACK).asText()));
    } else {
        mark.putNull(OUT_COMMENT);
    }

    String formattedDate =
formatUnixDate(item.path(FIELD_DATEGRADED).asLong(0));
    if (formattedDate != null) {
        mark.put(OUT_DATE_GRADED, formattedDate);
    } else {
        mark.putNull(OUT_DATE_GRADED);
    }
    return mark;
}

```

Реалізація функціональної задачі пошуку та фільтрації оцінок всередині системи Moodle.

```

public Integer getGraderFromAssignment(int assignmentId, int studentId) {
    String url = UriComponentsBuilder.fromUriString(moodleUrl)
        .queryParams(PARAM_WSTOKEN, token)
        .queryParams(PARAM_WSFUNC, "mod_assign_get_grades")
        .queryParams(PARAM_FORMAT, VAL_JSON)
        .queryParams("assignmentids[0]", assignmentId)
        .build().toUriString();

    JsonNode root = readJson(sendGetRequest(url));
    JsonNode assignments = root.path("assignments");

    if (assignments.isArray() && !assignments.isEmpty()) {
        JsonNode grades = assignments.get(0).path("grades");
        if (grades.isArray()) {
            for (JsonNode grade : grades) {
                if (grade.path(FIELD_USERID).asInt() == studentId) {
                    return grade.path("grader").asInt();
                }
            }
        }
    }
}

```

```

        }
    }
    return null;
}

public Map<Integer, JsonNode> getEnrolledUsersMap(int courseId) {
    String url = UriComponentsBuilder.fromUriString(moodleUrl)
        .queryParams(PARAM_WSTOKEN, token)
        .queryParams(PARAM_WSFUNC, WS_FUNC_ENROLLED_USERS)
        .queryParams(FIELD_COURSEID, courseId)
        .queryParams(PARAM_FORMAT, VAL_JSON)
        .build().toUriString();

    JsonNode allUsers = readJson(sendGetRequest(url));
    Map<Integer, JsonNode> userMap = new HashMap<>();

    if (allUsers.isArray()) {
        for (JsonNode user : allUsers) {
            userMap.put(user.path(FIELD_ID).asInt(), user);
        }
    }
    return userMap;
}

public ObjectNode getCourseTeachers(int courseId) {
    String url = UriComponentsBuilder.fromUriString(moodleUrl)
        .queryParams(PARAM_WSTOKEN, token)
        .queryParams(PARAM_WSFUNC, WS_FUNC_ENROLLED_USERS)
        .queryParams(FIELD_COURSEID, courseId)
        .queryParams(PARAM_FORMAT, VAL_JSON)
        .build().toUriString();

    JsonNode allUsers = readJson(sendGetRequest(url));
    ObjectNode result = objectMapper.createObjectNode();
    ArrayNode teachersArray = objectMapper.createArrayNode();

    if (allUsers.isArray()) {
        for (JsonNode user : allUsers) {
            boolean isTeacher = false;
            for (JsonNode role : user.path("roles")) {
                if
(role.path(FIELD_SHORTNAME).asText().toLowerCase().contains("teacher")) {
                    isTeacher = true;
                    break;
                }
            }
            if (isTeacher) {
                ObjectNode teacherInfo = objectMapper.createObjectNode();
                teacherInfo.put(FIELD_ID, user.path(FIELD_ID).asInt());
                teacherInfo.put(FIELD_FIRSTNAME,
user.path(FIELD_FIRSTNAME).asText());
                teacherInfo.put(FIELD_LASTNAME,
user.path(FIELD_LASTNAME).asText());
                teacherInfo.put("role_name",
user.path("roles").get(0).path(FIELD_SHORTNAME).asText());
                teacherInfo.put(FIELD_EMAIL,
user.path(FIELD_EMAIL).asText(null));
                teachersArray.add(teacherInfo);
            }
        }
    }
    result.put(OUT_COURSE_ID, courseId);
    result.set("teachers", teachersArray);
}

```

```
    return result;  
}
```

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**МЕТОДИ ТА ЗАСОБИ ІНТЕГРАЦІЇ СИСТЕМИ ЕЛЕКТРОННИЙ
КАМПУС З MOODLE**

Програма та методика тестування

КПІ.ПІ-2103.045430.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Андрій ГУБСЬКИЙ

Нормоконтроль:

_____ Ірина ВІТКОВСЬКА

Виконавець:

_____ Станіслав ВДОВИЧЕНКО

Київ – 2026

ЗМІСТ

1	ОБ'ЄКТ ВИПРОБУВАНЬ.....	3
2	МЕТА ТЕСТУВАННЯ	4
3	МЕТОДИ ТЕСТУВАННЯ.....	5
4	ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ.....	6

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є проміжне програмне забезпечення для автоматизованої інтеграції систем Електронний кампус та Moodle.

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка правильності роботи програмного забезпечення відповідно до функціональних вимог;
- перевірка збереження даних;
- перевірка сумісності застосунку з різними операційними системами (Windows, Linux);
- знаходження проблем, помилок і недоліків з метою їх усунення.

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування – перевіряється програма разом з усією документацією, яка аналізується на предмет дотримання стандартів програмування;
- динамічне тестування – застосовується в процесі виконання програми; коректність програмного засобу перевіряється на певній кількості тестів; при прогоні кожного з них збираються та аналізуються дані про проблеми та помилки в роботі програми;
- функціональне тестування – полягає у перевірці відповідності реальної поведінки програмного забезпечення очікуваній;
- системне тестування – перевіряється усе програмне забезпечення в цілому;
- мануальне тестування – тестування без використання автоматизації, тест-кейси пише особа, що тестує програмне забезпечення.

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Під час проведення тестування будуть використовуватись наступні допоміжні засоби:

- середовище для розробки та тестування API – Postman;
- платформа статичного аналізу коду – SonarQube;
- засоби контейнеризації та розгортання – Docker та Docker Compose.

Порядок проведення тестування буде наступним:

- статичне тестування вихідного коду за допомогою платформи SonarQube та аналіз супровідної програмної документації на предмет дотримання стандартів чистого коду та відсутності критичних вразливостей;
- розгортання розробленого модуля інтеграції в ізольованому середовищі (Docker-контейнері) для переходу до етапів виконання;
- динамічне тестування застосунку в процесі його виконання; здійснюється прогін програми на певному наборі тестових запитів до Moodle API зі збором та аналізом даних;
- мануальне тестування кінцевих точок REST API за допомогою середовища Postman;
- функціональне тестування алгоритмів агрегації та санітизації даних для перевірки того, чи відповідає реальна структура сформованих JSON-журналів та поведінка системи очікуваним функціональним вимогам;
- системне тестування, під час якого усе програмне забезпечення перевіряється в цілому як єдиний готовий до роботи механізм.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**МЕТОДИ ТА ЗАСОБИ ІНТЕГРАЦІЇ СИСТЕМИ ЕЛЕКТРОННИЙ
КАМПУС З MOODLE
Керівництво користувача
КПІ.ПІ-2103.045430.05.34**

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Андрій ГУБСЬКИЙ

Нормоконтроль:

_____ Ірина ВІТКОВСЬКА

Виконавець:

_____ Станіслав ВДОВИЧЕНКО

Київ – 2026

ЗМІСТ

1	ПРИЗНАЧЕННЯ ПРОГРАМИ	3
2	ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ....	4
2.1	Системні вимоги для коректної роботи.....	4
2.2	Завантаження застосунку	4
2.3	Перевірка коректної роботи.....	4
3	ВИКОНАННЯ ПРОГРАМИ	6

1 ПРИЗНАЧЕННЯ ПРОГРАМИ

Розроблене проміжне програмне забезпечення (Middleware API) призначене для автоматизованої синхронізації відомостей успішності студентів між системами Moodle та Електронний кампус.

Застосунок є серверним компонентом без графічного інтерфейсу користувача. Воно функціонує як інтеграційний шлюз, що приймає запити, звертається до Moodle Web Services, витягує масиви оцінок, здійснює їхню трансформацію та повертає результат у форматі JSON.

2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ

2.1 Системні вимоги для коректної роботи

Для успішної роботи даного застосунку необхідне виконання наступних вимог:

- наявність сервера або робочої станції з операційною системою сімейства Linux або Windows Server;
- наявність доступу до Інтернету;
- наявність встановленого рушія контейнеризації Docker та інструмент оркестрації Docker Compose.

2.2 Завантаження застосунку

Вихідний код програмного забезпечення, конфігураційні файли збірки та маніфести для контейнеризації зберігаються у репозиторії на платформі GitHub. Для завантаження застосунку користувачу (системному адміністратору) необхідно виконати команду копіювання репозиторію у робочу директорію на цільовому сервері.

Після отримання файлів необхідно виконати базове налаштування середовища та ініціалізувати запуск. Для цього у конфігураційному файлі `docker-compose.yml` треба внести актуальні значення для змінних середовища: адресу зовнішнього сервера (`MOODLE_API_URL`), токен доступу до вебсервісів (`MOODLE_API_TOKEN`) та встановити власний секретний ключ авторизації (`APP_API_KEY`), який буде використовуватись для захисту API. Після цього ініціювати процес збірки та запуску ізольованого контейнера за допомогою команди оркестратора: `docker-compose up -d --build`.

2.3 Перевірка коректної роботи

Перевірка коректної роботи застосунку здійснюється після його збірки та ініціалізації у Docker-середовищі. Оскільки ПЗ не має графічного інтерфейсу, для верифікації стану системи необхідно перевірити системний

вивід контейнера за допомогою перегляду логів. Відсутність повідомлень про критичні збої та наявність запису про успішний старт вебсервера на порту 8081 свідчить про вдалу ініціалізацію.

3 ВИКОНАННЯ ПРОГРАМИ

Користувач повинен мати дійсний сервер Moodle, на якому є створені курси, контрольні заходи, на курси зареєстровані викладачів та студенти, студенти мають оцінки за завдання (рисунки 3.1 – 3.4).

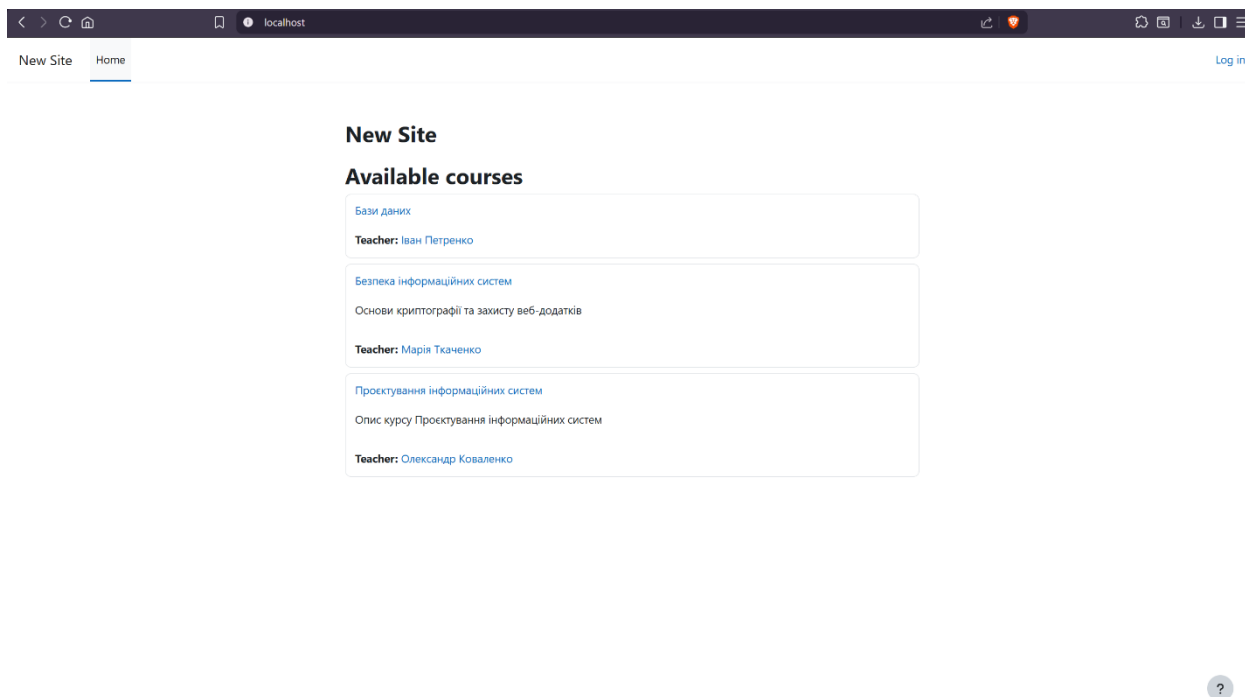


Рисунок 3.1 – Головна сторінка системи Moodle

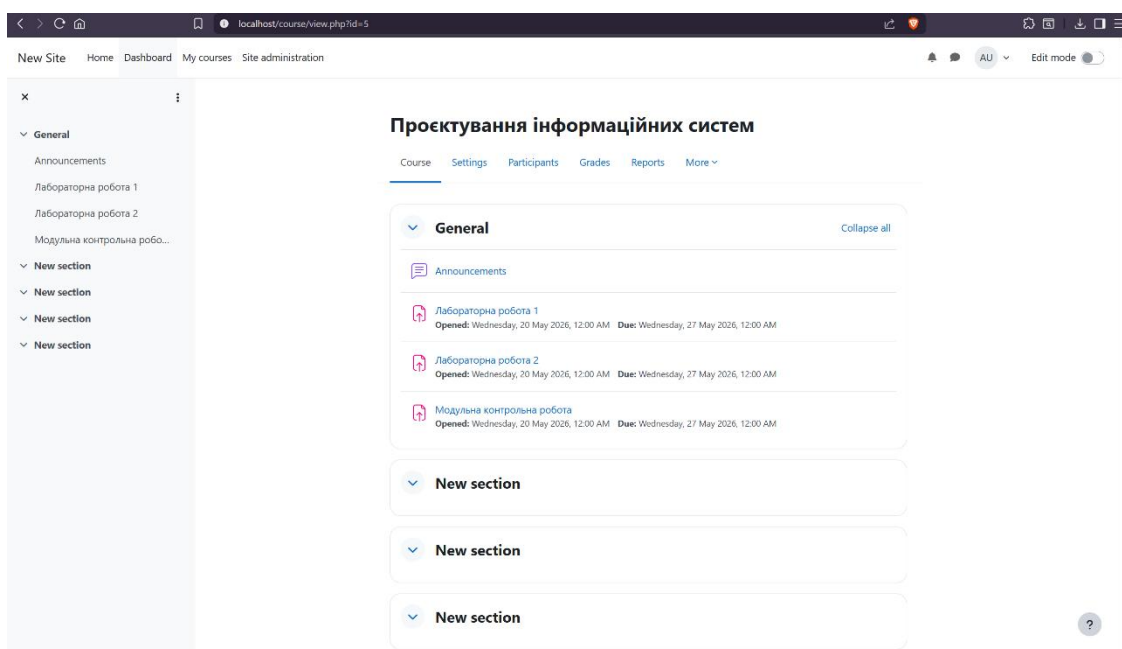


Рисунок 3.2 – Сторінка курсу

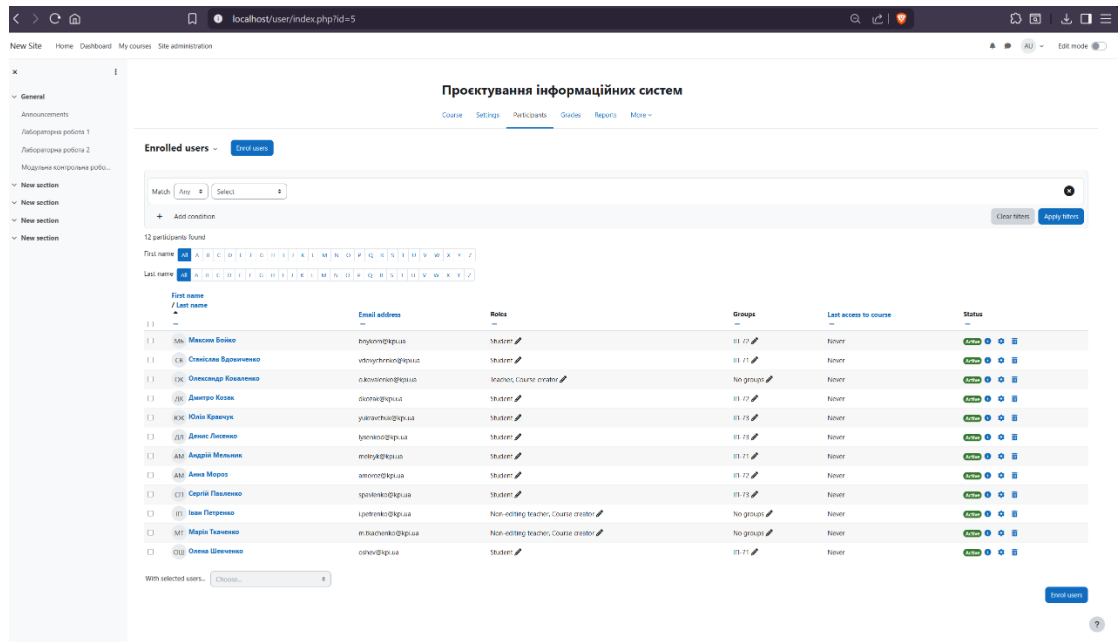


Рисунок 3.3 – Список викладачів та студентів курсу

First name / Last name	Email address	Лабораторна робота 1	Лабораторна робота 2	Модульна контрольна робота	Course total
МБ Максим Бойко	boykom@kpi.ua	0.00	55.00	67.00	122.00
СВ Станіслав Вдовиченко	vdovychenko@kpi.ua	99.00	30.00	20.00	149.00
ДК Дмитро Козак	dkozak@kpi.ua	11.00	88.00	22.00	121.00
ЮК Юлія Кравчук	yukravchuk@kpi.ua	23.00	20.00	23.00	66.00
ДЛ Денис Лисенко	lysenkod@kpi.ua	27.00	11.00	44.00	82.00
АМ Андрій Мельник	melnyk@kpi.ua	88.00	20.00	77.00	185.00
АМ Анна Мороз	amoroz@kpi.ua	11.00	11.00	100.00	122.00
СП Сергій Павленко	spavlenko@kpi.ua	22.00	5.00	12.00	39.00
ОШ Олена Шевченко	oshev@kpi.ua	100.00	100.00	100.00	300.00
Overall average		42.33	37.78	51.67	131.78

Рисунок 3.4 – Список оцінок студентів

Користувач має змогу отримати список курсів за ендпоінтом /courses (рисунок 3.5).

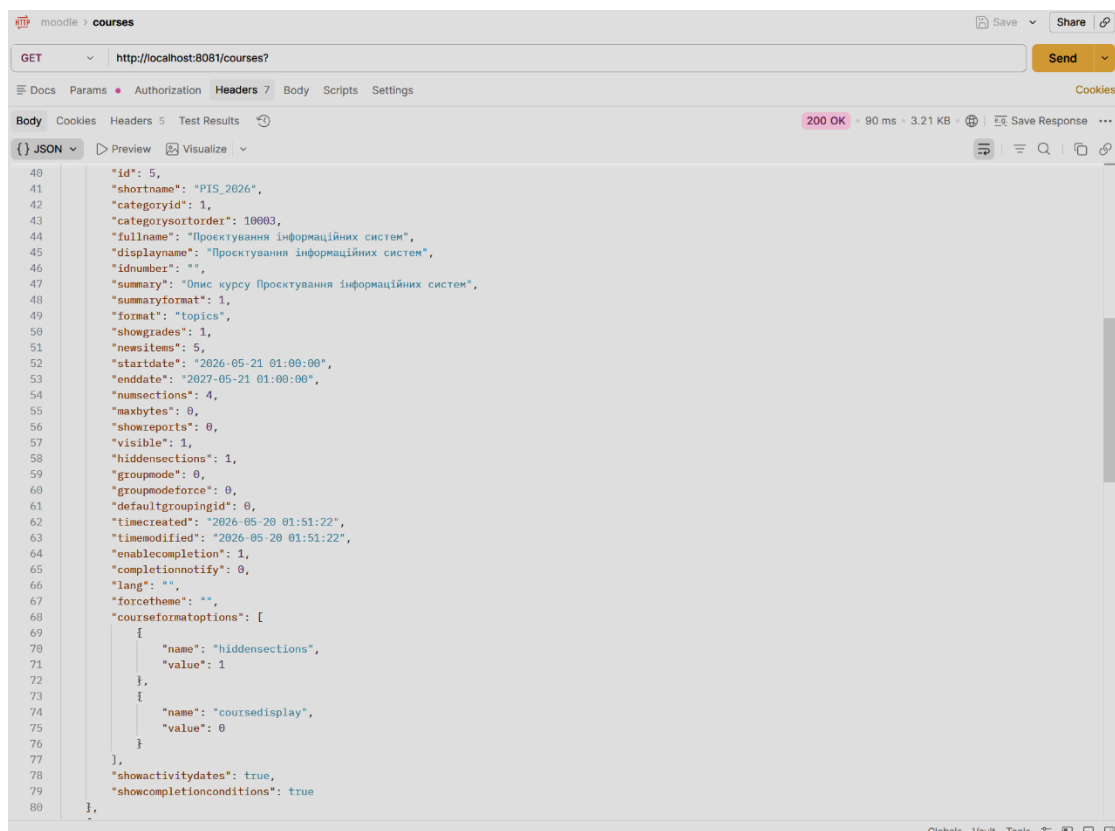


Рисунок 3.5 – Отриманий список курсів

Користувач має змогу отримати список викладачів курсу (для прикладу `id курсу = 5`) за ендпоінтом `/course/5/teachers` (рисунок 3.6).

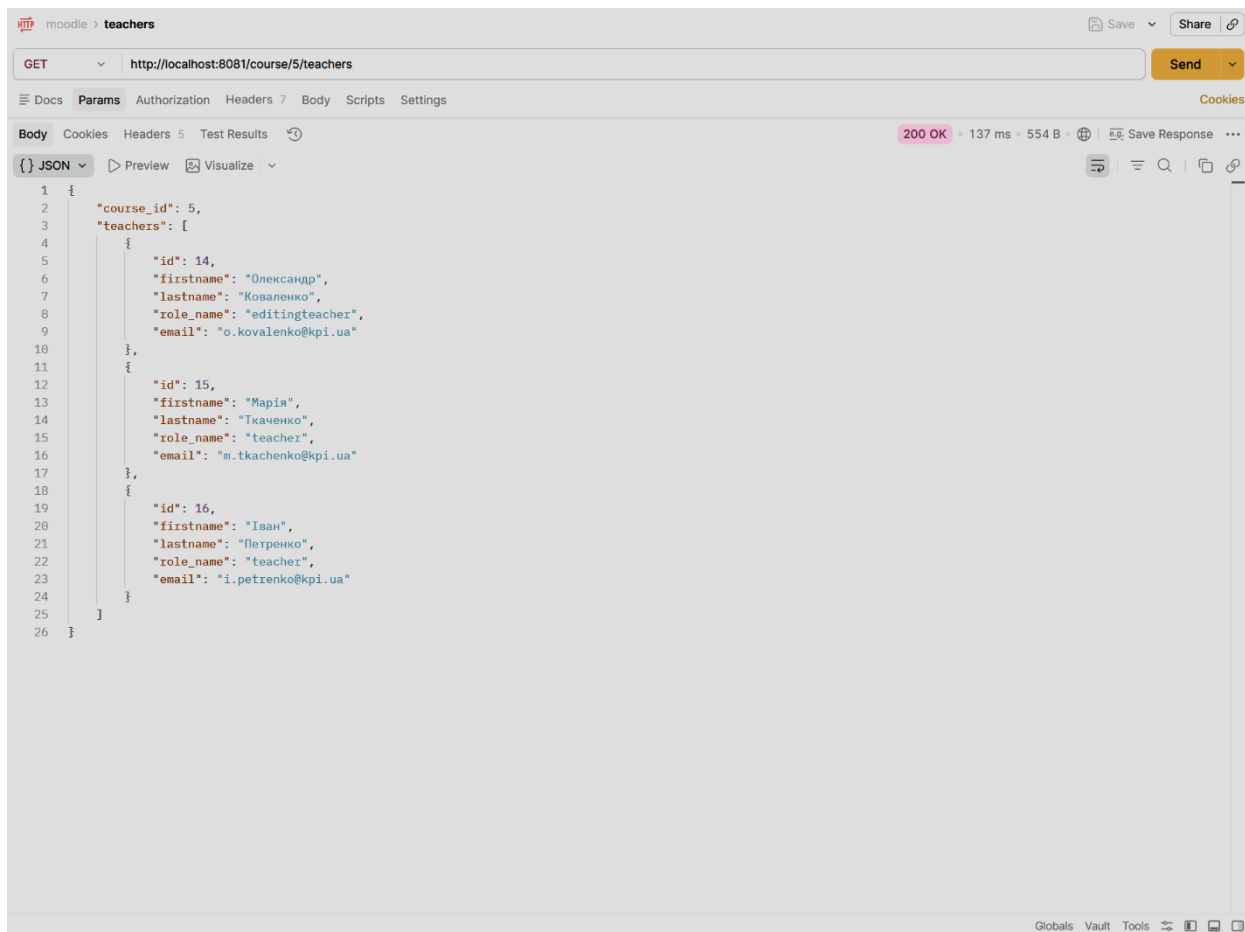


Рисунок 3.6 – Отриманий список викладачів

Користувач має змогу отримати список контрольних заходів курсу за ендпоінтом `/course/5/activities` (рисунок 3.7).

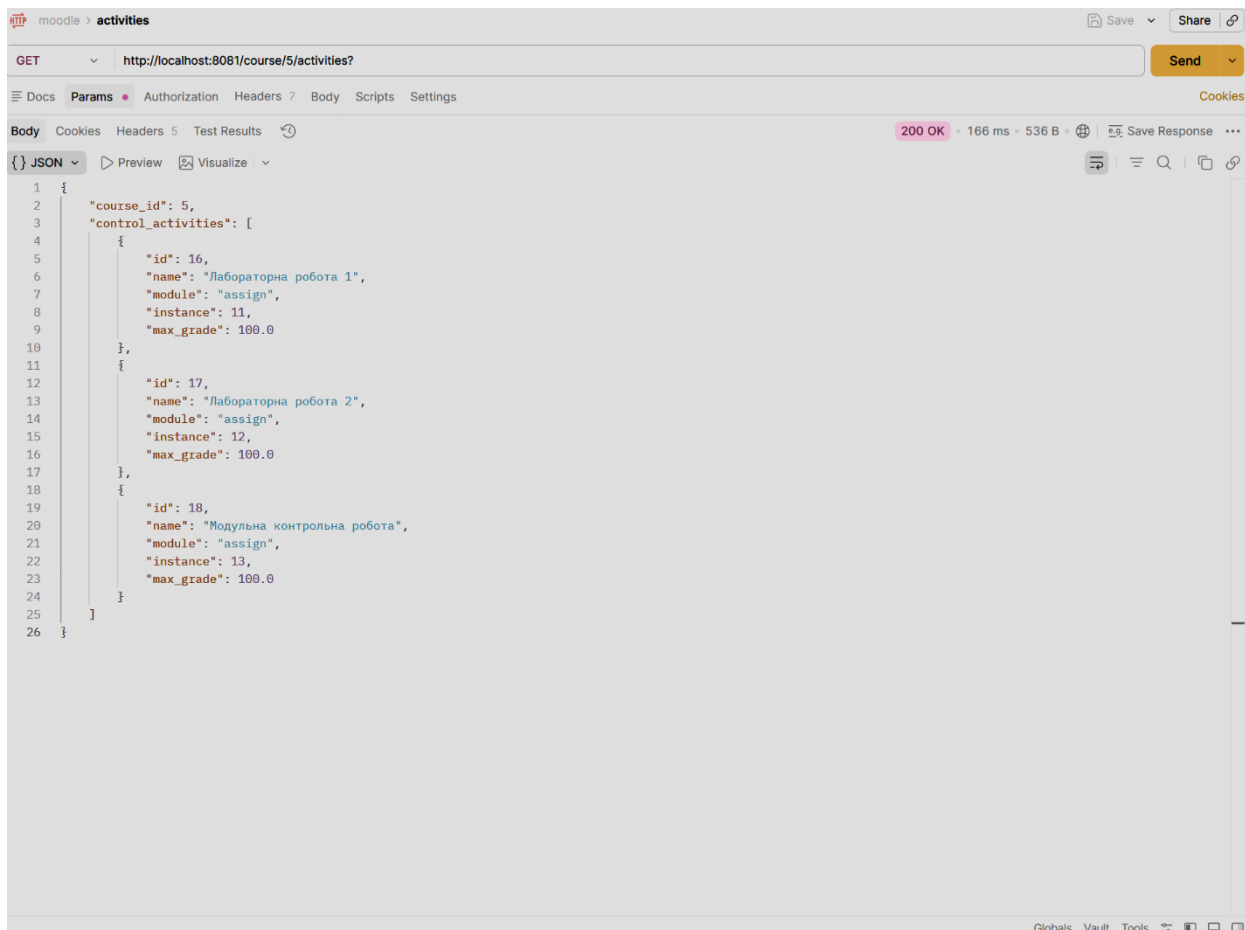


Рисунок 3.7 – Отриманий список контрольних заходів курсу

Користувач має змогу отримати журнал оцінок відсортований за контрольними заходами за ендпоінтом `/course/5/journal` (рисунок 3.8).

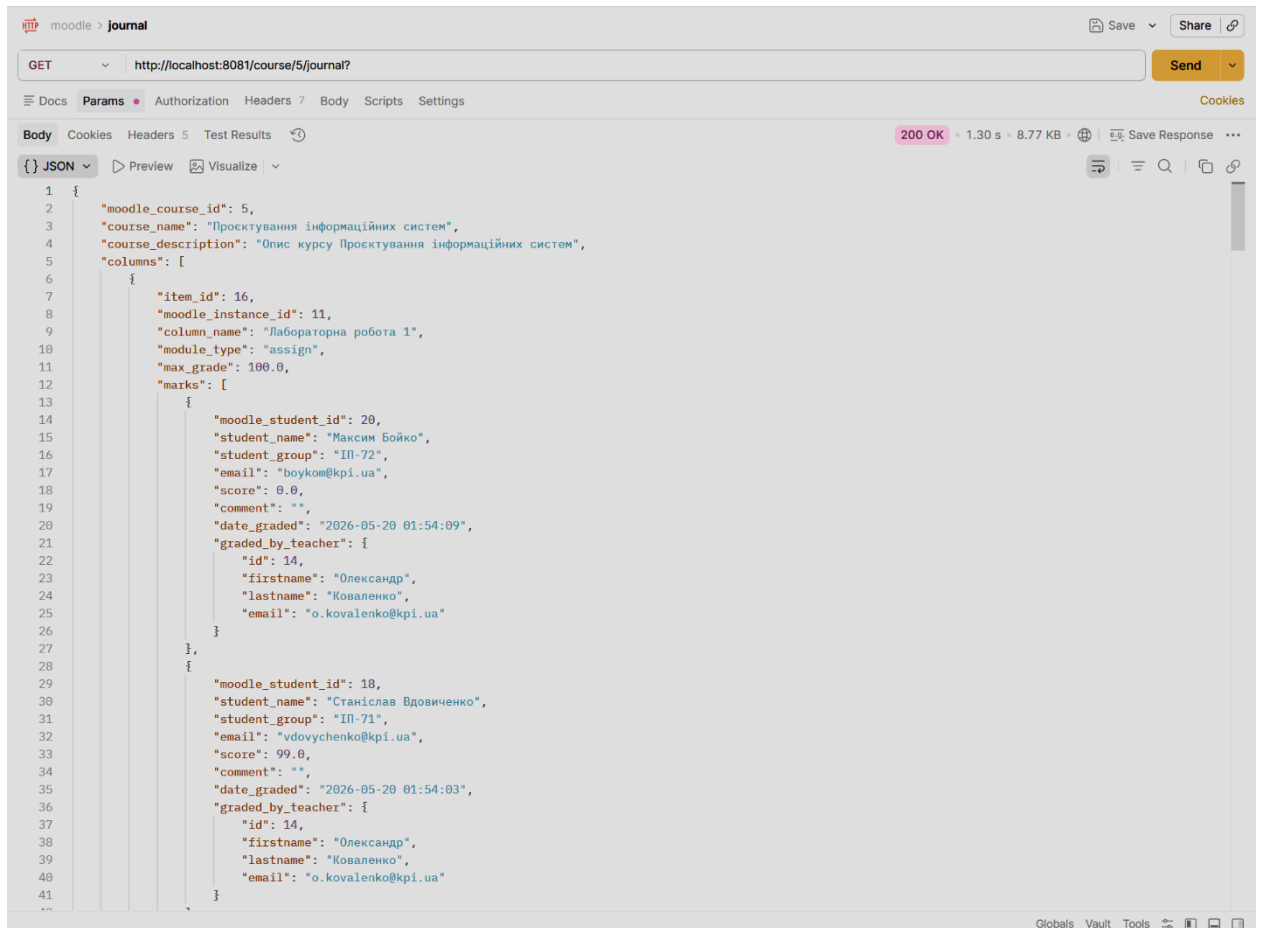


Рисунок 3.8 – Отриманий журнал оцінок, відсортований за контрольними заходами

Користувач має змогу отримати журнал відсортований за викладачами за ендпоінтом `/course/5/export-campus` (рисунок 3.9).

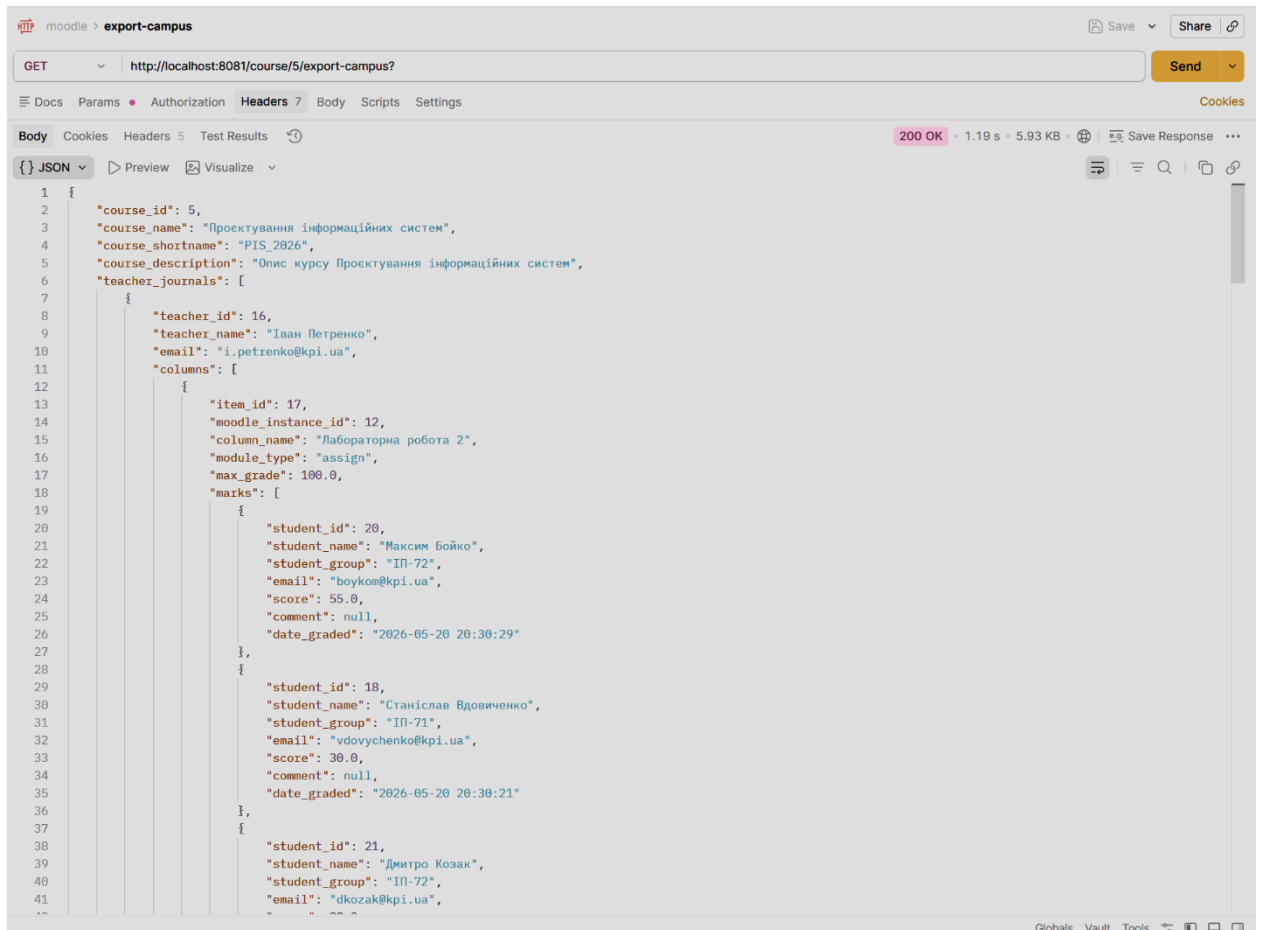


Рисунок 3.9 – Отриманий журнал оцінок, відсортований за викладачами

Користувач має змогу отримати інформацію про оцінки конкретного студента, використовуючи пошук за електронною адресою (для цього використовується файл `students.txt`) за ендпоінтом `/export-students` (рисунок 3.10).

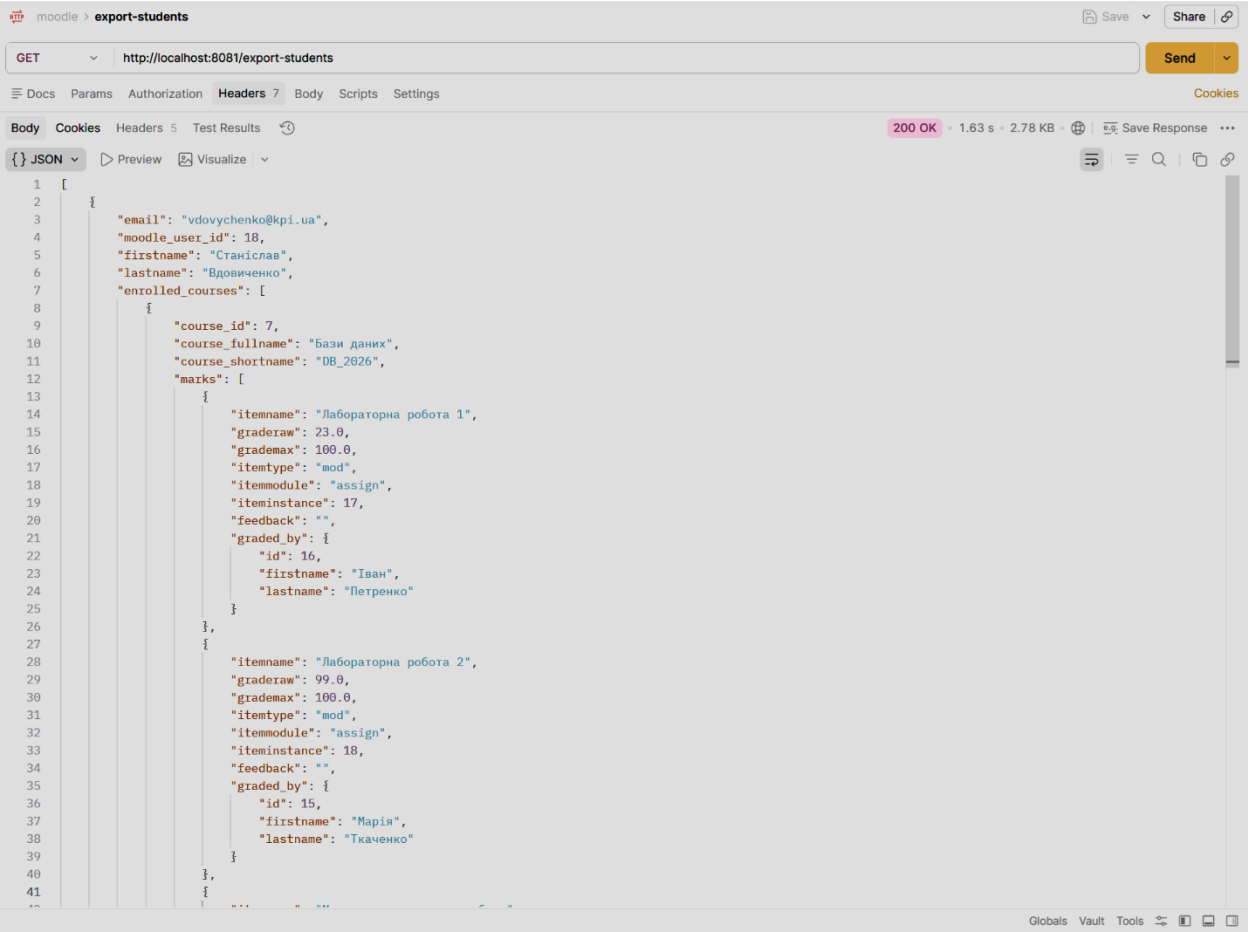


Рисунок 3.10 – Пошук студента за електронною адресою

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**МЕТОДИ ТА ЗАСОБИ ІНТЕГРАЦІЇ СИСТЕМИ ЕЛЕКТРОННИЙ
КАМПУС З MOODLE**

Керівництво системного програміста

КПІ.ПІ-2103.045430.06.32

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Андрій ГУБСЬКИЙ

Нормоконтроль:

_____ Ірина ВІТКОВСЬКА

Виконавець:

_____ Станіслав ВДОВИЧЕНКО

Київ – 2026

ЗМІСТ

1	ПРИЗНАЧЕННЯ ПРОГРАМИ	3
2	ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ....	4
2.1	Системні вимоги для коректної роботи.....	4
2.2	Завантаження застосунку	4
2.3	Перевірка коректної роботи.....	4
3	ВИКОНАННЯ ПРОГРАМИ	5

1 ПРИЗНАЧЕННЯ ПРОГРАМИ

Розроблене проміжне програмне забезпечення (Middleware API) призначене для автоматизованої синхронізації відомостей успішності студентів між системами Moodle та Електронний кампус.

Застосунок є серверним компонентом без графічного інтерфейсу користувача. Воно функціонує як інтеграційний шлюз, що приймає запити, звертається до Moodle Web Services, витягує масиви оцінок, здійснює їхню трансформацію та повертає результат у форматі JSON.

2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ

2.1 Системні вимоги для коректної роботи

Для успішної роботи даного застосунку необхідне виконання наступних вимог:

- наявність сервера або робочої станції з операційною системою сімейства Linux або Windows Server;
- наявність доступу до Інтернету;
- наявність встановленого рушія контейнеризації Docker та інструмент оркестрації Docker Compose.

2.2 Завантаження застосунку

Вихідний код програмного забезпечення, конфігураційні файли збірки та маніфести для контейнеризації зберігаються у репозиторії на платформі GitHub. Для завантаження застосунку користувачу (системному адміністратору) необхідно виконати команду копіювання репозиторію у робочу директорию на цільовому сервері.

2.3 Перевірка коректної роботи

Перевірка коректної роботи застосунку здійснюється після його збірки та ініціалізації у Docker-середовищі. Оскільки ПЗ не має графічного інтерфейсу, для верифікації стану системи необхідно перевірити системний вивід контейнера за допомогою перегляду логів. Відсутність повідомлень про критичні збої та наявність запису про успішний старт вебсервера на порту 8081 свідчить про вдалу ініціалізацію.

3 ВИКОНАННЯ ПРОГРАМИ

Системний програміст повинен авторизуватись в системі Moodle за допомогою облікового запису з правами адміністратора. Далі треба увімкнути Moodle Web Services. Для цього програміст має перейти до розділу Site administration, обрати вкладку Advanced features та активувати опцію Enable web services (рисунок 3.1).

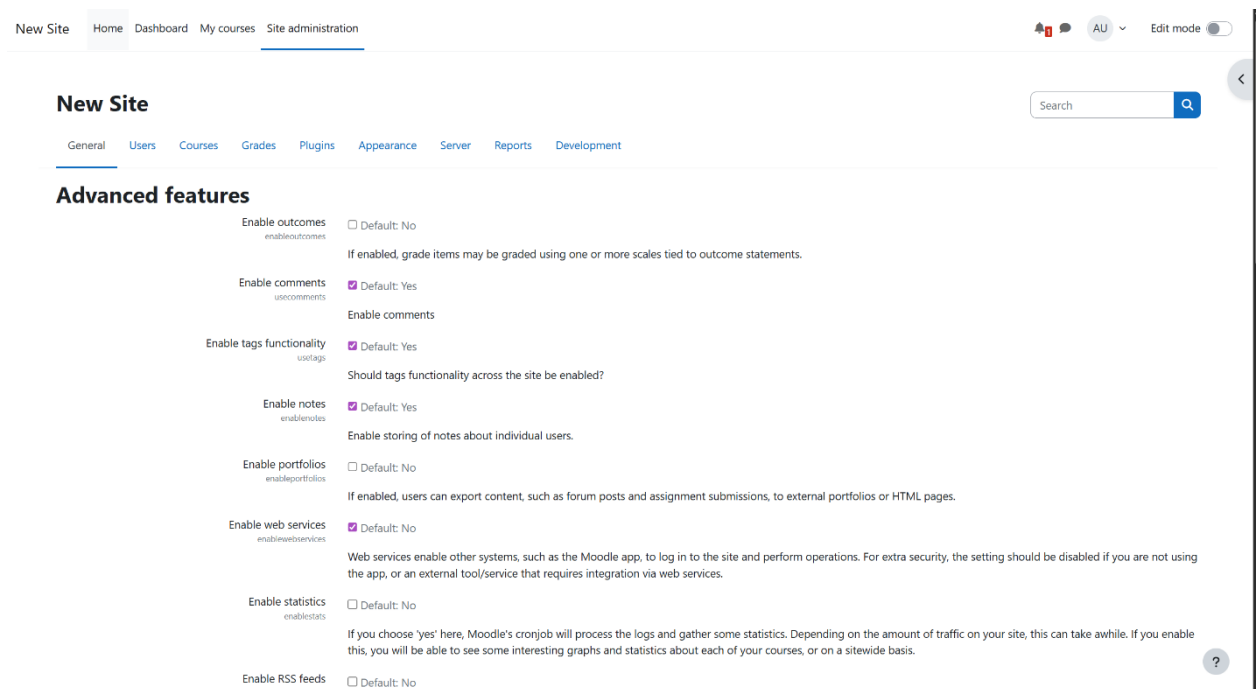


Рисунок 3.1 – Активація Moodle Web Services

Далі системний програміст має увімкнути REST протокол. Для цього треба перейти у Site administration, вкладку Server, розділ Web services, керування протоколами Manage protocols, натиснути на іконку “ока” біля REST protocol, щоб увімкнути його, та натиснути Save changes (рисунок 3.2).

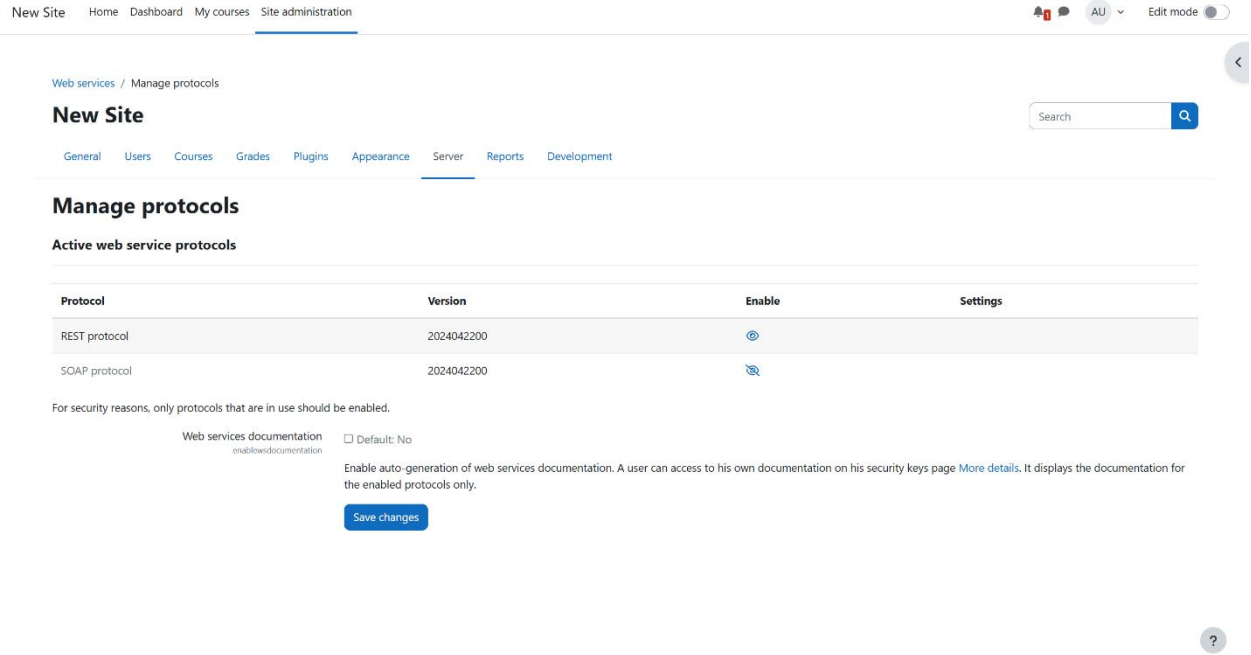


Рисунок 3.2 – Увімкнення REST протоколу

Далі системний програміст має створити External service для розробленого API. Для цього треба в розділі Web services натиснути External services, натиснути Add, дати назву сервісу, поставити галочку Enabled та натиснути Add service (рисунок 3.3).

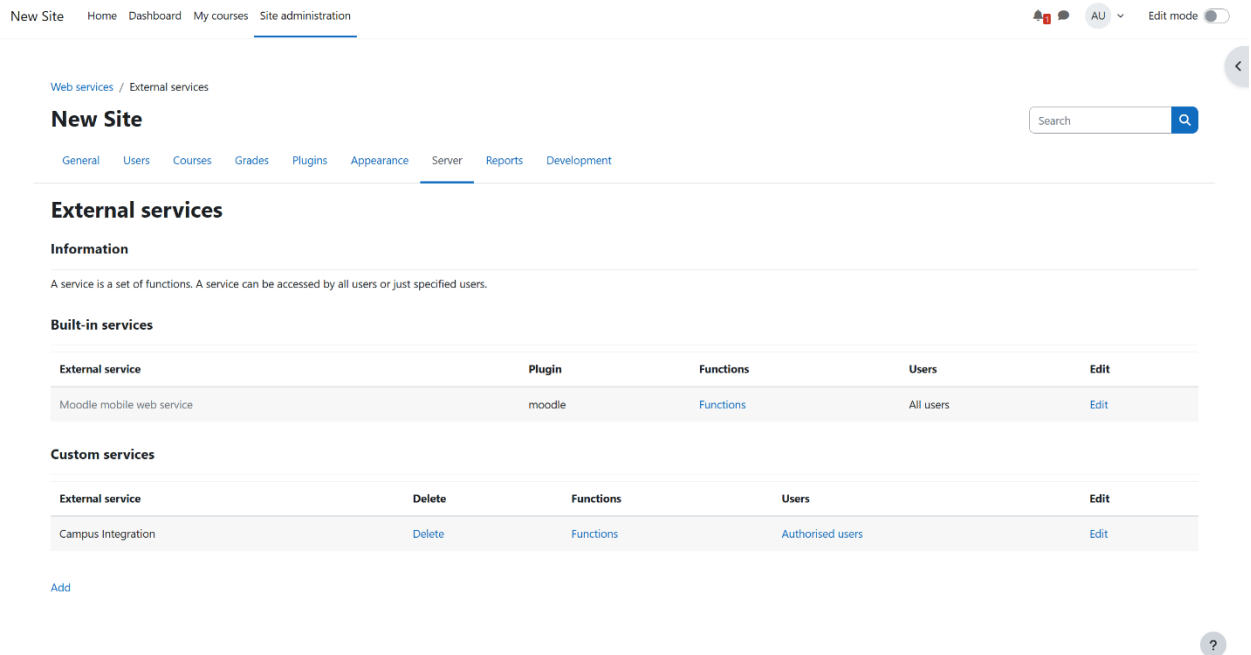


Рисунок 3.3 – Створення external service для API

Далі програміст має додати для сервісу користувача з правами на читання курсів, оцінок та студентів (рисунок 3.4).

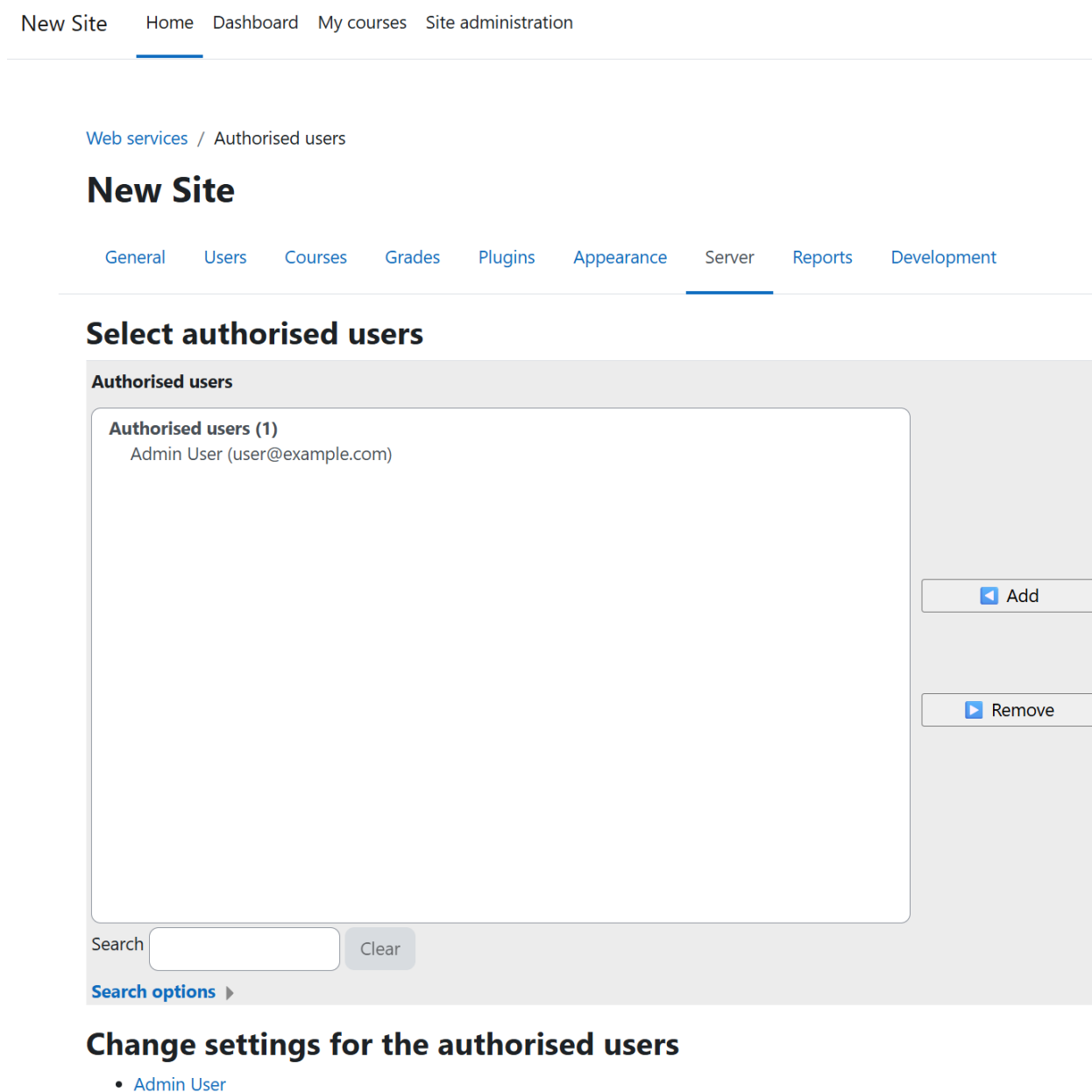


Рисунок 3.4 – Авторизований користувач для сервісу

Далі системний програміст має додати для сервісу функції, які використовує застосунок, а саме:

- `core_course_get_contents;`
- `core_course_get_courses;`
- `core_course_get_courses_by_field;`

- `core_enrol_get_enrolled_users;`
- `core_enrol_get_users_courses;`
- `core_user_get_users;`
- `gradereport_user_get_grade_items;`
- `mod_assign_get_grades.`

Правильно додані функції зображено на рисунку 3.5.

The screenshot shows the Moodle administration interface for adding functions to a service. The breadcrumb trail is 'Web services / External services / Functions'. The page title is 'New Site'. The 'Grades' tab is selected under the 'Server' category. The table lists functions to be added to the 'Campus Integration' service.

Function	Description	Required capabilities	Edit
<code>core_course_get_contents</code>	Get course contents	<code>moodle/course:update</code> , <code>moodle/course:viewhiddencourses</code>	Remove
<code>core_course_get_courses</code>	Return course details	<code>moodle/course:view</code> , <code>moodle/course:update</code> , <code>moodle/course:viewhiddencourses</code>	Remove
<code>core_course_get_courses_by_field</code>	Get courses matching a specific field (id/s, shortname, idnumber, category)		Remove
<code>core_enrol_get_enrolled_users</code>	Get enrolled users by course id.	<code>moodle/user:viewdetails</code> , <code>moodle/user:viewhiddendetails</code> , <code>moodle/course:useremail</code> , <code>moodle/user:update</code> , <code>moodle/site:accessallgroups</code>	Remove
<code>core_enrol_get_users_courses</code>	Get the list of courses where a user is enrolled in	<code>moodle/course:viewparticipants</code>	Remove
<code>core_user_get_users</code>	search for users matching the parameters	<code>moodle/user:viewdetails</code> , <code>moodle/user:viewhiddendetails</code> , <code>moodle/course:useremail</code> , <code>moodle/user:update</code>	Remove
<code>gradereport_user_get_grade_items</code>	Returns the complete list of grade items for users in a course	<code>gradereport/user:view</code>	Remove
<code>mod_assign_get_grades</code>	Returns grades from the assignment		Remove

[Add functions](#)

Рисунок 3.5 – Увімкнення функцій для сервісу

Далі системний програміст має згенерувати токен доступу. Для цього треба перейти у вкладку Web services та Manage tokens. Натиснути Create token, вибрати відповідного користувача (з правами на читання), вибрати створений сервіс, натиснути Save changes (рисунок 3.6).

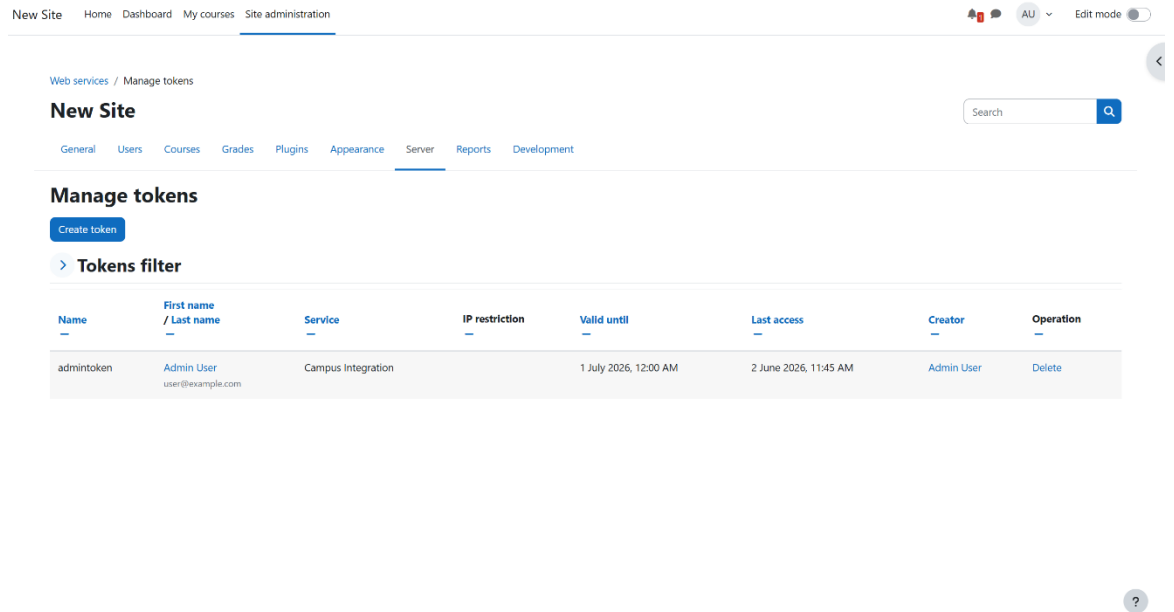


Рисунок 3.6 – Створений токен доступу

Після цього треба скопіювати токен доступу та використати його на етапі конфігурації Docker-контейнера або змінити значення змінних у коді програми.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**МЕТОДИ ТА ЗАСОБИ ІНТЕГРАЦІЇ СИСТЕМИ ЕЛЕКТРОННИЙ
КАМПУС З MOODLE**
Графічний матеріал
КПІ.ПІ-2103.045430.07.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Андрій ГУБСЬКИЙ

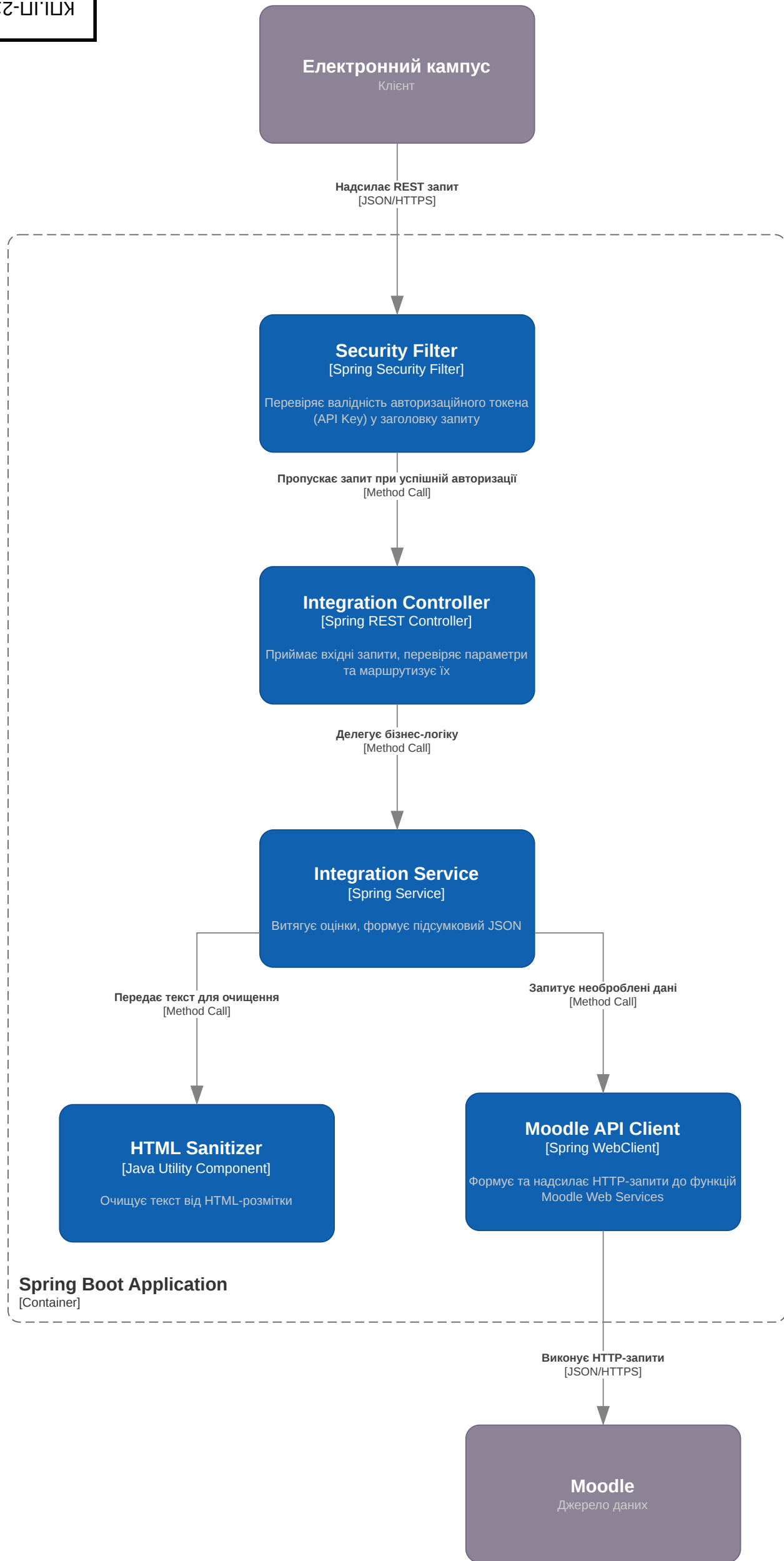
Нормоконтроль:

_____ Ірина ВІТКОВСЬКА

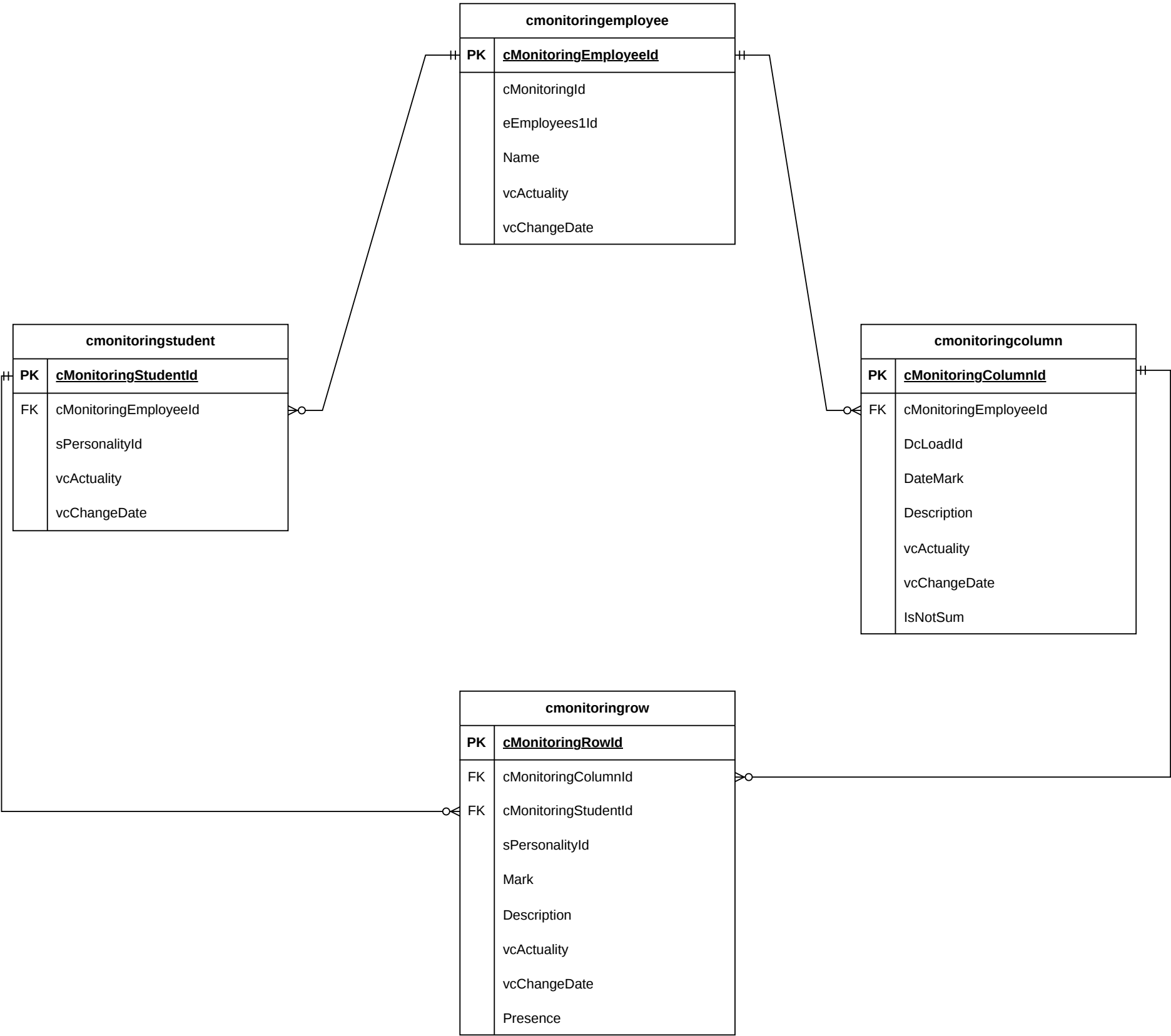
Виконавець:

_____ Станіслав ВДОВИЧЕНКО

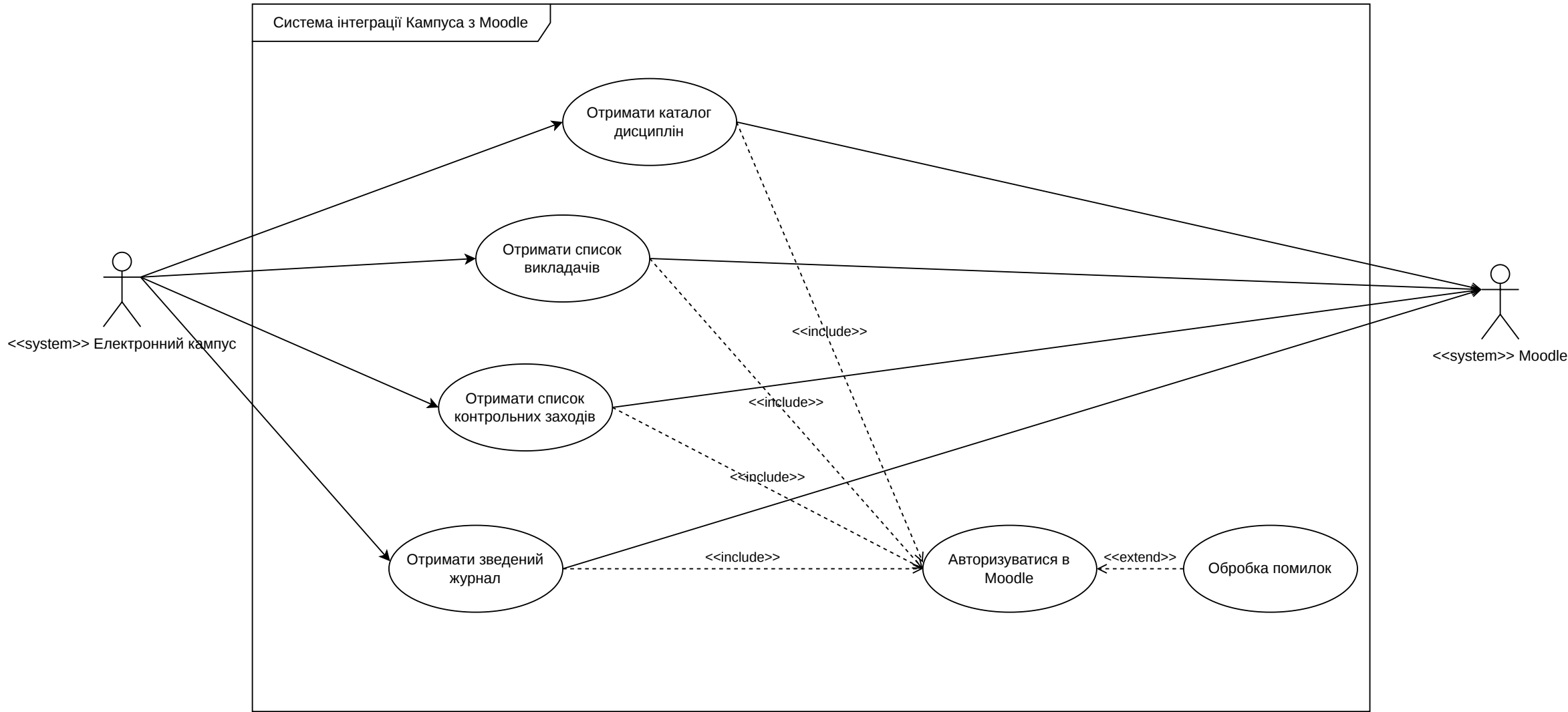
Київ – 2026



					КПІ.ІП-2103.045430.07.99.CCM						
					Схема структурна компонентів програмного забезпечення	Лит.		Маса		Масштаб	
Зм.	Арк.	№ докум.	Підп.	Дата							
Розробив		Вдовиченко С.Ю.									
Перевірів		Губський А.М.									
Т. контр.						Аркуш		Аркушів			
					Методи та засоби інтеграції системи Електронний кампус з Moodle	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-21					
Н. контр.		Вітковська І.І.									
Затвердив		Жаріков Е.В.									



					КПІ.ІП-2103.045430.07.99.СБД								
					Схема бази даних				Лит.		Маса	Масштаб	
Зм.	Арк.	№ докум.	Підп.	Дата									
Розробив		Вдовиченко С.Ю.											
Перевішив		Губський А.М.											
Т. контр.													
					Методи та засоби інтеграції системи Електронний кампус з Moodle				Аркуш		Аркушів		
									КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-21				
Н. контр.		Вітковська І.І.											
Затвердив		Жаріков Е.В.											



					КПІ.ІП-2103.045430.07.99.CCB						
					Схема структурна варіантів використань	Літера		Маса		Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата							
Розробив		Вдовиченко С.Ю.									
Перевірів		Губський А.М.									
Т. контр.											
					Методи та засоби інтеграції системи Електронний кампус з Moodle	Аркуш		Аркушів			
Н. контр.		Вітковська І.І.				КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-21					
Затвердив		Жаріков Е.В.									